

Encrypt Developer Guide

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Encrypt enables smart contracts to **compute on encrypted data** without ever decrypting it on-chain. Your program operates on ciphertexts — the actual values are never visible to validators, indexers, or anyone else.

How It Works

1. **You write FHE logic** using the `#[encrypt_fn]` DSL — it looks like normal Rust
2. **The macro compiles it** into a computation graph (a DAG of FHE operations)
3. **On-chain**, `execute_graph` creates output ciphertext accounts and emits events
4. **Off-chain**, the executor evaluates the graph using real FHE and commits results
5. **When needed**, you request decryption — the decryptor responds with plaintext

```
#[encrypt_fn]
fn transfer(from: EUint64, to: EUint64, amount: EUint64) -> (EUint64, EUint64)
{
    let has_funds = from >= amount;
    let new_from = if has_funds { from - amount } else { from };
    let new_to = if has_funds { to + amount } else { to };
    (new_from, new_to)
}
```

This compiles into an FHE computation graph that operates on encrypted balances. Nobody on-chain ever sees the actual amounts.

What You'll Learn

- **Getting Started:** Install dependencies, create your first encrypted program

- **Tutorial:** Build a complete confidential voting application step by step
- **DSL Reference:** All supported types, operations, and patterns
- **On-Chain Integration:** Ciphertext accounts, access control, graph execution, decryption
- **Framework Guides:** Pinocchio, Anchor, and Native examples
- **Testing:** Local test framework, CLI tools, mock vs real FHE
- **Reference:** Complete instruction, account, event, and fee documentation

Supported Frameworks

Encrypt works with all three major Solana program frameworks:

Framework	SDK Crate	Best For
Pinocchio	<code>encrypt-pinocchio</code>	Maximum CU efficiency, <code>#![no_std]</code> programs
Anchor	<code>encrypt-anchor</code>	Rapid development, declarative accounts
Native	<code>encrypt-native</code>	<code>solana-program</code> users, no framework lock-in

All three use the same `#[encrypt_fn]` DSL and the same `EncryptCpi` trait.

Installation

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Prerequisites

- **Rust** (edition 2024): `curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh`
- **Solana CLI** 3.x+: `sh -c "$(curl -sSfL https://release.anza.xyz/stable/install)"`
- **Bun** (for TypeScript clients): `curl -fsSL https://bun.sh/install | bash`

Add Dependencies

For Pinocchio Programs

[dependencies]

```
encrypt-types = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-dsl = { package = "encrypt-solana-dsl", git =
  "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-pinocchio = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha"
}
pinocchio = "0.10"
```

[dev-dependencies]

```
encrypt-solana-test = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
```

For Anchor Programs

```
[dependencies]
encrypt-types = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-dsl = { package = "encrypt-solana-dsl", git =
  "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-anchor = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
anchor-lang = "0.32"
```

```
[dev-dependencies]
encrypt-solana-test = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
```

For Native Programs

```
[dependencies]
encrypt-types = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-dsl = { package = "encrypt-solana-dsl", git =
  "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-native = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
solana-program = "4"
```

```
[dev-dependencies]
encrypt-solana-test = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
```

Client SDKs

Rust gRPC Client

```
[dependencies]
encrypt-solana-client = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
tokio = { version = "1", features = ["rt-multi-thread", "macros"] }
```

TypeScript gRPC Client

```
bun add @encrypt.xyz/pre-alpha-solana-client
```

Pre-Alpha Environment

The Encrypt program is deployed to **Solana devnet**. An executor is running at:

Resource	Endpoint
Encrypt gRPC	https://pre-alpha-dev-1.encrypt.ika-network.net:443
Solana RPC	https://api.devnet.solana.com
Program ID	4ebfzWdKnrnGseuQpezXdG8yCdHqwQ1SSBHD3bWArND8

No local executor or validator setup needed — just connect to devnet.

Quick Start

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Build your first encrypted program in 5 minutes.

1. Write an FHE Function

```
use encrypt_dsl::prelude::*;

#[encrypt_fn]
fn add(a: EUint64, b: EUint64) -> EUint64 {
    a + b
}
```

The `#[encrypt_fn]` macro generates:

- `add()` — returns the serialized computation graph bytes
- `AddCpi` — an extension trait on `EncryptCpi` with method `ctx.add(a, b, output)?`

2. Use It in Your Program

Pinocchio

```
use encrypt_pinocchio::EncryptContext;

let ctx = EncryptContext { /* ... */ };
ctx.add(input_a, input_b, output_ct)?;
```

Anchor

```
use encrypt_anchor::EncryptContext;

let ctx = EncryptContext { /* ... */ };
ctx.add(input_a.to_account_info(), input_b.to_account_info(),
output.to_account_info())?;
```

Native

```
use encrypt_native::EncryptContext;

let ctx = EncryptContext { /* ... */ };
ctx.add(input_a.clone(), input_b.clone(), output.clone())?;
```

3. Test It

```
#[cfg(test)]
mod tests {
    use encrypt_solana_test::EncryptTestContext;
    use encrypt_types::encrypted::Uint64;

    #[test]
    fn test_add() {
        let mut ctx = EncryptTestContext::new_default();
        let user = ctx.new_funded_keypair();

        let a = ctx.create_input::<Uint64>(10, &user.pubkey());
        let b = ctx.create_input::<Uint64>(32, &user.pubkey());

        let graph = super::add();
        let outputs = ctx.execute_and_commit(&graph, &[a, b], 1, &[], &user);

        let result = ctx.decrypt::<Uint64>(&outputs[0], &user);
        assert_eq!(result, 42);
    }
}
```

4. Client SDK (gRPC)

Submit encrypted inputs and read ciphertexts via the gRPC client:

Rust

```
use encrypt_solana_client::grpc::{EncryptClient, TypedInput};
use encrypt_types::encrypted::{Uint64, Bool};

// Connect to pre-alpha endpoint
let mut client = EncryptClient::connect().await?;

// Create a single encrypted input
let ct = client.create_input::<Uint64>(42u64, &program_id,
&network_key).await?;

// Create batch inputs (one proof covers all)
let cts = client.create_inputs(
    &[TypedInput::new::<Uint64>(&10u64), TypedInput::new::<Bool>(&true)],
    &program_id, &network_key,
).await?;

// Read a ciphertext off-chain (signs request with keypair)
let result = client.read_ciphertext(&ct, &reencryption_key, epoch,
&keypair).await?;
// result.value = plaintext bytes (mock) or re-encrypted ciphertext
// (production)
// result.fhe_type, result.digest
```

TypeScript

```
import { createEncryptClient, encodeReadCiphertextMessage, Chain } from
"@encrypt.xyz/pre-alpha-solana-client/grpc";

const client = createEncryptClient();

// Create encrypted input
const { ciphertextIdentifiers } = await client.createInput({
  chain: Chain.SOLANA,
  inputs: [{ ciphertextBytes: ciphertext, fheType: 4 }],
  proof: proofBytes,
  authorized: programId.toBytes(),
  networkEncryptionPublicKey: networkKey,
});

// Read ciphertext off-chain
const msg = encodeReadCiphertextMessage(Chain.SOLANA, ctId, reencryptionKey,
epoch);
const result = await client.readCiphertext({ message: msg, signature, signer
});
```

What Happens Under the Hood

1. Your program calls `execute_graph` → on-chain creates output ciphertext accounts (status=PENDING)
2. The executor detects the event → evaluates the computation graph → calls `commit_ciphertext` (status=VERIFIED)
3. When you call `request_decryption` → the decryptor responds with the plaintext result
4. Your program reads the result from the DecryptionRequest account
5. Off-chain reads via `read_ciphertext` gRPC — public ciphertexts are open, private ones require signed request

In test mode, `EncryptTestContext` handles all of this automatically via `process_pending()` .

Pre-Alpha Environment

Resource	Endpoint
Encrypt gRPC	<code>pre-alpha-dev-1.encrypt.ika-network.net:443</code> (TLS)
Solana Network	Devnet (<code>https://api.devnet.solana.com</code>)
Program ID	<code>4ebfzWdKnrnGseuQpezXdG8yCdHqwQ1SSBHD3bWArND8</code>

Core Concepts

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Ciphertext

A **ciphertext** is an encrypted value stored on-chain. It's a regular Solana keypair account (not a PDA) owned by the Encrypt program. The account pubkey IS the ciphertext identifier.

Ciphertext account (98 bytes):

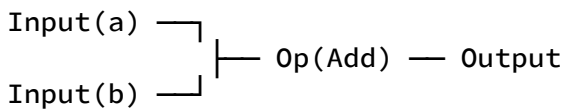
<code>ciphertext_digest(32)</code>	– hash of the actual encrypted blob
<code>authorized(32)</code>	– who can use this (zero = public)
<code>network_encryption_public_key(32)</code>	– FHE key it was encrypted under
<code>fhe_type(1)</code>	– EBool, EUint64, etc.
<code>status(1)</code>	– Pending(0) or Verified(1)

Ciphertexts are created in three ways:

- **Authority input** (`create_input_ciphertext`): user submits encrypted data + ZK proof → executor verifies → creates on-chain
- **Plaintext** (`create_plaintext_ciphertext`): user provides plaintext value → encrypted off-chain by executor
- **Graph output** (`execute_graph`): computation produces new ciphertexts (status=PENDING until executor commits)

Computation Graph

FHE operations are compiled into a **computation graph** — a DAG of operations:



The `#[encrypt_fn]` macro compiles your Rust code into this graph at compile time. The graph is serialized into the `execute_graph` instruction data. The executor evaluates it off-chain using real FHE.

Executor & Decryptor

The **executor** and **decryptor** are off-chain services managed by the Encrypt network:

- **Executor**: listens for `GraphExecuted` events, evaluates computation graphs, commits results on-chain
- **Decryptor**: listens for `DecryptionRequested` events, performs threshold decryption, writes plaintext results on-chain

In the pre-alpha environment, these are hosted at `pre-alpha-dev-1.encrypt.ika-network.net:443`. You don't need to run them — just submit encrypted inputs via gRPC and let the network handle the rest.

For **local testing**, `EncryptTestContext` simulates both services in-process via `process_pending()`.

Access Control

Every ciphertext has an `authorized` field:

- `authorized = [0; 32] → public` — anyone can compute on it or decrypt it
- `authorized = <pubkey> → only that address can use it`

Access is managed via:

- **transfer_ciphertext** : change who's authorized
- **copy_ciphertext** : create a copy with different authorization
- **make_public** : set authorized to zero (irreversible)

Digest Verification

When requesting decryption, the `ciphertext_digest` is stored in the `DecryptionRequest` as a snapshot. At reveal time, verify the digest matches to ensure the ciphertext wasn't updated between request and response:

```
let digest = ctx.request_decryption(request_acct, ciphertext)?;  
proposal.pending_digest = digest; // store for later  
  
// ... later, at reveal time ...  
let value = read_decrypted_verified::&proposal.pending_digest)?;
```

Tutorial: Confidential Voting

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

This tutorial builds a complete **confidential voting** program on Solana using Encrypt. Individual votes are encrypted – nobody can see how anyone voted – but the final tally is computed via FHE and can be decrypted by the proposal authority.

What You Will Build

A Solana program with five instructions:

Instruction	Description
<code>create_proposal</code>	Creates a proposal with two encrypted-zero tallies (yes, no)
<code>cast_vote</code>	Adds an encrypted vote to the tally via FHE computation
<code>close_proposal</code>	Authority closes voting
<code>request_tally_decryption</code>	Authority requests decryption of yes or no tally
<code>reveal_tally</code>	Authority reads decrypted result and writes plaintext to proposal

How It Works

1. The authority creates a proposal. Two ciphertext accounts are initialized to encrypted zero (`EUint64`).
2. Each voter provides an encrypted boolean vote (`EBool`): 1 = yes, 0 = no.
3. The `cast_vote_graph` FHE function conditionally increments the correct counter:
 - If `vote == 1`: `yes_count += 1`, `no_count` unchanged

- If `vote == 0`: `no_count += 1`, `yes_count` unchanged
- 4. The tally ciphertext accounts are updated **in-place** (update mode) – the same account serves as both input and output.
- 5. A `VoteRecord` PDA prevents double-voting. Its existence proves the voter already voted.
- 6. After closing, the authority requests decryption and verifies the result against a stored digest.

Key Concepts Covered

- `#[encrypt_fn]` – writing FHE computation as normal Rust
- **Plaintext ciphertext creation** – initializing encrypted zeros via `create_plaintext_typed`
- **Update mode** – passing the same account as both input and output to `execute_graph`
- **Digest verification** – store-and-verify pattern for safe decryption
- **EncryptTestContext** – testing the full lifecycle in a single test

Framework Variants

The tutorial uses Pinocchio for maximum CU efficiency. Equivalent examples exist for all three frameworks:

Framework	Source
Pinocchio	<code>chains/solana/examples/confidential-voting-pinocchio/</code>
Anchor	<code>chains/solana/examples/confidential-voting-anchor/</code>
Native	<code>chains/solana/examples/confidential-voting-native/</code>

Prerequisites

- [Installation](#) complete
- Familiarity with [Core Concepts](#)
- Basic Solana program development experience

Create the Program

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Cargo.toml

Create a new Solana program crate with Encrypt dependencies:

```
[package]
name = "confidential-voting-pinocchio"
version = "0.1.0"
edition = "2024"

[dependencies]
encrypt-types = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-dsl = { package = "encrypt-solana-dsl", git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-pinocchio = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
pinocchio = "0.10"
pinocchio-system = "0.5"

[dev-dependencies]
encrypt-solana-test = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }

[lib]
crate-type = ["cdylib", "lib"]
```

Key crates:

- **encrypt-dsl** (actually `encrypt-solana-dsl`) – the `#[encrypt_fn]` macro that generates both the computation graph and the CPI extension trait
- **encrypt-pinocchio** – `EncryptContext` and account helpers for Pinocchio programs
- **encrypt-types** – FHE types (`EUint64`, `EBool`, `Uint64`) and graph utilities

lib.rs Skeleton

```
#![allow(unexpected_cfgs)]

use encrypt_dsl::prelude::encrypt_fn;
use encrypt_pinocchio::accounts::{self, DecryptionRequestStatus};
use encrypt_pinocchio::EncryptContext;
use encrypt_types::encrypted::{EBool, EUint64, Uint64};
use pinocchio::{
    cpi::{Seed, Signer},
    entrypoint,
    error::ProgramError,
    AccountView, Address, ProgramResult,
};
use pinocchio_system::instructions::CreateAccount;

entrypoint!(process_instruction);

pub const ID: Address = Address::new_from_array([3u8; 32]);
```

Account Discriminators

Define discriminators for your program's account types:

```
const PROPOSAL: u8 = 1;
const VOTE_RECORD: u8 = 2;
```

Proposal Account

The proposal stores the authority, proposal ID, references to the encrypted tally ciphertexts, voting status, and fields for decryption verification:

```
#[repr(C)]
pub struct Proposal {
    pub discriminator: u8,
    pub authority: [u8; 32],
    pub proposal_id: [u8; 32],
    pub yes_count: EUint64,           // ciphertext account pubkey
    pub no_count: EUint64,          // ciphertext account pubkey
    pub is_open: u8,
    pub total_votes: [u8; 8],       // plaintext total for transparency
    pub revealed_yes: [u8; 8],     // written after decryption
    pub revealed_no: [u8; 8],      // written after decryption
    pub pending_yes_digest: [u8; 32], // stored at request_decryption time
    pub pending_no_digest: [u8; 32], // stored at request_decryption time
    pub bump: u8,
}
```

The `yes_count` and `no_count` fields store the **pubkeys** of the ciphertext accounts. Since `EUint64` is a 32-byte type alias, this works naturally – the ciphertext account’s Solana pubkey IS the ciphertext identifier.

The `pending_*_digest` fields are critical for the [store-and-verify pattern](#). When requesting decryption, `request_decryption` returns the current `ciphertext_digest`. You store it here and verify it at reveal time to ensure the ciphertext was not modified between request and response.

```
impl Proposal {
    pub const LEN: usize = core::mem::size_of::<Self>();

    pub fn from_bytes(data: &[u8]) -> Result<&Self, ProgramError> {
        if data.len() < Self::LEN || data[0] != PROPOSAL {
            return Err(ProgramError::InvalidAccountData);
        }
        Ok(unsafe { &*(data.as_ptr() as *const Self) })
    }

    pub fn from_bytes_mut(data: &mut [u8]) -> Result<&mut Self, ProgramError> {
        if data.len() < Self::LEN {
            return Err(ProgramError::InvalidAccountData);
        }
        Ok(unsafe { &mut *(data.as_mut_ptr() as *mut Self) })
    }

    pub fn total_votes(&self) -> u64 {
        u64::from_le_bytes(self.total_votes)
    }

    pub fn set_total_votes(&mut self, val: u64) {
        self.total_votes = val.to_le_bytes();
    }
}
```

VoteRecord Account

The vote record is a PDA seeded by `["vote", proposal_id, voter]`. Its existence proves the voter already voted. It contains no vote data – the vote is only in the encrypted tally.

```
#[repr(C)]
pub struct VoteRecord {
    pub discriminator: u8,
    pub voter: [u8; 32],
    pub bump: u8,
}

impl VoteRecord {
    pub const LEN: usize = core::mem::size_of::<Self>();
}
```

Instruction Dispatch

```
fn process_instruction(
    program_id: &Address,
    accounts: &[AccountView],
    data: &[u8],
) -> ProgramResult {
    match data.split_first() {
        Some((&0, rest)) => create_proposal(program_id, accounts, rest),
        Some((&1, rest)) => cast_vote(program_id, accounts, rest),
        Some((&2, _rest)) => close_proposal(accounts),
        Some((&3, rest)) => request_tally_decryption(accounts, rest),
        Some((&4, rest)) => reveal_tally(accounts, rest),
        _ => Err(ProgramError::InvalidInstructionData),
    }
}
```

Next Step

With the program skeleton in place, the next chapter writes the FHE computation logic.

Write FHE Logic

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

The core of confidential voting is a single FHE function that conditionally increments the yes or no counter based on an encrypted vote.

The cast_vote_graph Function

```
use encrypt_dsl::prelude::encrypt_fn;
use encrypt_types::encrypted::{EBool, EUint64};

#[encrypt_fn]
fn cast_vote_graph(
    yes_count: EUint64,
    no_count: EUint64,
    vote: EBool,
) -> (EUint64, EUint64) {
    let new_yes = if vote { yes_count + 1 } else { yes_count };
    let new_no = if vote { no_count } else { no_count + 1 };
    (new_yes, new_no)
}
```

What the Macro Generates

The `#[encrypt_fn]` macro generates two things:

1. Graph bytes function

```
fn cast_vote_graph() -> Vec<u8>
```

Returns the serialized computation graph. The graph has:

- 3 inputs: `yes_count` (EUInt64), `no_count` (EUInt64), `vote` (EBool)
- 1 constant: the literal `1` (auto-promoted to an encrypted EUInt64 constant)
- Operations: two `Add`, two `Select` (from the `if / else` expressions)
- 2 outputs: `new_yes` (EUInt64), `new_no` (EUInt64)

2. CPI extension trait

```
trait CastVoteGraphCpi: EncryptCpi {
  fn cast_vote_graph(
    &self,
    yes_count: Self::Account<'_>, // EUInt64 input
    no_count: Self::Account<'_>,  // EUInt64 input
    vote: Self::Account<'_>,      // EBool input
    __out_0: Self::Account<'_>,   // EUInt64 output
    __out_1: Self::Account<'_>,   // EUInt64 output
  ) -> Result<(), Self::Error>;
}

impl<T: EncryptCpi> CastVoteGraphCpi for T {}
```

The trait is automatically implemented for all `EncryptCpi` types, so you call it as a method on `EncryptContext`.

How if/else Works in FHE

FHE does not support branching – both branches are always evaluated. The `if / else` syntax compiles to a **Select** operation:

```
1. has_funds = IsEqual(vote, 1)    -- condition (already EBool)
2. yes_plus  = Add(yes_count, 1)   -- both branches computed
3. no_plus   = Add(no_count, 1)
4. new_yes   = Select(vote, yes_plus, yes_count)
5. new_no    = Select(vote, no_plus, no_plus)
```

Both `yes_count + 1` and `yes_count` (unchanged) are computed; `select` picks one based on the condition. This is secure because the executor never learns which path was “taken.”

The Literal 1

The integer literal `1` in `yes_count + 1` is auto-promoted to an encrypted constant in the graph. The constant is stored in the graph’s constants section and deduplicated – both

occurrences of `+ 1` share the same constant node.

Type Safety

The generated CPI method verifies each input account's `fhe_type` at runtime before making the CPI call:

- `yes_count` must be a Ciphertext with `fhe_type == EUint64`
- `no_count` must be a Ciphertext with `fhe_type == EUint64`
- `vote` must be a Ciphertext with `fhe_type == EBool`

If any type mismatches, the transaction fails before the CPI is invoked.

Graph Shape

You can verify the graph structure in tests:

```
#[test]
fn graph_shape() {
    let d = cast_vote_graph();
    let pg = parse_graph(&d).unwrap();
    assert_eq!(pg.header().num_inputs(), 3, "yes_count + no_count + vote");
    assert_eq!(pg.header().num_outputs(), 2, "new_yes + new_no");
}
```

Next Step

With the FHE logic defined, the next chapter implements proposal creation and encrypted-zero initialization.

Create Proposal

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

The `create_proposal` instruction creates the proposal PDA and initializes two ciphertext accounts to encrypted zero.

Instruction Layout

```
discriminator: 0
data: proposal_bump(1) | cpi_authority_bump(1) | proposal_id(32)
accounts: [proposal_pda(w), authority(s),
            yes_ct(w), no_ct(w),
            encrypt_program, config, deposit(w), cpi_authority,
            caller_program, network_encryption_key, payer(s,w),
            event_authority, system_program]
```

Implementation

Create the Proposal PDA

```
fn create_proposal(
    program_id: &Address,
    accounts: &[AccountView],
    data: &[u8],
) -> ProgramResult {
    let [proposal_acct, authority, yes_ct, no_ct, encrypt_program, config,
        deposit, cpi_authority, caller_program, network_encryption_key,
        payer, event_authority, system_program, ..] = accounts
    else {
        return Err(ProgramError::NotEnoughAccountKeys);
    };
    if !authority.is_signer() || !payer.is_signer() {
        return Err(ProgramError::MissingRequiredSignature);
    }

    let proposal_bump = data[0];
    let cpi_authority_bump = data[1];
    let proposal_id: [u8; 32] = data[2..34].try_into().unwrap();

    // Create proposal PDA
    let bump_byte = [proposal_bump];
    let seeds = [
        Seed::from(b"proposal" as &[u8]),
        Seed::from(proposal_id.as_ref()),
        Seed::from(&bump_byte),
    ];
    let signer = [Signer::from(&seeds)];

    CreateAccount {
        from: payer,
        to: proposal_acct,
        lamports: minimum_balance(Proposal::LEN),
        space: Proposal::LEN as u64,
        owner: program_id,
    }
    .invoke_signed(&signer)?;
}
```

Create Encrypted Zeros

This is where Encrypt comes in. Create two ciphertext accounts initialized to encrypted zero using `create_plaintext_typed`:

```

let ctx = EncryptContext {
  encrypt_program,
  config,
  deposit,
  cpi_authority,
  caller_program,
  network_encryption_key,
  payer,
  event_authority,
  system_program,
  cpi_authority_bump,
};

ctx.create_plaintext_typed::<Uint64>(&0u64, yes_ct)?;
ctx.create_plaintext_typed::<Uint64>(&0u64, no_ct)?;

```

`create_plaintext_typed::<Uint64>` is a type-safe helper that:

1. Serializes the value (`0u64`) as little-endian bytes
2. Calls `create_plaintext_ciphertext` with `fhe_type = EUint64`
3. Creates a Ciphertext account with `status = PENDING` and `authorized` set to the calling program

The executor detects the `CiphertextCreated` event, encrypts the plaintext value off-chain, and calls `commit_ciphertext` to write the digest and set `status = VERIFIED`.

Write Proposal State

```

let d = unsafe { proposal_acct.borrow_unchecked_mut() };
let prop = Proposal::from_bytes_mut(d)?;
prop.discriminator = PROPOSAL;
prop.authority.copy_from_slice(authority.address().as_ref());
prop.proposal_id.copy_from_slice(&proposal_id);
prop.yes_count = EUint64::from_le_bytes(*yes_ct.address().as_array());
prop.no_count = EUint64::from_le_bytes(*no_ct.address().as_array());
prop.is_open = 1;
prop.set_total_votes(0);
prop.bump = proposal_bump;
Ok(())
}

```

The ciphertext account pubkeys are stored in the proposal so that later instructions can verify the correct accounts are passed.

EncryptContext Fields

Every CPI to the Encrypt program requires an `EncryptContext`. Here is what each field is:

Field	Description
encrypt_program	The Encrypt program account
config	EncryptConfig PDA (fee schedule, epoch)
deposit	EncryptDeposit PDA for fee payment
cpi_authority	PDA derived from ["__encrypt_cpi_authority", caller_program_id]
caller_program	Your program's account (the executable that invokes CPI)
network_encryption_key	NetworkEncryptionKey PDA (the FHE public key)
payer	Signer who pays for new account rent
event_authority	Encrypt program's event authority PDA
system_program	System program
cpi_authority_bump	PDA bump for the CPI authority

Next Step

With the proposal and encrypted tallies created, the next chapter implements vote casting.

Cast Votes

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

The `cast_vote` instruction is where FHE computation happens. The voter’s encrypted vote is combined with the current tallies via the `cast_vote_graph` function, and the tally ciphertext accounts are updated in-place.

Instruction Layout

```
discriminator: 1
data: vote_record_bump(1) | cpi_authority_bump(1)
accounts: [proposal(w), vote_record_pda(w), voter(s), vote_ct,
           yes_ct(w), no_ct(w),
           encrypt_program, config, deposit(w), cpi_authority,
           caller_program, network_encryption_key, payer(s,w),
           event_authority, system_program]
```

Implementation

Parse and Validate

```
fn cast_vote(program_id: &Address, accounts: &[AccountView], data: &[u8]) ->
ProgramResult {
    let [proposal_acct, vote_record_acct, voter, vote_ct, yes_ct, no_ct,
        encrypt_program, config, deposit, cpi_authority, caller_program,
        network_encryption_key, payer, event_authority, system_program, ..] =
accounts
    else {
        return Err(ProgramError::NotEnoughAccountKeys);
    };
    if !voter.is_signer() {
        return Err(ProgramError::MissingRequiredSignature);
    }

    let vote_record_bump = data[0];
    let cpi_authority_bump = data[1];

    // Verify proposal is open
    let prop_data = unsafe { proposal_acct.borrow_unchecked() };
    let prop = Proposal::from_bytes(prop_data)?;
    if prop.is_open == 0 {
        return Err(ProgramError::InvalidArgument);
    }
    let proposal_id = prop.proposal_id;
```

Prevent Double Voting

Create a VoteRecord PDA. If the voter already voted, `CreateAccount` fails because the PDA already exists:

```

let vr_bump_byte = [vote_record_bump];
let vr_seeds = [
  Seed::from(b"vote" as &[u8]),
  Seed::from(proposal_id.as_ref()),
  Seed::from(voter.address().as_ref()),
  Seed::from(&vr_bump_byte),
];
let vr_signer = [Signer::from(&vr_seeds)];

CreateAccount {
  from: payer,
  to: vote_record_acct,
  lamports: minimum_balance(VoteRecord::LEN),
  space: VoteRecord::LEN as u64,
  owner: program_id,
}
.invoke_signed(&vr_signer)?;

let vr_data = unsafe { vote_record_acct.borrow_unchecked_mut() };
vr_data[0] = VOTE_RECORD;
vr_data[1..33].copy_from_slice(voter.address().as_ref());
vr_data[33] = vote_record_bump;

```

Execute the FHE Graph

This is the key line – call the DSL-generated CPI method:

```

let ctx = EncryptContext {
  encrypt_program,
  config,
  deposit,
  cpi_authority,
  caller_program,
  network_encryption_key,
  payer,
  event_authority,
  system_program,
  cpi_authority_bump,
};

ctx.cast_vote_graph(yes_ct, no_ct, vote_ct, yes_ct, no_ct)?;

```

Notice: `yes_ct` and `no_ct` appear as both **inputs** (positions 1-2) and **outputs** (positions 4-5). This is **update mode**.

Update Mode

When an output account already contains ciphertext data, `execute_graph` operates in update mode:

- The existing ciphertext is **read** as an input
- The same account is **reset** as an output (digest zeroed, status set to PENDING)
- The executor evaluates the graph and commits the new digest

This means the tally accounts keep the same pubkey across all votes. No new accounts are created per vote.

Increment Total Votes

After the FHE computation, increment the plaintext vote counter for transparency:

```
let prop_data_mut = unsafe { proposal_acct.borrow_unchecked_mut() };
let prop_mut = Proposal::from_bytes_mut(prop_data_mut)?;
prop_mut.set_total_votes(prop_mut.total_votes() + 1);

Ok(())
}
```

The Voter's Vote Ciphertext

The `vote_ct` account is an encrypted boolean (`EBool`) created by the voter before calling `cast_vote` . The voter:

1. Generates a keypair for the ciphertext account
2. Encrypts their vote (1 = yes, 0 = no) off-chain
3. Submits it to the executor via `create_input_ciphertext` (with ZK proof that the value is 0 or 1)
4. The executor verifies the proof and creates the on-chain ciphertext

The vote value is never visible on-chain. The program only sees the ciphertext account pubkey.

Anchor Equivalent

In Anchor, the same logic uses `to_account_info()` and `.clone()` :

```
let yes_ct = ctx.accounts.yes_ct.to_account_info();
let no_ct = ctx.accounts.no_ct.to_account_info();
let vote_ct = ctx.accounts.vote_ct.to_account_info();
encrypt_ctx.cast_vote_graph(
    yes_ct.clone(), no_ct.clone(), vote_ct,
    yes_ct, no_ct,
)?;
```

Next Step

With voting implemented, the next chapter covers decryption of the final tallies.

Decrypt Results

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

After the proposal is closed, the authority requests decryption of the tally ciphertexts, then reads and verifies the results.

Close the Proposal

First, the authority closes voting:

```
fn close_proposal(accounts: &[AccountView]) -> ProgramResult {
    let [proposal_acct, authority, ..] = accounts else {
        return Err(ProgramError::NotEnoughAccountKeys);
    };
    if !authority.is_signer() {
        return Err(ProgramError::MissingRequiredSignature);
    }

    let prop_data = unsafe { proposal_acct.borrow_unchecked_mut() };
    let prop = Proposal::from_bytes_mut(prop_data)?;

    if authority.address().as_array() != &prop.authority {
        return Err(ProgramError::InvalidArgument);
    }
    if prop.is_open == 0 {
        return Err(ProgramError::InvalidArgument);
    }

    prop.is_open = 0;
    Ok(())
}
```

Request Decryption

The authority calls `request_tally_decryption` for each tally (yes and no separately):

```
fn request_tally_decryption(accounts: &[AccountView], data: &[u8]) ->
ProgramResult {
    let [proposal_acct, request_acct, ciphertext, encrypt_program, config,
        deposit, cpi_authority, caller_program, network_encryption_key,
        payer, event_authority, system_program, ..] = accounts
    else {
        return Err(ProgramError::NotEnoughAccountKeys);
    };

    let cpi_authority_bump = data[0];
    let is_yes = data[1] != 0;

    // Verify proposal is closed
    let prop_data = unsafe { proposal_acct.borrow_unchecked() };
    let prop = Proposal::from_bytes(prop_data)?;
    if prop.is_open != 0 {
        return Err(ProgramError::InvalidArgument);
    }

    let ctx = EncryptContext {
        encrypt_program, config, deposit, cpi_authority, caller_program,
        network_encryption_key, payer, event_authority, system_program,
        cpi_authority_bump,
    };

    // request_decryption returns the ciphertext_digest -- store it
    let digest = ctx.request_decryption(request_acct, ciphertext)?;

    let prop_data_mut = unsafe { proposal_acct.borrow_unchecked_mut() };
    let prop_mut = Proposal::from_bytes_mut(prop_data_mut)?;
    if is_yes {
        prop_mut.pending_yes_digest = digest;
    } else {
        prop_mut.pending_no_digest = digest;
    }

    Ok(())
}
```

What request_decryption Does

1. Creates a `DecryptionRequest` keypair account
2. Stores a snapshot of the ciphertext's current `ciphertext_digest`
3. Returns the digest as `[u8; 32]`
4. Emits a `DecryptionRequested` event

The decryptor detects the event, performs threshold MPC decryption (or mock decryption locally), and calls `respond_decryption` to write the plaintext result into the request account.

Why Store the Digest?

The ciphertext could be updated between request and response (e.g., another vote sneaks in). By storing the digest at request time and verifying it at reveal time, you ensure the decrypted value corresponds to the exact ciphertext you requested.

Reveal the Tally

Once the decryptor has responded, the authority reads the result:

```

fn reveal_tally(accounts: &[AccountView], data: &[u8]) -> ProgramResult {
    let [proposal_acct, request_acct, authority, ..] = accounts else {
        return Err(ProgramError::NotEnoughAccountKeys);
    };
    if !authority.is_signer() {
        return Err(ProgramError::MissingRequiredSignature);
    }

    let is_yes = data[0] != 0;

    // Verify authority and closed status
    let prop_data = unsafe { proposal_acct.borrow_unchecked() };
    let prop = Proposal::from_bytes(prop_data)?;
    if authority.address().as_array() != &prop.authority {
        return Err(ProgramError::InvalidArgument);
    }
    if prop.is_open != 0 {
        return Err(ProgramError::InvalidArgument);
    }

    // Get the digest stored at request time
    let expected_digest = if is_yes {
        &prop.pending_yes_digest
    } else {
        &prop.pending_no_digest
    };

    // Verify and read the decrypted value
    let req_data = unsafe { request_acct.borrow_unchecked() };
    let value: &u64 = accounts::read_decrypted_verified::<Uint64>(req_data,
expected_digest)?;

    // Write plaintext to proposal
    let prop_data_mut = unsafe { proposal_acct.borrow_unchecked_mut() };
    let prop_mut = Proposal::from_bytes_mut(prop_data_mut)?;
    if is_yes {
        prop_mut.revealed_yes = value.to_le_bytes();
    } else {
        prop_mut.revealed_no = value.to_le_bytes();
    }

    Ok(())
}

```

read_decrypted_verified

This function:

1. Reads the `DecryptionRequestHeader` from the request account
2. Verifies `bytes_written == total_len` (decryption is complete)
3. Verifies the stored `ciphertext_digest` matches `expected_digest`
4. Returns a reference to the plaintext value

If the digest does not match, it returns an error – protecting against stale or tampered values.

Full Decryption Flow

1. `close_proposal` -- authority closes voting
2. `request_tally_decryption(is_yes=true)` -- store yes digest
3. `request_tally_decryption(is_yes=false)` -- store no digest
4. [decryptor responds automatically]
5. `reveal_tally(is_yes=true)` -- read yes result, verify digest
6. `reveal_tally(is_yes=false)` -- read no result, verify digest

After step 6, the proposal's `revealed_yes` and `revealed_no` fields contain the plaintext tallies, readable by anyone.

Cleanup

After revealing, close the decryption request accounts to reclaim rent:

```
ctx.close_decryption_request(request_acct, destination)?;
```

Next Step

The next chapter covers testing the complete voting flow with `EncryptTestContext`.

Testing

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Encrypt provides four levels of testing for your programs:

1. **Unit tests** — verify graph logic with mock arithmetic (no SBF needed)
2. **LiteSVM e2e tests** — fast in-process lifecycle with deployed programs and CPI
3. **solana-program-test e2e tests** — official Solana runtime, full sysvar support
4. **Mollusk tests** — isolated instruction-level validation

Setup

```
[dev-dependencies]
encrypt-solana-test = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-types = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-dsl = { package = "encrypt-solana-dsl", git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
solana-sdk = "4"
mollusk-svm = "0.11"
solana-account = "3"
solana-pubkey = "4"
solana-instruction = "3"
```

Unit Testing the Graph

The simplest tests verify graph correctness with mock plaintext arithmetic:

```

#[cfg(test)]
mod tests {
    use super::cast_vote_graph;

    #[test]
    fn vote_yes_increments_yes_count() {
        let r = run_mock(
            cast_vote_graph,
            &[10, 5, 1],
            &[FheType::EUint64, FheType::EUint64, FheType::EBool],
        );
        assert_eq!(r[0], 11);
        assert_eq!(r[1], 5);
    }

    #[test]
    fn graph_shape() {
        let d = cast_vote_graph();
        let pg = parse_graph(&d).unwrap();
        assert_eq!(pg.header().num_inputs(), 3);
        assert_eq!(pg.header().num_outputs(), 2);
    }
}

```

Run with `cargo test -p your-program --lib --no-sbf-build-needed`.

LiteSVM End-to-End Tests

Test the full lifecycle: deploy your program → send transactions → CPI to Encrypt → verify results.

```

use encrypt_dsl::prelude::encrypt_fn;
use encrypt_solana_test::litesvm::EncryptTestContext;
use encrypt_types::encrypted::{EBool, EUint64, Bool, Uint64};

// Redefine graph for off-chain evaluation
#[encrypt_fn]
fn cast_vote_graph(yes_count: EUint64, no_count: EUint64, vote: EBool) ->
(EUint64, EUint64) {
    let new_yes = if vote { yes_count + 1 } else { yes_count };
    let new_no = if vote { no_count } else { no_count + 1 };
    (new_yes, new_no)
}

#[test]
fn test_full_voting_lifecycle() {
    let mut ctx = EncryptTestContext::new_default();

    // Deploy your program
    let program_id = ctx.deploy_program("path/to/your_program.so");
    let (cpi_authority, cpi_bump) = ctx.cpi_authority_for(&program_id);

    // 1. Create proposal (CPI creates yes/no ciphertexts)
    ctx.send_transaction(&[create_proposal_ix(...)], &[&authority, &yes_ct,
&no_ct]);
    ctx.register_ciphertext(&yes_pubkey);
    ctx.register_ciphertext(&no_pubkey);

    // 2. Cast vote (CPI to execute_graph)
    let vote_ct = ctx.create_input::<Bool>(1, &program_id);
    ctx.send_transaction(&[cast_vote_ix(...)], &[&voter]);

    // 3. Process the graph execution off-chain
    let graph = cast_vote_graph();
    ctx.enqueue_graph_execution(&graph, &[yes_pubkey, no_pubkey, vote_ct], &
[yes_pubkey, no_pubkey]);
    ctx.process_pending();
    ctx.register_ciphertext(&yes_pubkey);
    ctx.register_ciphertext(&no_pubkey);

    // 4. Close proposal
    ctx.send_transaction(&[close_ix(...)], &[&authority]);

    // 5. Verify results
    assert_eq!(ctx.decrypt_from_store(&yes_pubkey), 1);
    assert_eq!(ctx.decrypt_from_store(&no_pubkey), 0);
}

```

Key patterns for CPI e2e tests

- **register_ciphertext** — call after CPI creates/updates ciphertexts the harness doesn't know about
- **enqueue_graph_execution + process_pending** — simulate the off-chain executor evaluating graphs triggered by CPI

- **decrypt_from_store** — read results from the mock store (no on-chain decryption request needed)
- **Ciphertext authorization** — authorize to the *program* ID (not the voter), since the program is the CPI caller

Mollusk Instruction Tests

Test individual instructions in isolation without CPI. Best for:

- Signer/authority checks
- Account validation
- Edge cases (already closed, wrong digest, missing accounts)

```
use mollusk_svm::Mollusk;

#[test]
fn test_close_proposal_rejects_wrong_authority() {
    let (mollusk, program_id) = setup();
    let auth = Pubkey::new_unique();
    let wrong = Pubkey::new_unique();

    let prop_data = build_proposal_data(&auth, &proposal_id, true, 0);

    let result = mollusk.process_instruction(
        &Instruction::new_with_bytes(program_id, &[2u8], vec![
            AccountMeta::new(prop_key, false),
            AccountMeta::new_readonly(wrong, true),
        ]),
        &[(prop_key, program_account(&program_id, prop_data)), (wrong,
funded_account())],
    );
    assert!(result.program_result.is_err());
}

#[test]
fn test_reveal_tally_rejects_digest_mismatch() {
    let (mollusk, program_id) = setup();
    // ... build proposal with digest A, request with digest B
    // ... verify the reveal fails
}
```

solana-program-test

Same API as LiteSVM but uses the official Solana runtime. Programs must be declared upfront:

```
use encrypt_solana_test::program_test::ProgramTestEncryptContext;

#[test]
fn test_with_official_runtime() {
    let mut ctx = ProgramTestEncryptContext::builder()
        .add_program("my_program", program_id)
        .build();
    // Same API as EncryptTestContext
}
```

`ProgramTestEncryptContext` wraps `EncryptTestHarness<ProgramTestRuntime>`. The `ProgramTestRuntime` blocks async `BanksClient` calls on a tokio runtime, so tests remain synchronous.

When to use which:

- **LiteSVM** — fastest, good for iteration. Partial sysvar support.
- **solana-program-test** — slower, but uses the real Solana runtime. Full sysvar + rent support. Use for CI or when LiteSVM behavior diverges.

Running Tests

```
# All tests (builds SBF first)
just test

# Unit tests only (fast, no SBF)
just test-unit

# Example tests only
just test-examples          # All (unit + litesvm + mollusk + program-
test)
just test-examples-litesvm  # LiteSVM e2e only
just test-examples-mollusk  # Mollusk only
just test-examples-program-test # solana-program-test e2e only

# Single example
cargo test -p confidential-voting-pinocchio
```

Mock vs Real FHE

In test mode, `EncryptTestContext` uses `MockComputeEngine` — operations are performed as plaintext arithmetic. The 32-byte ciphertext digest directly encodes the plaintext value. This means:

- Graph evaluation is instantaneous
- Decryption is trivial

- No privacy (values visible in account data)

The same test code will work unchanged when real REFHE is available. See [Mock vs Real FHE](#) for details.

The Encrypt DSL

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

The `#[encrypt_fn]` attribute macro lets you write FHE computation as normal Rust. The macro compiles it into a computation graph at compile time.

Two Macros

Macro	Crate	Generates
<code>#[encrypt_fn_graph]</code>	<code>encrypt-dsl</code>	Graph bytes function only (<code>fn name() -> Vec<u8></code>)
<code>#[encrypt_fn]</code>	<code>encrypt-solana-dsl</code>	Graph bytes + Solana CPI extension trait

Use `#[encrypt_fn]` for Solana programs. Use `#[encrypt_fn_graph]` for chain-agnostic graph generation (testing, analysis).

What Gets Generated

```
#[encrypt_fn]
fn transfer(from: EUint64, to: EUint64, amount: EUint64) -> (EUint64, EUint64)
{
    let has_funds = from >= amount;
    let new_from = if has_funds { from - amount } else { from };
    let new_to = if has_funds { to + amount } else { to };
    (new_from, new_to)
}
```

This generates:

1. **transfer()** → `Vec<u8>` — the serialized computation graph
2. **TransferCpi** — an extension trait implemented for all `EncryptCpi` types:

```
// Generated (simplified):
trait TransferCpi: EncryptCpi {
  fn transfer(
    &self,
    from: Self::Account<'_>, // EUint64 input
    to: Self::Account<'_>,   // EUint64 input
    amount: Self::Account<'_>, // EUint64 input
    __out_0: Self::Account<'_>, // EUint64 output
    __out_1: Self::Account<'_>, // EUint64 output
  ) -> Result<(), Self::Error>;
}

impl<T: EncryptCpi> TransferCpi for T {}
```

Method Syntax

Call the generated function as a method on your `EncryptContext`:

```
ctx.transfer(from_ct, to_ct, amount_ct, new_from_ct, new_to_ct)?;
```

The trait is automatically in scope (generated in the same module as your `#[encrypt_fn]`).

Type Safety

The generated function:

- Has one parameter per encrypted input (in order)
- Has one parameter per output (in order)
- Verifies each input's `fhe_type` matches the graph at runtime
- Returns an error if types don't match

This catches bugs like passing an `EBool` where an `EUint64` is expected.

Update Mode

Output accounts can be either:

- **New accounts** (empty) → `execute_graph` creates a new Ciphertext

- **Existing accounts** (already has data) → `execute_graph` resets digest/status (reuses the account)

For update mode, pass the same account as both input and output:

```
// yes_ct is both input[0] and output[0]  
ctx.cast_vote_graph(yes_ct, no_ct, vote_ct, yes_ct, no_ct)?;
```

FHE Types

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Scalar Types (16)

Type	Byte Width	Rust Equivalent
EBool	1	u8 (0 or 1)
EUInt8	1	u8
EUInt16	2	u16
EUInt32	4	u32
EUInt64	8	u64
EUInt128	16	u128
EUInt256	32	[u8; 32]
EAddress	32	[u8; 32]
EUInt512	64	[u8; 64]
EUInt1024	128	[u8; 128]
... up to EUInt65536	8192	[u8; 8192]

Boolean Vectors (16)

EBitVector2 through EBitVector65536 — packed boolean arrays.

Arithmetic Vectors (13)

`EVectorU8` through `EVectorU32768` — SIMD-style encrypted integer arrays (8,192 bytes each).

Plaintext Types

For inputs that don't need encryption:

Type	Encrypted Equivalent
<code>PBool</code>	<code>EBool</code>
<code>PUint8</code>	<code>EUint8</code>
<code>PUint16</code>	<code>EUint16</code>
<code>PUint32</code>	<code>EUint32</code>
<code>PUint64</code>	<code>EUint64</code>
...	...

Plaintext inputs are embedded in the instruction data (not ciphertext accounts).

Type Safety

Each type has a compile-time `FHE_TYPE_ID` :

- Operations between incompatible types fail at compile time
- The on-chain processor verifies `fhe_type` of each input account matches the graph
- The CPI extension trait verifies `fhe_type` at runtime before CPI

Operations

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Arithmetic

```
let sum = a + b;    // Add
let diff = a - b;   // Subtract
let prod = a * b;   // Multiply
let quot = a / b;   // Divide
let rem = a % b;    // Modulo
let neg = -a;       // Negate
```

Bitwise

```
let and = a & b;    // AND
let or = a | b;     // OR
let xor = a ^ b;    // XOR
let not = !a;       // NOT
let shl = a << b;   // Shift left
let shr = a >> b;   // Shift right
```

Comparison

All comparisons return the **same encrypted type** (0 or 1), not `EBool` :

```
let eq = a == b;      // Equal
let ne = a != b;      // Not equal
let lt = a < b;        // Less than
let le = a <= b;       // Less or equal
let gt = a > b;        // Greater than
let ge = a >= b;       // Greater or equal
```

Method Syntax

Same operations, explicit names:

```
let sum = a.add(&b);
let cmp = a.is_greater_or_equal(&b);
let min_val = a.min(&b);
let max_val = a.max(&b);
let rotated = a.rotate_left(&n);
```

Constants

Bare integer literals are auto-promoted to encrypted constants:

```
let incremented = count + 1;      // 1 becomes an encrypted constant
let doubled = value * 2;          // 2 becomes an encrypted constant
```

For explicit construction:

```
let one = EUint64::from(1u64);
let big = EUint256::from([0xABu8; 32]);
let vec = EVectorU32::from_elements([1u32, 2, 3, 4]);
let ones = EVectorU64::splat(1u128);
let bits = EBitVector16::from(0b1010u128);
```

Identical constants are automatically deduplicated in the graph.

Constants

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Constants are plaintext values embedded directly in the computation graph. The executor applies encryption automatically.

Bare Literals

The simplest way — integer literals in expressions auto-promote:

```
#[encrypt_fn]
fn increment(count: EUint64) -> EUint64 {
    count + 1 // 1 is auto-promoted to an encrypted EUint64 constant
}
```

Explicit Construction

For types that need explicit creation:

```
// Scalars (up to 128 bits)
let zero = EUint64::from(0u64);
let max = EUint128::from(u128::MAX);

// Big types (byte arrays)
let addr = EUint256::from([0xABu8; 32]);

// Vectors – from elements
let vec = EVectorU32::from_elements([1u32, 2, 3, 4]);

// Vectors – all same value
let ones = EVectorU64::splat(1u128);

// Boolean vectors – from bitmask
let mask = EBitVector16::from(0b1010_1010u128);
```

Deduplication

Constants with the same (fhe_type, bytes) are automatically deduplicated in the graph. Writing `count + 1` twice produces a single constant node, not two.

Vectors

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Encrypt supports SIMD-style encrypted vectors — fixed-size arrays of encrypted integers where every element-wise operation runs in a single FHE computation. Vectors enable batch processing (e.g., updating 2048 balances in one graph execution).

Vector Types

All arithmetic vectors are exactly **8,192 bytes** (65,536 bits). The element count depends on element size:

Type	Element	Elements	FHE Type ID
<code>EUint8Vector</code>	<code>u8</code>	8,192	32
<code>EUint16Vector</code>	<code>u16</code>	4,096	33
<code>EUint32Vector</code>	<code>u32</code>	2,048	34
<code>EUint64Vector</code>	<code>u64</code>	1,024	35
<code>EUint128Vector</code>	<code>u128</code>	512	36
... up to <code>EUint32768Vector</code>	4,096 bytes	2	44

Boolean vectors (`EBitVector2` through `EBitVector65536`) store packed boolean arrays.

Using Vectors in `#[encrypt_fn]`

Vectors work like scalars in the DSL — all operations are element-wise:

```

use encrypt_dsl::prelude::encrypt_fn;
use encrypt_types::encrypted::EUInt32Vector;

#[encrypt_fn]
fn add_vectors(a: EUInt32Vector, b: EUInt32Vector) -> EUInt32Vector {
    a + b // element-wise: result[i] = a[i] + b[i]
}

```

Scalar Operations

Literals auto-promote to scalar operations that broadcast across all elements:

```

#[encrypt_fn]
fn scale_and_shift(v: EUInt32Vector) -> EUInt32Vector {
    v * 3 + 7 // each element: result[i] = v[i] * 3 + 7
}

```

This generates `MultiplyScalar` and `AddScalar` ops — the constant `3` is stored as a single scalar, not replicated 2,048 times.

All Arithmetic Operations

Every operation that works on scalars also works element-wise on vectors:

```

#[encrypt_fn]
fn all_ops(a: EUInt32Vector, b: EUInt32Vector) -> EUInt32Vector {
    let sum = a + b;
    let diff = a - b;
    let prod = a * b;
    let quot = a / b;
    let rem = a % b;
    let neg = -a;
    let and = a & b;
    let or = a | b;
    let xor = a ^ b;
    let not = !a;
    let min = a.min(&b);
    let max = a.max(&b);
    sum // return any of these
}

```

Comparisons

Comparisons return a vector of 0/1 values (same type, not `EBool`):

```
#[encrypt_fn]
fn compare(a: EUint32Vector, b: EUint32Vector) -> EUint32Vector {
    a == b // result[i] = 1 if a[i] == b[i], else 0
}
```

All comparison operators work: `==`, `!=`, `<`, `<=`, `>`, `>=`.

Conditionals

Use `if cond { a } else { b }` with a scalar `EBool` to select entire vectors:

```
use encrypt_types::encrypted::EBool;

#[encrypt_fn]
fn conditional(cond: EBool, a: EUint32Vector, b: EUint32Vector) ->
EUint32Vector {
    if cond { a } else { b } // selects entire vector a or b
}
```

For element-wise selection (different condition per element), use `select_scalar`:

```
#[encrypt_fn]
fn elementwise_select(
    mask: EUint32Vector, // 0 or nonzero per element
    a: EUint32Vector,
    b: EUint32Vector,
) -> EUint32Vector {
    mask.select_scalar(&a, &b) // result[i] = mask[i] != 0 ? a[i] : b[i]
}
```

Multiple Outputs

A single graph can produce multiple output vectors:

```
#[encrypt_fn]
fn sum_and_diff(a: EUint32Vector, b: EUint32Vector) -> (EUint32Vector,
EUint32Vector) {
    (a + b, a - b)
}
```

Vector-Specific Operations

Gather

Index-based lookup: `result[i] = source[indices[i]]`

```
#[encrypt_fn]
fn permute(data: EUint32Vector, indices: EUint32Vector) -> EUint32Vector {
    data.gather(&indices)
}
```

Scatter

Inverse of gather: `result[indices[i]] = data[i]`

```
#[encrypt_fn]
fn scatter(data: EUint32Vector, indices: EUint32Vector) -> EUint32Vector {
    data.scatter(&indices)
}
```

Assign

Overwrite elements at specific positions: `result = base; result[indices[i]] = values[i]`

```
#[encrypt_fn]
fn update_positions(
    base: EUint32Vector,
    indices: EUint32Vector,
    values: EUint32Vector,
) -> EUint32Vector {
    base.assign(&indices, &values)
}
```

Copy

Copy entire vector:

```
#[encrypt_fn]
fn clone_vec(a: EUint32Vector, src: EUint32Vector) -> EUint32Vector {
    a.copy(&src) // returns src
}
```

Get

Extract a single element by index (result at position 0):

```
#[encrypt_fn]
fn extract(data: EUint32Vector, index: EUint32Vector) -> EUint32Vector {
    data.get(&index) // result[0] = data[index[0]], rest = 0
}
```

Rotate Entries

Cyclically rotate vector elements left by an encrypted scalar amount. The shift count is a scalar of the matching scalar type (e.g. `EUint32` for `EUint32Vector`):

```
use encrypt_types::encrypted::{EUint32Vector, EUint32};

#[encrypt_fn]
fn rotate(data: EUint32Vector, n: EUint32) -> EUint32Vector {
    data.rotate_entries(&n) // result[i] = data[(i + n) mod len]
}
```

The rotation wraps within the vector's element count, so positions that fall outside the populated prefix wrap back to the zero region.

Reductions

Reductions collapse a vector down to a single scalar. The output ciphertext must be allocated with the **scalar** type, not the vector type — the graph carries the result-type re-tagging from vector to scalar.

Sum / Min / Max

Numeric reductions over the entire vector:

```
use encrypt_types::encrypted::{EUint32Vector, EUint32};

#[encrypt_fn] fn sum(v: EUint32Vector) -> EUint32 { v.reduce_add() }
#[encrypt_fn] fn smallest(v: EUint32Vector) -> EUint32 { v.reduce_min() }
#[encrypt_fn] fn largest(v: EUint32Vector) -> EUint32 { v.reduce_max() }
```

Reductions span **every entry of the vector**, not just the populated prefix. For `reduce_min`, unset slots are zero — so unless every slot is filled, the minimum is always 0. Pad the vector

with the appropriate sentinel (typically the maximum value of the element type) for non-prefix workloads.

Boolean Reductions

`reduce_any` / `reduce_all` operate on `EUint8Vector` (treating each element as a boolean: 0 = false, nonzero = true) and return `EBool`:

```
use encrypt_types::encrypted::{EUint8Vector, EBool};

#[encrypt_fn] fn any_set(v: EUint8Vector) -> EBool { v.reduce_any() }
#[encrypt_fn] fn all_set(v: EUint8Vector) -> EBool { v.reduce_all() }
```

`reduce_any` returns 1 if any element is nonzero, 0 otherwise. `reduce_all` returns 1 if every element is nonzero, 0 otherwise. Both inspect the full element count of the input vector — same padding caveat as above for `reduce_all`.

Composing Reductions

Reductions chain naturally with scalar arithmetic inside the same graph:

```
#[encrypt_fn]
fn range(v: EUint32Vector) -> EUint32 {
    let mx = v.reduce_max();
    let mn = v.reduce_min();
    mx - mn
}
```

Chained Operations

Multiple operations compose naturally in a single graph:

```

#[encrypt_fn]
fn dot_product_pair(
    a: EUint32Vector, b: EUint32Vector,
    c: EUint32Vector, d: EUint32Vector,
) -> EUint32Vector {
    a * b + c * d // (a[i]*b[i]) + (c[i]*d[i])
}

#[encrypt_fn]
fn linear_transform(a: EUint32Vector, b: EUint32Vector) -> EUint32Vector {
    a * 5 + b * 3 + 7
}

#[encrypt_fn]
fn conditional_accumulate(
    cond: EBool,
    acc: EUint32Vector,
    val: EUint32Vector,
) -> EUint32Vector {
    let added = acc + val;
    if cond { added } else { acc }
}

```

Creating Vectors

Vectors are **8,192 bytes** — too large for Solana instruction data (max ~1,232 bytes). They must be created off-chain via gRPC `CreateInput` :

Rust Client

```

use encrypt_solana_client::grpc::{EncryptClient, TypedInput};
use encrypt_types::types::FheType;

// Build 8192-byte vector with elements at the start, rest zeros
let mut bytes = vec![0u8; 8192];
bytes[0..4].copy_from_slice(&100u32.to_le_bytes());
bytes[4..8].copy_from_slice(&200u32.to_le_bytes());

let ct_pubkey = client
    .create_inputs(
        &[TypedInput::from_raw(FheType::EVectorU32, bytes)],
        &authorized_pubkey,
        &network_key,
    )
    .await?;

```

TypeScript Client

```
const bytes = new Uint8Array(8192);
new DataView(bytes.buffer).setUint32(0, 100, true);
new DataView(bytes.buffer).setUint32(4, 200, true);

const [ctPubkey] = await client.createInput({
  fheType: 34, // EVectorU32
  plaintextBytes: bytes,
  authorized: programId,
  networkKey,
});
```

Testing Vectors

The test harness provides vector-specific helpers:

```
use encrypt_solana_test::litesvm::EncryptTestContext;
use encrypt_types::types::FheType;

let mut ctx = EncryptTestContext::new_default();

// Create a vector with specific elements
let mut bytes = vec![0u8; 8192];
bytes[0..4].copy_from_slice(&42u32.to_le_bytes());
bytes[4..8].copy_from_slice(&99u32.to_le_bytes());

let ct = ctx.create_input_bytes(FheType::EVectorU32, &bytes, &program_id);

// After graph execution + commit:
let result = ctx.decrypt_bytes(&ct);
let elem0 = u32::from_le_bytes(result[0..4].try_into().unwrap());
assert_eq!(elem0, 42);
```

Decryption

Vector decryption responses are automatically chunked — the 8,192-byte plaintext is split across multiple transactions (~12 txs at 700 bytes each). The on-chain `DecryptionRequest` account tracks `bytes_written / total_len` and the executor writes chunks until complete. This is transparent to the developer.

On-Chain Representation

Vectors use the same 98-byte `Ciphertext` account as scalars:

```
ciphertext_digest(32) + authorized(32) + network_encryption_public_key(32) +  
fhe_type(1) + status(1)
```

The 32-byte digest commits to the full 8,192-byte value. The actual encrypted data lives off-chain in the executor. The `fhe_type` field (e.g., 34 for `EVectorU32`) tells the executor how to interpret the data.

Limitations

- **No on-chain plaintext creation:** `create_plaintext_ciphertext` can't handle 8,192 bytes in instruction data. Use gRPC `CreateInput` instead.
- **Index range:** For `EVectorU8`, indices are `u8` values (max 255) but the vector has 8,192 elements — only the first 256 are addressable by gather/scatter/assign.
- **Reductions span the full element count:** `reduce_min` / `reduce_all` see unset slots (zero) as participating values; pad the vector if you only want to reduce over a populated prefix.

Conditionals

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

FHE doesn't support branching — both paths are always evaluated. The `if / else` syntax compiles to a **select** operation.

Syntax

```
let result = if condition { value_a } else { value_b };
```

Rules:

- Both branches must be the **same encrypted type**
- Condition must be an encrypted comparison result (0 or 1)
- `else` is **mandatory** — no bare `if`
- Both branches are always evaluated (FHE requirement)

Example

```
#[encrypt_fn]
fn conditional_transfer(
    from: EUint64,
    to: EUint64,
    amount: EUint64,
) -> (EUint64, EUint64) {
    let has_funds = from >= amount;
    let new_from = if has_funds { from - amount } else { from };
    let new_to = if has_funds { to + amount } else { to };
    (new_from, new_to)
}
```

This compiles to:

1. `has_funds = IsGreaterOrEqual(from, amount) → 0 or 1`
2. `from_minus = Subtract(from, amount)`
3. `to_plus = Add(to, amount)`
4. `new_from = Select(has_funds, from_minus, from)`
5. `new_to = Select(has_funds, to_plus, to)`

Both `from - amount` and `from` are computed; `select` picks one based on the condition.

Nested Conditionals

```
let tier = if amount >= 1000 {  
    3  
} else if amount >= 100 {  
    2  
} else {  
    1  
};
```

Each `if/else` becomes a `select` operation. Nested conditionals produce a chain of `select` nodes.

Graph Compilation

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Binary Format

The `#[encrypt_fn]` macro compiles your function into a binary graph at compile time:

`[Header 13B] [Nodes N×9B] [Constants section]`

Header (13 bytes)

`version(1) | num_inputs(2) | num_plaintext_inputs(2) | num_constants(2) |
num_ops(2) | num_outputs(2) | constants_len(2)`

Counts are ordered by node kind. `num_nodes` is derived (sum of all counts).

Nodes (9 bytes each)

`kind(1) | op_type(1) | fhe_type(1) | input_a(2) | input_b(2) | input_c(2)`

Kind	Value	Description
Input	0	Encrypted ciphertext account
PlaintextInput	1	Plaintext value in instruction data
Constant	2	Literal value in constants section
Op	3	FHE operation
Output	4	Graph result

Nodes are topologically sorted — every node's operands appear earlier in the list.

Constants Section

Variable-length byte blob. Constant nodes reference it by byte offset (`input_a`). Values stored as little-endian bytes at `fhe_type.byte_width()` .

Example

```
#[encrypt_fn]  
fn add(a: EUint64, b: EUint64) -> EUint64 { a + b }
```

Produces 4 nodes:

- Node 0: Input (EUint64) — `a`
- Node 1: Input (EUint64) — `b`
- Node 2: Op (Add, EUint64, inputs: 0, 1)
- Node 3: Output (EUint64, source: 2)

Header: `version=1, num_inputs=2, num_constants=0, num_ops=1, num_outputs=1, constants_len=0`

Registered Graphs

For frequently used graphs, register them on-chain to avoid re-sending graph data:

```
ctx.register_graph(graph_pda, bump, &graph_hash, &graph_data)?;  
ctx.execute_registered_graph(graph_pda, ix_data, remaining)?;
```

Registered graphs enable exact per-op fee calculation (no max-charge gap).

Ciphertext Accounts

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Structure

Ciphertext accounts are **regular keypair accounts** (not PDAs). The Encrypt program is the Solana owner.

Field	Size	Description
<code>ciphertext_digest</code>	32	Hash of the encrypted blob (zero until committed)
<code>authorized</code>	32	Who can use this (zero address = public)
<code>network_encryption_public_key</code>	32	FHE key it was encrypted under
<code>fhe_type</code>	1	Type discriminant (EBool=0, EUint64=4, etc.)
<code>status</code>	1	Pending(0) or Verified(1)

Total: 98 bytes data + 2 bytes prefix (discriminator + version) = **100 bytes**.

Account Pubkey = Identifier

The account's Solana pubkey IS the ciphertext identifier. There is no separate `ciphertext_id` field. This means:

- Client generates a keypair for each new ciphertext
- The pubkey is used in events, store lookups, and all references
- Update mode reuses the same account (same pubkey, new digest)

Creating Ciphertexts

Authority Input (`create_input_ciphertext`, disc 1)

User encrypts off-chain → submits to executor with ZK proof → executor verifies → calls this instruction. Status = Verified.

Plaintext (`create_plaintext_ciphertext`, disc 2)

User provides plaintext value directly. Executor encrypts off-chain and commits digest later. Status = Pending until committed.

```
ctx.create_plaintext_typed::<Uint64>(&0u64, ciphertext_account)?;
```

Graph Output (`execute_graph`, disc 4)

Computation outputs are created automatically by `execute_graph`:

- **New account** (empty) → creates Ciphertext with status=Pending
- **Existing account** (has data) → resets digest/status (update mode)

Status Lifecycle

```
Created (by execute_graph) → PENDING → commit_ciphertext → VERIFIED
Created (by create_input)  → VERIFIED (immediately)
Created (by plaintext)     → PENDING → commit_ciphertext → VERIFIED
```

Access Control

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

The authorized Field

Every ciphertext has an `authorized` field (32 bytes):

Value	Meaning
<code>[0; 32]</code> (zero)	Public — anyone can compute on it and decrypt it
<code><pubkey></code>	Only that address can use it (wallet signer or program)

There are no separate guard/permission accounts. The ciphertext IS the access token.

Managing Access

Transfer Authorization

Move authorization from current party to a new party:

```
// Pinocchio
ctx.transfer_ciphertext(ciphertext, new_authorized)?;

// Anchor
ctx.transfer_ciphertext(&ciphertext.to_account_info(),
&new_auth.to_account_info())?;
```

The current authorized party must sign the transaction.

Copy with Different Authorization

Create a copy of the ciphertext authorized to a different party:

```
ctx.copy_ciphertext(  
  source_ciphertext,  
  new_ciphertext,      // empty keypair account  
  new_authorized,  
  false,               // permanent (rent-exempt)  
)?;
```

Set `transient: true` for copies that only live within the current transaction (0 lamports, GC'd after tx).

Make Public

Set authorized to zero — irreversible, anyone can use it:

```
ctx.make_public(ciphertext)?;
```

Idempotent — calling on an already-public ciphertext is a no-op.

CPI Authorization

When a program calls Encrypt via CPI:

- **Signer path:** `caller` is a wallet signer → `authorized` checked against signer pubkey
- **Program path:** `caller` is executable → next account is CPI authority PDA
(`__encrypt_cpi_authority`) → `authorized` checked against program address

Detection is automatic via `caller.executable()`.

Execute Graph

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

How It Works

`execute_graph` (disc 4) processes a computation graph:

1. Parses the graph binary from instruction data
2. Verifies each input ciphertext's `fhe_type` matches the graph
3. Verifies each input's `authorized` matches the caller
4. Charges fees (per input + constant + plaintext input + output + operation)
5. Creates or updates output ciphertext accounts (status=PENDING)
6. Emits `GraphExecutedEvent` for the executor

Instruction Data

`discriminator(1) | graph_data_len(2) | graph_data(N) | num_inputs(2)`

Account Layout

Position	Account	Writable	Signer
0	config	no	no
1	deposit	yes	no
2	caller	no	yes (signer path)
3	network_encryption_key	no	no

Position	Account	Writable	Signer
4	payer	yes	yes
5	event_authority	no	no
6	program	no	no
7..7+N	input ciphertexts	no	no
7+N..7+N+M	output ciphertexts	yes	no

For CPI path: `cpi_authority` is inserted at position 3, shifting subsequent accounts.

Update Mode

Output accounts can be existing ciphertexts:

- If the output account **already has data** → update mode: resets `ciphertext_digest` and `status` to PENDING
- If the output account **is empty** → create mode: creates a new Ciphertext

This means the same account can be used as both input and output (e.g., `yes_count` is read, then updated in the same `execute_graph` call).

Type Verification

The processor verifies each input ciphertext's `fhe_type` matches the graph's Input node `fhe_type`. If they don't match, the transaction fails with `InvalidArgument`.

Using the DSL

Instead of building instruction data manually, use the generated CPI method:

```
// Generated by #[encrypt_fn]:
ctx.cast_vote_graph(yes_ct, no_ct, vote_ct, yes_ct, no_ct)?;
//                ↑inputs↑                ↑outputs↑
```

Decryption

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Request → Respond → Read

Decryption is an async on-chain request/response pattern:

1. Request Decryption

```
let digest = ctx.request_decryption(request_acct, ciphertext)?;  
// Store `digest` in your program state for later verification  
proposal.pending_digest = digest;
```

- Creates a `DecryptionRequest` keypair account
- Stores a `ciphertext_digest` snapshot (stale-value protection)
- Returns the digest — **store it for verification at read time**
- The decryptor detects the event and responds

2. Process (Automatic)

The decryptor:

1. Detects `DecryptionRequestedEvent`
2. Performs threshold MPC decryption (or mock decryption locally)
3. Calls `respond_decryption` to write plaintext bytes into the request account

3. Read Result

```
let req_data = request_acct.try_borrow_data()?;  
let value = read_decrypted_verified::(&req_data,  
&proposal.pending_digest)?;
```

Always verify against the stored digest — if the ciphertext was updated between request and response, the digest won't match and `read_decrypted_verified` returns an error.

4. Close Request

After reading the result, reclaim rent:

```
ctx.close_decryption_request(request_acct, destination)?;
```

DecryptionRequest Account

Field	Size	Description
<code>ciphertext</code>	32	Ciphertext account pubkey
<code>ciphertext_digest</code>	32	Digest snapshot at request time
<code>requester</code>	32	Who requested
<code>fhe_type</code>	1	Type (determines result byte width)
<code>total_len</code>	4	Expected result size
<code>bytes_written</code>	4	Progress (0=pending, ==total_len=complete)
<i>result data</i>	variable	Plaintext bytes (appended after header)

Total: 2 (prefix) + 105 (header) + `byte_width(fhe_type)` bytes.

Type-Safe Reading

Use the SDK helpers:

```
// Pinocchio
use encrypt_pinocchio::accounts::{read_decrypted_verified, ciphertext_digest};

// Read digest from ciphertext account
let ct_data = ciphertext.borrow_unchecked();
let digest = ciphertext_digest(ct_data)?;

// Verify and read result
let value: &u64 = read_decrypted_verified::<Uint64>(req_data, digest)?;
```

Best Practice: Store-and-Verify

```
// At request time:
let digest = ctx.request_decryption(request, ciphertext)?;
state.pending_digest = digest;

// At reveal time:
let value = read_decrypted_verified::<Uint64>(req_data,
&state.pending_digest)?;
```

This pattern protects against the ciphertext being updated between request and reveal.

CPI Framework

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

EncryptCpi Trait

All three framework SDKs implement the same trait:

```
pub trait EncryptCpi {
    type Error;
    type Account<'a>: Clone where Self: 'a;

    fn invoke_execute_graph<'a>(
        &'a self, ix_data: &[u8], accounts: &[Self::Account<'a>],
    ) -> Result<(), Self::Error>;

    fn read_fhe_type<'a>(&'a self, account: Self::Account<'a>) -> Option<u8>;
    fn type_mismatch_error(&self) -> Self::Error;
}
```

EncryptContext

Each framework provides `EncryptContext` :

```
let ctx = EncryptContext {
  encrypt_program,
  config,
  deposit,
  cpi_authority,
  caller_program,
  network_encryption_key,
  payer,
  event_authority,
  system_program,
  cpi_authority_bump,
};
```

The struct is identical across frameworks — only the account types differ:

- **Pinocchio:** `&'a AccountView`
- **Native:** `&'a AccountInfo<'info>`
- **Anchor:** `AccountInfo<'info>`

Available Methods

Method	Description
<code>create_plaintext(fhe_type, bytes, ct)</code>	Create plaintext ciphertext
<code>create_plaintext_typed::<T>(value, ct)</code>	Type-safe plaintext creation
<code>execute_graph(ix_data, remaining)</code>	Execute computation graph
<code>execute_registered_graph(graph_pda, ix_data, remaining)</code>	Execute registered graph
<code>register_graph(pda, bump, hash, data)</code>	Register a reusable graph
<code>transfer_ciphertext(ct, new_authorized)</code>	Transfer authorization
<code>copy_ciphertext(source, new_ct, new_auth, transient)</code>	Copy with different auth
<code>make_public(ct)</code>	Make ciphertext public
<code>request_decryption(request, ct)</code>	Request decryption (returns digest)
<code>close_decryption_request(request, destination)</code>	Close and reclaim rent

DSL Extension Traits

`#[encrypt_fn]` generates extension traits that add graph-specific methods:

```
// Your DSL function:  
#[encrypt_fn]  
fn add(a: EUint64, b: EUint64) -> EUint64 { a + b }  
  
// Call as a method on any EncryptContext:  
ctx.add(input_a, input_b, output)?;
```

The generated method:

1. Verifies each input account's `fhe_type` at runtime
2. Builds the `execute_graph` instruction data
3. Assembles remaining accounts (inputs then outputs)
4. Invokes CPI

Pinocchio

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Dependencies

```
[dependencies]
encrypt-types = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-dsl = { package = "encrypt-solana-dsl", git =
  "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-pinocchio = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha"
}
pinocchio = "0.10"
pinocchio-system = "0.5"
```

Setup EncryptContext

```
use encrypt_pinocchio::EncryptContext;

let ctx = EncryptContext {
  encrypt_program,
  config,
  deposit,
  cpi_authority,
  caller_program,
  network_encryption_key,
  payer,
  event_authority,
  system_program,
  cpi_authority_bump,
};
```

Create Encrypted Zeros

```
use encrypt_types::encrypted::Uint64;

ctx.create_plaintext_typed::<Uint64>(&0u64, ciphertext_acct)?;
```

Execute Graph

```
// Via DSL-generated method (preferred)
ctx.cast_vote_graph(yes_ct, no_ct, vote_ct, yes_ct, no_ct)?;

// Via manual execute_graph
ctx.execute_graph(&ix_data, &[yes_ct, no_ct, vote_ct, yes_ct, no_ct])?;
```

Request Decryption

```
let digest = ctx.request_decryption(request_acct, ciphertext)?;
// Store digest for later verification
```

Read Decrypted Value

```
use encrypt_pinocchio::accounts::{read_decrypted_verified, ciphertext_digest};

let ct_data = unsafe { ciphertext.borrow_unchecked() };
let digest = ciphertext_digest(ct_data)?;
let req_data = unsafe { request_acct.borrow_unchecked() };
let value: &u64 = read_decrypted_verified::<Uint64>(req_data, digest)?;
```

Full Example

See `chains/solana/examples/voting/pinocchio/` for a complete confidential voting program.

Framework Comparison

Consideration	Pinocchio	Native	Anchor	Quasar
CU efficiency	Best	Good	Good	Best
Binary size	Small	Medium	Largest	Smallest
no_std support	Yes	No	No	Yes
Account validation	Manual	Manual	Declarative	Declarative
Zero-copy	Manual	No	No	Built-in

All four SDKs implement the same `EncryptCpi` trait with identical CPI authority seeds and instruction discriminators. Consider [Quasar](#) for declarative validation with Pinocchio-level performance.

Anchor

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Dependencies

```
[dependencies]
encrypt-types = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-dsl = { package = "encrypt-solana-dsl", git =
  "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-anchor = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
anchor-lang = "0.32"
```

Setup EncryptContext

```
use encrypt_anchor::EncryptContext;

let ctx = EncryptContext {
  encrypt_program: ctx.accounts.encrypt_program.to_account_info(),
  config: ctx.accounts.config.to_account_info(),
  deposit: ctx.accounts.deposit.to_account_info(),
  cpi_authority: ctx.accounts.cpi_authority.to_account_info(),
  caller_program: ctx.accounts.caller_program.to_account_info(),
  network_encryption_key:
ctx.accounts.network_encryption_key.to_account_info(),
  payer: ctx.accounts.payer.to_account_info(),
  event_authority: ctx.accounts.event_authority.to_account_info(),
  system_program: ctx.accounts.system_program.to_account_info(),
  cpi_authority_bump,
};
```

Execute Graph

```
let yes_ct = ctx.accounts.yes_ct.to_account_info();
let no_ct = ctx.accounts.no_ct.to_account_info();
let vote_ct = ctx.accounts.vote_ct.to_account_info();
encrypt_ctx.cast_vote_graph(
    yes_ct.clone(), no_ct.clone(), vote_ct,
    yes_ct, no_ct,
)?;
```

Note: Anchor's `AccountInfo` is `Clone`, so you can pass the same account as both input and output.

Request Decryption

```
let digest = encrypt_ctx.request_decryption(
    &ctx.accounts.request_acct.to_account_info(),
    &ctx.accounts.ciphertext.to_account_info(),
)?;
```

Read Decrypted Value

```
use encrypt_anchor::accounts::{read_decrypted_verified, ciphertext_digest};

let ct_data = ctx.accounts.ciphertext.try_borrow_data()?;
let digest = ciphertext_digest(&ct_data)?;
let req_data = ctx.accounts.request_acct.try_borrow_data()?;
let value = read_decrypted_verified::<Uint64>(&req_data, digest)?;
```

Account Structs

Include Encrypt accounts in your Anchor `#[derive(Accounts)]` :

```
#[derive(Accounts)]
pub struct CastVote<'info> {
    #[account(mut)]
    pub proposal: Account<'info, Proposal>,
    pub voter: Signer<'info>,
    /// CHECK: Vote ciphertext
    #[account(mut)]
    pub vote_ct: UncheckedAccount<'info>,
    /// CHECK: Yes count ciphertext
    #[account(mut)]
    pub yes_ct: UncheckedAccount<'info>,
    /// CHECK: No count ciphertext
    #[account(mut)]
    pub no_ct: UncheckedAccount<'info>,
    /// CHECK: Encrypt program
    pub encrypt_program: UncheckedAccount<'info>,
    // ... config, deposit, cpi_authority, etc.
}
```

Full Example

See [chains/solana/examples/confidential-voting-anchor/](#) for a complete program.

Native (solana-program)

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Dependencies

```
[dependencies]
encrypt-types = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-dsl = { package = "encrypt-solana-dsl", git =
  "https://github.com/dwallet-labs/encrypt-pre-alpha" }
encrypt-native = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
solana-program = "4"
```

Setup EncryptContext

```
use encrypt_native::EncryptContext;
```

```
let ctx = EncryptContext {
    encrypt_program,
    config,
    deposit,
    cpi_authority,
    caller_program,
    network_encryption_key,
    payer,
    event_authority,
    system_program,
    cpi_authority_bump,
};
```

Create Encrypted Zeros

```
use encrypt_types::encrypted::Uint64;  
  
ctx.create_plaintext_typed::<Uint64>(&0u64, ciphertext_acct)?;
```

Execute Graph

```
ctx.cast_vote_graph(  
    yes_ct.clone(), no_ct.clone(), vote_ct.clone(),  
    yes_ct.clone(), no_ct.clone(),  
)?;
```

Note: Native `AccountInfo` is `Clone`, so you can clone for duplicate references.

Request Decryption

```
let digest = ctx.request_decryption(request_acct, ciphertext)?;
```

Read Decrypted Value

```
use encrypt_native::accounts::{read_decrypted_verified, ciphertext_digest};  
  
let ct_data = ciphertext.try_borrow_data()?;  
let digest = ciphertext_digest(&ct_data)?;  
let req_data = request_acct.try_borrow_data()?;  
let value = read_decrypted_verified::<Uint64>(&req_data, digest)?;
```

Full Example

See `chains/solana/examples/confidential-voting-native/` for a complete program.

Quasar Framework

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption – all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

The `encrypt-quasar` crate provides a Quasar-native CPI SDK for the Encrypt program. `Quasar` is a zero-copy Solana program framework with alignment-1 Pod types, declarative account validation, and `invoke_signed_unchecked` CPI.

Dependencies

```
[dependencies]
encrypt-types = { git = "https://github.com/nicedwalletlabs/encrypt-pre-alpha"
}
encrypt-dsl = { package = "encrypt-solana-dsl", git =
"https://github.com/nicedwalletlabs/encrypt-pre-alpha" }
encrypt-quasar = { git = "https://github.com/nicedwalletlabs/encrypt-pre-alpha"
}
quasar-lang = { git = "https://github.com/blueshift-gg/quasar", branch =
"master" }
solana-address = { version = "2.4", features = ["curve25519"] }

[lib]
crate-type = ["cdylib", "lib"]
```

EncryptContext

```
use encrypt_quasar::EncryptContext;

let ctx = EncryptContext {
    encrypt_program: self.encrypt_program.to_account_view(),
    config: self.config.to_account_view(),
    deposit: self.deposit.to_account_view(),
    cpi_authority: self.cpi_authority.to_account_view(),
    caller_program: self.caller_program.to_account_view(),
    network_encryption_key: self.network_encryption_key.to_account_view(),
    payer: self.payer.to_account_view(),
    event_authority: self.event_authority.to_account_view(),
    system_program: self.system_program.to_account_view(),
    cpi_authority_bump,
};
```

Convert Quasar owned types (`Signer` , `UncheckedAccount` , `Program<System>`) to `&AccountView` using `.to_account_view()` .

Creating Encrypted Zeros

```
use encrypt_types::encrypted::Uint64;

ctx.create_plaintext_typed::<Uint64>(
    &0u64,
    self.value_ct.to_account_view(),
)?;
```

Executing FHE Graphs

Define graphs with `#[encrypt_fn]` :

```
use encrypt_dsl::prelude::encrypt_fn;
use encrypt_types::encrypted::EUint64;

#[encrypt_fn]
fn increment_graph(value: EUint64) -> EUint64 {
    value + 1
}
```

Execute via CPI (generated `_cpi` function on `EncryptContext`):

```
ctx.increment_graph(
    self.value_ct.to_account_view(), // input
    self.value_ct.to_account_view(), // output (same account for in-place)
)?;
```

Requesting Decryption

```
let digest = ctx.request_decryption(
    self.request_acct.to_account_view(),
    self.ciphertext.to_account_view(),
)?;

// Store digest in your program state for later verification
self.my_state.pending_digest = digest;
```

Reading Decrypted Values

```
use encrypt_quasar::accounts;
use encrypt_types::encrypted::Uint64;

let req_data = unsafe { self.request_acct.to_account_view().borrow_unchecked() };
let value: &u64 = accounts::read_decrypted_verified::<Uint64>(
    req_data,
    &self.my_state.pending_digest,
)?;
```

Quasar Program Patterns

Quasar programs use owned types, explicit discriminators, and `impl` handlers:

```

#[no_std]

use encrypt_quasar::EncryptContext;
use quasar_lang::prelude::*;

declare_id!("...");

#[program]
mod my_program {
    use super::*;

    #[instruction(discriminator = 0)]
    pub fn create(ctx: Ctx<Create>, /* args */) -> Result<(), ProgramError> {
        ctx.accounts.create(/* args */)
    }
}

#[derive(Accounts)]
pub struct Create {
    #[account(init, payer = payer, seeds = MyState::seeds(state_id), bump)]
    pub state: Account<MyState>,

    // Encrypt program accounts
    pub encrypt_program: UncheckedAccount,
    pub config: UncheckedAccount,
    #[account(mut)]
    pub deposit: UncheckedAccount,
    pub cpi_authority: UncheckedAccount,
    pub caller_program: UncheckedAccount,
    pub network_encryption_key: UncheckedAccount,
    #[account(mut)]
    pub payer: Signer,
    pub event_authority: UncheckedAccount,
    pub system_program: Program<System>,
}

```

Performance

Quasar produces the smallest binaries and near-lowest CU usage of any declarative framework:

Consideration	Pinocchio	Native	Anchor	Quasar
CU efficiency	Best	Good	Good	Best
Binary size	Small	Medium	Largest	Smallest
no_std support	Yes	No	No	Yes
Account validation	Manual	Manual	Declarative	Declarative
Zero-copy	Manual	No	No	Built-in

All four SDKs use the same CPI authority seed (`b"__encrypt_cpi_authority"`), the same instruction discriminators, and the same `EncryptCpi` trait. Programs built with any SDK are fully interoperable.

Test Framework

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Overview

`encrypt-solana-test` provides three testing modes:

- **LiteSVM** (`EncryptTestContext`) — fast in-process e2e tests
- **solana-program-test** (`ProgramTestEncryptContext`) — official Solana runtime e2e tests
- **Mollusk** — single-instruction unit tests with pre-built account data

[dev-dependencies]

```
encrypt-solana-test = { git = "https://github.com/dwallet-labs/encrypt-pre-alpha" }
```

Architecture

```

encrypt-dev (chains/solana/dev/) – production-safe, no test deps
├── SolanaRuntime                # Production (send_transaction,
get_account_data, ...)
├── TestRuntime                  # Dev/test (adds airdrop, deploy_program)
├── InProcessTestRuntime         # In-process only (adds set_account,
advance_slot)
└── EncryptTxBuilder<R>         # Tx construction for all Encrypt
instructions

encrypt-solana-test (chains/solana/test/)
├── LiteSvmRuntime               # LiteSVM backend (InProcessTestRuntime)
├── ProgramTestRuntime           # solana-program-test backend
(InProcessTestRuntime)
├── EncryptTestHarness<R>       # Wraps TxBuilder + MockComputeEngine +
store + work queue
├── EncryptTestContext           # Ergonomic LiteSVM wrapper
├── ProgramTestEncryptContext    # Ergonomic solana-program-test wrapper
└── mollusk helpers              # Account builders, discriminators, setup

```

`encrypt-dev` has no test framework dependencies — only the runtime trait hierarchy and `EncryptTxBuilder`. Test runtimes and harness live in `encrypt-solana-test`.

EncryptTestContext

```

use encrypt_solana_test::litesvm::EncryptTestContext;
use encrypt_types::encrypted::Uint64;

#[test]
fn test_my_program() {
    let mut ctx = EncryptTestContext::new_default();
    let user = ctx.new_funded_keypair();

    let a = ctx.create_input::<Uint64>(10, &user.pubkey());
    let b = ctx.create_input::<Uint64>(32, &user.pubkey());

    let graph = my_add_graph();
    let outputs = ctx.execute_and_commit(&graph, &[a, b], 1, &[], &user);

    let result = ctx.decrypt::<Uint64>(&outputs[0], &user);
    assert_eq!(result, 42);
}

```

How It Works

1. **LiteSVM** runs in-process — no external validator needed

2. A **local authority keypair** signs `commit_ciphertext` and `respond_decryption`
3. An **in-memory CiphertextStore** tracks all ciphertext digests
4. `execute_and_commit()` calls `execute_graph` on-chain, then evaluates the graph off-chain using `MockComputeEngine` and commits results
5. `decrypt()` calls `request_decryption` on-chain, then decrypts and responds

All off-chain processing happens synchronously — no event polling needed.

API Reference

Method	Description
<code>new(elf_path)</code>	Create context with custom program path
<code>new_default()</code>	Create with default build output path
<code>new_funded_keypair()</code>	Create and fund a new keypair (10 SOL)
<code>create_input::<T>(value, authorized)</code>	Create verified encrypted input (authority-driven)
<code>create_plaintext::<T>(value, creator)</code>	Create plaintext ciphertext (user-signed)
<code>execute_and_commit(graph, inputs, n_outputs, existing_outputs, caller)</code>	Execute + commit in one call
<code>decrypt::<T>(ct_pubkey, requester)</code>	Decrypt and return plaintext value
<code>decrypt_from_store(ct_pubkey)</code>	Read value from mock store (no on-chain request)
<code>deploy_program(elf_path)</code>	Deploy an additional program, returns ID
<code>deploy_program_at(id, elf_path)</code>	Deploy at a specific address
<code>cpi_authority_for(caller_program)</code>	Derive CPI authority PDA for a program
<code>send_transaction(ixs, signers)</code>	Sign and send a transaction
<code>get_account_data(pubkey)</code>	Read raw account data
<code>register_ciphertext(pubkey)</code>	Register CPI-created ciphertext in the store
<code>enqueue_graph_execution(graph, inputs, outputs)</code>	Enqueue CPI-triggered graph for processing

Method	Description
<code>process_pending()</code>	Process all queued graph executions and decryptions
<code>program_id()</code> / <code>config_pda()</code> / <code>deposit_pda()</code> / etc.	Access Encrypt program PDAs

Testing CPI Programs (e2e)

For programs that call the Encrypt program via CPI (like the voting examples):

```
use encrypt_solana_test::litesvm::EncryptTestContext;
use encrypt_types::encrypted::{Bool, Uint64};

#[test]
fn test_voting_lifecycle() {
    let mut ctx = EncryptTestContext::new_default();

    // Deploy your program
    let program_id = ctx.deploy_program("path/to/your_program.so");
    let (cpi_authority, cpi_bump) = ctx.cpi_authority_for(&program_id);

    // Create proposal (CPI creates ciphertexts)
    // ... send create_proposal transaction ...

    // Register CPI-created ciphertexts in the harness store
    ctx.register_ciphertext(&yes_ct_pubkey);
    ctx.register_ciphertext(&no_ct_pubkey);

    // Cast vote (CPI to execute_graph)
    // ... send cast_vote transaction ...

    // Enqueue the graph execution for off-chain processing
    ctx.enqueue_graph_execution(&graph_data, &inputs, &outputs);
    ctx.process_pending();

    // Re-register updated ciphertexts
    ctx.register_ciphertext(&yes_ct_pubkey);
    ctx.register_ciphertext(&no_ct_pubkey);

    // Verify results from the mock store
    let yes = ctx.decrypt_from_store(&yes_ct_pubkey);
    assert_eq!(yes, 1);
}
```

Testing Update Mode

For programs that reuse ciphertext accounts:

```
let yes_ct = ctx.create_input::<Uint64>(0, &program_id);
let no_ct = ctx.create_input::<Uint64>(0, &program_id);
let vote = ctx.create_input::<Bool>(1, &program_id);

// Pass yes_ct and no_ct as both inputs and existing outputs (update mode)
let outputs = ctx.execute_and_commit(
    &cast_vote_graph(),
    &[yes_ct, no_ct, vote],
    0, // no new outputs
    &[yes_ct, no_ct], // existing outputs (update mode)
    &caller,
);
```

Mollusk Mode

For single-instruction unit tests:

```
use encrypt_solana_test::mollusk::*;

let (mollusk, program_id) = setup();
let ct_data = build_ciphertext_data(&digest, &authorized, &nk, fhe_type,
status);

let result = mollusk.process_instruction(
    &Instruction::new_with_bytes(program_id, &ix_data, accounts),
    &[(key, program_account(&program_id, ct_data))],
);
assert!(result.program_result.is_ok());
```

Mollusk is best for testing individual instructions in isolation — signer checks, discriminator validation, authority verification, digest matching, etc.

Mock vs Real FHE

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Mock Mode (Pre-Alpha)

The pre-alpha environment uses **mock FHE** — operations are performed as plaintext arithmetic with keccak256 digests. This means:

- `add(encrypt(10), encrypt(32)) → encrypt(42)` — correct result, no actual encryption
- Graph evaluation is instantaneous (no FHE overhead)
- Decryption is trivial
- **No security** — values are not encrypted on-chain

Your program logic, computation graphs, and client code all work identically in mock and real mode. Only the off-chain executor differs.

Real REFHE Mode (Coming Soon)

In production, the executor will use the REFHE library:

- Actual homomorphic encryption on ciphertext blobs
- Decryption requires threshold MPC (multiple decryptor nodes)
- Full privacy — values are never visible on-chain

No code changes required — the same `#[encrypt_fn]` graphs, CPI calls, and gRPC client calls work in both modes.

Examples

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Complete example programs demonstrating Encrypt on Solana. Each example includes on-chain programs in multiple frameworks (Pinocchio, Native, Anchor, Quasar), tests, and where applicable a React frontend that runs against the pre-alpha executor on devnet.

All examples connect to the pre-alpha environment automatically:

Resource	Endpoint
Encrypt gRPC	<code>https://pre-alpha-dev-1.encrypt.ika-network.net:443</code>
Solana Network	Devnet (<code>https://api.devnet.solana.com</code>)

Confidential Counter

An always-encrypted counter. Increment and decrement happen via FHE – the on-chain program never sees the plaintext. Demonstrates the core Encrypt patterns: `#[encrypt_fn]` , CPI via `EncryptContext` , and the store-and-verify digest pattern for decryption.

Covers: FHE graphs, in-place ciphertext updates, polling for executor completion, React frontend with wallet adapter.

Encrypted Coin Flip

Provably fair coin flip with on-chain escrow. Two sides commit encrypted values, the executor computes XOR via FHE, and the winner receives 2x from escrow. Neither side can see the other’s value before committing.

Covers: XOR-based fairness, escrow pattern, player-vs-house architecture with automated Bun backend, full-stack React app.

Confidential Voting

Encrypted voting where individual votes are hidden but the tally is computed via FHE. Voters cast encrypted yes/no votes (EBool), and the program conditionally increments encrypted counters using a Select operation. Only the authority can reveal final tallies.

Covers: Conditional FHE logic (if/else → Select), multi-output graphs, double-vote prevention via VoteRecord PDA, multi-wallet URL sharing, E2E demos in Rust + TypeScript (web3.js, kit, gill).

Encrypted ACL

An on-chain access control list where permissions are stored as encrypted 64-bit bitmasks. Grant, revoke, and check operations use FHE bitwise operations (OR, AND). Nobody can see what permissions are set.

Covers: Multiple FHE graphs in one program, inverse mask pattern for revocation, separate state accounts with independent decryption flows, admin-gated vs public operations.

PC-Token (Confidential Performant Token)

A composable confidential token program — Anza's [P-Token](#) architecture rebuilt with Encrypt FHE. All balances and transfer amounts are encrypted; nobody can see how many tokens any account holds or how much is being transferred. Follows P-Token's COption flags, AccountState enum, instruction discriminators, and freeze/thaw patterns.

Covers: Encrypted balances (EUInt64), client-encrypted transfer amounts, conditional FHE logic (insufficient funds → silent no-op), approve/transfer_from delegation for composability, freeze/thaw, vault-backed wrap/unwrap. Demonstrates how existing Solana token standards can be made confidential with Encrypt.

PC-Swap (Confidential UniV2 AMM)

A confidential constant-product AMM that composes with PC-Token. All reserves, swap amounts, and LP positions are encrypted. The swap formula ($x \times y = k$), fee calculation

(0.3%), slippage protection, and LP ownership checks all run in the encrypted domain via FHE. The only public value is the price, published as a public ciphertext readable by anyone via gRPC.

Covers: FHE arithmetic (multiply, divide) on EUint128, composability (AMM CPI into Encrypt for swap math), public ciphertexts (`make_public` for price oracle), LP position enforcement in FHE graphs, self-settling no-ops for invalid swaps.

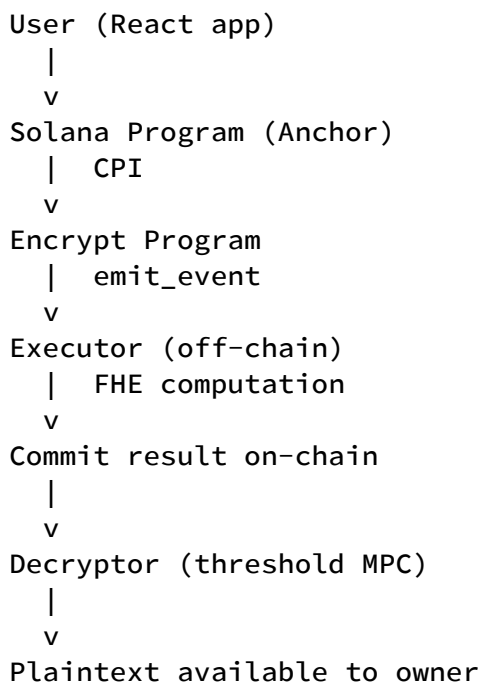
Confidential Counter: Overview

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

What We’re Building

A Solana counter whose value is always encrypted. Increment and decrement happen via FHE – the on-chain program never sees the plaintext. Only the owner can request decryption to reveal the current count.

Architecture



1. The **Anchor program** stores a `Counter` PDA with a reference to a ciphertext account.

2. When you call `increment`, the program issues a CPI to the Encrypt program with a precompiled FHE graph (`value + 1`). No computation happens on-chain.
3. An off-chain **executor** picks up the event, evaluates the graph using FHE, and commits the result back to the same ciphertext account.
4. To read the value, the owner calls `request_value_decryption` . A threshold **decryptor** network processes the request and writes the plaintext into a decryption request account.
5. The owner calls `reveal_value` to copy the verified plaintext into the counter state.

What You'll Learn

- Writing FHE graphs with `#[encrypt_fn]`
- CPI to the Encrypt program via `EncryptContext`
- The store-and-verify digest pattern for decryption
- Building a React frontend that polls for executor/decryptor completion

Prerequisites

- Rust (edition 2024, nightly or stable with Solana toolchain)
- Solana CLI + Platform Tools v1.54
- Anchor framework
- Bun (for the React frontend)

The executor and gRPC server are running on the pre-alpha environment at <https://pre-alpha-dev-1.encrypt.ika-network.net:443> – no local setup needed.

Confidential Counter: Building the Program

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

1. Cargo.toml

```
[package]
name = "confidential-counter-anchor"
edition.workspace = true

[dependencies]
encrypt-types = { workspace = true }
encrypt-dsl = { package = "encrypt-solana-dsl", path = "../../../../../program-sdk/dsl" }
encrypt-anchor = { workspace = true }
anchor-lang = { workspace = true }

[lib]
crate-type = ["cdylib", "lib"]
```

Three Encrypt crates:

- `encrypt-types` – FHE type definitions (`EUint64` , `Uint64` , etc.)
- `encrypt-dsl` (aliased from `encrypt-solana-dsl`) – the `#[encrypt_fn]` macro that generates FHE graphs + Solana CPI glue
- `encrypt-anchor` – `EncryptContext` struct and account helpers for Anchor

2. FHE Graphs

```
use encrypt_dsl::prelude::encrypt_fn;
use encrypt_types::encrypted::EUint64;

#[encrypt_fn]
fn increment_graph(value: EUint64) -> EUint64 {
    value + 1
}

#[encrypt_fn]
fn decrement_graph(value: EUint64) -> EUint64 {
    value - 1
}
```

The `#[encrypt_fn]` macro does two things at compile time:

1. **Generates a graph function** (`increment_graph() -> Vec<u8>`) that returns a serialized computation graph in the Encrypt binary format. The graph has one `Input` node (the encrypted value), one `Constant` node (the literal `1`), one `Op` node (add or subtract), and one `Output` node.
2. **Generates a CPI extension trait** (`IncrementGraphCpi`) with a blanket implementation on `EncryptContext`. This gives you a method like `encrypt_ctx.increment_graph(input_ct, output_ct)` that builds and executes the `execute_graph` CPI to the Encrypt program.

The graph is embedded in the program binary. When the CPI fires, the Encrypt program emits an event that the off-chain executor picks up. The executor deserializes the graph, evaluates each node using real FHE operations, and commits the result ciphertext on-chain.

Key point: the same ciphertext account can be both input and output (in-place update). That's how `increment` works – the counter value is updated without creating new accounts.

3. Counter State

```
#[account]
#[derive(InitSpace)]
pub struct Counter {
    pub authority: Pubkey,           // who can increment/decrypt
    pub counter_id: [u8; 32],        // unique ID, used as PDA seed
    pub value: [u8; 32],             // pubkey of the ciphertext account
    pub pending_digest: [u8; 32],    // digest from request_decryption
    pub revealed_value: u64,         // plaintext after decryption
    pub bump: u8,                   // PDA bump
}
```

- `value` stores the **pubkey** of a ciphertext account, not the ciphertext itself. Ciphertext accounts are owned by the Encrypt program.
- `pending_digest` is the store-and-verify pattern: when you request decryption, the Encrypt program returns a digest of the ciphertext at that moment. You store it and later verify the decryption result matches.
- `revealed_value` holds the plaintext once decrypted. Until then it's 0.

4. create_counter

```
pub fn create_counter(
    ctx: Context<CreateCounter>,
    counter_id: [u8; 32],
    initial_value_id: [u8; 32],
) -> Result<()> {
    let ctr = &mut ctx.accounts.counter;
    ctr.authority = ctx.accounts.authority.key();
    ctr.counter_id = counter_id;
    ctr.value = initial_value_id;
    ctr.pending_digest = [0u8; 32];
    ctr.revealed_value = 0;
    ctr.bump = ctx.bumps.counter;
    Ok(())
}
```

The caller creates an encrypted zero off-chain (via the gRPC `CreateInput` RPC), which produces a ciphertext account on Solana. The caller passes that account's pubkey as `initial_value_id`. The counter PDA just stores the reference.

Account constraints:

```
#[derive(Accounts)]
#[instruction(counter_id: [u8; 32])]
pub struct CreateCounter<'info> {
    #[account(
        init,
        payer = payer,
        space = 8 + Counter::INIT_SPACE,
        seeds = [b"counter", counter_id.as_ref()],
        bump,
    )]
    pub counter: Account<'info, Counter>,
    pub authority: Signer<'info>,
    #[account(mut)]
    pub payer: Signer<'info>,
    pub system_program: Program<'info, System>,
}
```

The PDA is seeded by `[b"counter", counter_id]`. The `counter_id` is an arbitrary 32-byte value chosen by the caller (typically a random keypair's pubkey bytes).

5. increment / decrement

```
pub fn increment(ctx: Context<Increment>, cpi_authority_bump: u8) -> Result<()>
{
    let encrypt_ctx = EncryptContext {
        encrypt_program: ctx.accounts.encrypt_program.to_account_info(),
        config: ctx.accounts.config.to_account_info(),
        deposit: ctx.accounts.deposit.to_account_info(),
        cpi_authority: ctx.accounts.cpi_authority.to_account_info(),
        caller_program: ctx.accounts.caller_program.to_account_info(),
        network_encryption_key:
ctx.accounts.network_encryption_key.to_account_info(),
        payer: ctx.accounts.payer.to_account_info(),
        event_authority: ctx.accounts.event_authority.to_account_info(),
        system_program: ctx.accounts.system_program.to_account_info(),
        cpi_authority_bump,
    };

    let value_ct = ctx.accounts.value_ct.to_account_info();
    encrypt_ctx.increment_graph(value_ct.clone(), value_ct)?;

    Ok(())
}
```

Step by step:

1. Build an `EncryptContext` with all the Encrypt program accounts. These are infrastructure accounts (config, deposit, CPI authority PDA, network encryption key, event authority). Every Encrypt CPI needs them.
2. Call `encrypt_ctx.increment_graph(input, output)` . This method was generated by `# [encrypt_fn]` . It:
 - Serializes the graph bytes
 - Verifies the input ciphertext's `fhe_type` matches `EUint64`
 - Builds an `execute_graph` CPI instruction
 - Invokes the Encrypt program
3. The input and output are the **same account** (`value_ct`). This is an in-place update – the executor will overwrite the ciphertext with the computed result.

The `cpi_authority_bump` is the bump for the PDA `["__encrypt_cpi_authority"]` derived from your program ID. The Encrypt program uses this to verify the CPI came from an authorized program.

`decrement` is identical except it calls `encrypt_ctx.decrement_graph(...)` .

The Increment accounts struct shows the full set of accounts needed for any Encrypt CPI:

```
#[derive(Accounts)]
pub struct Increment<'info> {
    #[account(mut)]
    pub counter: Account<'info, Counter>,
    /// CHECK: Value ciphertext account
    #[account(mut)]
    pub value_ct: UncheckedAccount<'info>,
    /// CHECK: Encrypt program
    pub encrypt_program: UncheckedAccount<'info>,
    /// CHECK: Encrypt config
    pub config: UncheckedAccount<'info>,
    /// CHECK: Encrypt deposit
    #[account(mut)]
    pub deposit: UncheckedAccount<'info>,
    /// CHECK: CPI authority PDA
    pub cpi_authority: UncheckedAccount<'info>,
    /// CHECK: Caller program
    pub caller_program: UncheckedAccount<'info>,
    /// CHECK: Network encryption key
    pub network_encryption_key: UncheckedAccount<'info>,
    #[account(mut)]
    pub payer: Signer<'info>,
    /// CHECK: Event authority PDA
    pub event_authority: UncheckedAccount<'info>,
    pub system_program: Program<'info, System>,
}
```

6. request_value_decryption

```
pub fn request_value_decryption(
    ctx: Context<RequestValueDecryption>,
    cpi_authority_bump: u8,
) -> Result<()> {
    let ctr = &ctx.accounts.counter;
    require!(
        ctr.authority == ctx.accounts.payer.key(),
        CounterError::Unauthorized
    );

    let encrypt_ctx = EncryptContext { /* ... same fields ... */ };

    let digest = encrypt_ctx.request_decryption(
        &ctx.accounts.request_acct.to_account_info(),
        &ctx.accounts.ciphertext.to_account_info(),
    );

    let ctr = &mut ctx.accounts.counter;
    ctr.pending_digest = digest;

    Ok(())
}
```

`request_decryption` does two things:

1. Creates a `DecryptionRequest` account (keypair account, passed as a signer)
2. Returns a `[u8; 32]` digest – a snapshot of the ciphertext’s current state

You **must** store this digest. It prevents stale-value attacks: if someone modifies the ciphertext between your request and the decryptor’s response, the digest won’t match and `reveal_value` will fail.

The decryption request account is a keypair account (not a PDA). The caller generates a fresh keypair and passes it as a signer. This avoids seed conflicts when making multiple decryption requests.

7. reveal_value

```
pub fn reveal_value(ctx: Context<RevealValue>) -> Result<()> {
    let ctr = &mut ctx.accounts.counter;
    require!(
        ctr.authority == ctx.accounts.authority.key(),
        CounterError::Unauthorized
    );

    let expected_digest = &ctr.pending_digest;

    let req_data = ctx.accounts.request_acct.try_borrow_data()?;
    use encrypt_types::encrypted::Uint64;
    let value = encrypt_anchor::accounts::read_decrypted_verified::<Uint64>(
        &req_data,
        expected_digest,
    )
    .map_err(|_| CounterError::DecryptionNotComplete)?;

    ctr.revealed_value = *value;
    Ok(())
}
```

`read_decrypted_verified::<Uint64>` does three checks:

1. The decryption request is complete (decryptor has written the plaintext)
2. The ciphertext digest in the request matches `expected_digest`
3. The FHE type matches `uint64` (the plaintext type corresponding to `EUint64`)

If all checks pass, it returns a reference to the plaintext value. The `uint64` type parameter is the **plaintext** counterpart of `EUint64`.

The `RevealValue` accounts are minimal – no Encrypt CPI needed:

```
#[derive(Accounts)]
pub struct RevealValue<'info> {
    #[account(mut)]
    pub counter: Account<'info, Counter>,
    /// CHECK: Completed decryption request account
    pub request_acct: UncheckedAccount<'info>,
    pub authority: Signer<'info>,
}
```

Error Codes

```
#[error_code]
pub enum CounterError {
    #[msg("Unauthorized")]
    Unauthorized,
    #[msg("Decryption not complete")]
    DecryptionNotComplete,
}
```

Confidential Counter: Testing

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

1. Unit Tests (Graph Logic)

Unit tests verify the FHE graph produces correct results using a mock evaluator. No SBF build or Solana runtime needed.

```
cargo test -p confidential-counter-anchor --lib
```

The tests use a `run_mock` helper that walks the graph nodes and evaluates them with mock arithmetic (operating on plaintext values encoded as mock digests):

```

#[test]
fn increment_from_zero() {
    let r = run_mock(increment_graph, &[0], &[FheType::EUint64]);
    assert_eq!(r[0], 1, "0 + 1 = 1");
}

#[test]
fn increment_from_ten() {
    let r = run_mock(increment_graph, &[10], &[FheType::EUint64]);
    assert_eq!(r[0], 11, "10 + 1 = 11");
}

#[test]
fn decrement_from_ten() {
    let r = run_mock(decrement_graph, &[10], &[FheType::EUint64]);
    assert_eq!(r[0], 9, "10 - 1 = 9");
}

#[test]
fn graph_shapes() {
    let inc = increment_graph();
    let pg = parse_graph(&inc).unwrap();
    assert_eq!(pg.header().num_inputs(), 1);
    assert_eq!(pg.header().num_outputs(), 1);
}

```

2. LiteSVM Integration Tests (E2E)

Full lifecycle tests using LiteSVM – a lightweight Solana runtime that runs in-process. Tests deploy the SBF binary, create ciphertexts, execute graphs, and verify results.

```

# Build SBF first
just build-sbf-examples

# Run LiteSVM tests
cargo test -p confidential-counter-anchor --test litesvm

```

The test uses `EncryptTestContext` which bundles a LiteSVM instance with the Encrypt program pre-deployed and a mock compute engine:

```
#[test]
fn test_increment() {
    let mut ctx = EncryptTestContext::new_default();
    let (program_id, cpi_authority, cpi_bump) = setup_anchor_program(&mut ctx);
    let authority = ctx.new_funded_keypair();

    // Create encrypted zero
    let value_ct = ctx.create_input::<Uint64>(0, &program_id);

    // Create counter PDA
    // ... send create_counter ix ...

    // Increment via CPI
    // ... send increment ix ...

    // Simulate executor: evaluate graph + commit result
    let graph = increment_graph();
    ctx.enqueue_graph_execution(&graph, &[value_ct], &[value_ct]);
    ctx.process_pending();
    ctx.register_ciphertext(&value_ct);

    // Verify
    let result = ctx.decrypt_from_store(&value_ct);
    assert_eq!(result, 1);
}
```

Key `EncryptTestContext` methods:

- `create_input::<Uint64>(value, program_id)` – creates a ciphertext account
- `enqueue_graph_execution(graph, inputs, outputs)` – queues a graph for mock evaluation
- `process_pending()` – runs the mock FHE engine
- `register_ciphertext(pubkey)` – syncs the on-chain account with the mock store
- `decrypt_from_store(pubkey)` – returns the plaintext value

3. Mollusk Instruction-Level Tests

Mollusk tests individual instructions in isolation without CPI. Useful for testing `reveal_value` logic (authorization checks, digest verification) without needing the full Encrypt program.

```
just build-sbf-examples
cargo test -p confidential-counter-anchor --test mollusk
```

Tests construct raw account data and verify instruction behavior:

```

#[test]
fn test_reveal_value_success() {
    let (mollusk, pid) = setup();
    let authority = Pubkey::new_unique();
    let digest = [0xABu8; 32];

    let counter_data = build_anchor_counter_with_digest(
        &authority, &[1u8; 32], &Pubkey::new_unique(), &digest, 0,
    );
    let request_data = build_decryption_request_data(&digest, 42);

    let result = mollusk.process_instruction(/* ... */);
    assert!(result.program_result.is_ok());
    // Check revealed_value == 42
}

#[test]
fn test_reveal_value_rejects_wrong_authority() { /* ... */ }

#[test]
fn test_reveal_value_rejects_digest_mismatch() { /* ... */ }

```

4. Running All Example Tests

```

# Everything (build + all test types)
just test-examples

# Just LiteSVM e2e
just test-examples-litesvm

# Just Mollusk
just test-examples-mollusk

# Just program-test (solana-program-test runtime)
just test-examples-program-test

```

Confidential Counter: React Frontend

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

1. Project Setup

The frontend uses Vite + React + Solana wallet adapter.

```
cd chains/solana/examples/counter/react
bun install
```

Dependencies in `package.json`:

```
{
  "dependencies": {
    "@solana/wallet-adapter-base": "^0.9.23",
    "@solana/wallet-adapter-react": "^0.15.35",
    "@solana/wallet-adapter-react-ui": "^0.9.35",
    "@solana/wallet-adapter-wallets": "^0.19.32",
    "@solana/web3.js": "^1.95.3",
    "react": "^18.3.1",
    "react-dom": "^18.3.1"
  }
}
```

Entry point (`main.tsx`) wraps the app with Solana providers:

```
const RPC_URL = "https://api.devnet.solana.com";

function Root() {
  const wallets = useMemo(() => [], []);
  return (
    <ConnectionProvider endpoint={RPC_URL}>
      <WalletProvider wallets={wallets} autoConnect>
        <WalletModalProvider>
          <App />
        </WalletModalProvider>
      </WalletProvider>
    </ConnectionProvider>
  );
}
```

2. Program IDs

[illegible]

Update these to match your deployed program IDs.

3. PDA Derivation

All Encrypt infrastructure PDAs derive from known seeds:

```
function deriveEncryptPdas(payer: PublicKey) {
    const [configPda] = findPda([Buffer.from("encrypt_config")],
    ENCRYPT_PROGRAM);
    const [eventAuthority] = findPda([Buffer.from("__event_authority")],
    ENCRYPT_PROGRAM);
    const [depositPda, depositBump] = findPda(
        [Buffer.from("encrypt_deposit"), payer.toBuffer()], ENCRYPT_PROGRAM
    );
    const networkKey = Buffer.alloc(32, 0x55);
    const [networkKeyPda] = findPda(
        [Buffer.from("network_encryption_key"), networkKey], ENCRYPT_PROGRAM
    );
    const [cpiAuthority, cpiBump] = findPda(
        [Buffer.from("__encrypt_cpi_authority")], COUNTER_PROGRAM
    );
    return { configPda, eventAuthority, depositPda, depositBump, networkKeyPda,
    cpiAuthority, cpiBump };
}
```

The `cpiAuthority` is derived from the **counter program** (not the Encrypt program). Each program that CPIs into Encrypt has its own CPI authority PDA.

4. Encrypt CPI Account List

Every Encrypt CPI needs these accounts in order:

```
function encryptCpiAccounts(payer: PublicKey, enc: ReturnType<typeof
deriveEncryptPdas>) {
  return [
    { pubkey: ENCRYPT_PROGRAM, isSigner: false, isWritable: false },
    { pubkey: enc.configPda, isSigner: false, isWritable: true },
    { pubkey: enc.depositPda, isSigner: false, isWritable: true },
    { pubkey: enc.cpiAuthority, isSigner: false, isWritable: false },
    { pubkey: COUNTER_PROGRAM, isSigner: false, isWritable: false },
    { pubkey: enc.networkKeyPda, isSigner: false, isWritable: false },
    { pubkey: payer, isSigner: true, isWritable: true },
    { pubkey: enc.eventAuthority, isSigner: false, isWritable: false },
    { pubkey: SystemProgram.programId, isSigner: false, isWritable: false },
  ];
}
```

5. Polling Pattern

After a CPI, the executor processes the FHE computation off-chain. The frontend polls until the ciphertext account is verified:

```
async function pollUntil(  
  connection: any, account: PublicKey,  
  check: (data: Buffer) => boolean,  
  timeoutMs = 120_000, intervalMs = 1_000  
) : Promise<Buffer> {  
  const start = Date.now();  
  while (Date.now() - start < timeoutMs) {  
    try {  
      const info = await connection.getAccountInfo(account);  
      if (info && check(info.data as Buffer)) return info.data as Buffer;  
    } catch {}  
    await new Promise((r) => setTimeout(r, intervalMs));  
  }  
  throw new Error("Timeout waiting for executor");  
}  
  
// Ciphertext is verified when status byte (offset 99) == 1  
const isVerified = (d: Buffer) => d.length >= 100 && d[99] === 1;  
  
// Decryption is complete when written_bytes == total_bytes and total > 0  
const isDecrypted = (d: Buffer) => {  
  if (d.length < 107) return false;  
  const total = d.readUInt32LE(99);  
  const written = d.readUInt32LE(103);  
  return written === total && total > 0;  
};
```

6. Create Counter Flow

```
const handleInitialize = useCallback(async () => {
  await ensureDeposit(); // create deposit account if needed
  const enc = getEnc();
  const id = Buffer.from(Keypair.generate().publicKey.toBytes());
  const [pda, bump] = findPda([Buffer.from("counter"), id], COUNTER_PROGRAM);
  const valueKeypair = Keypair.generate();

  await sendTx(
    [new TransactionInstruction({
      programId: COUNTER_PROGRAM,
      data: Buffer.concat([Buffer.from([0, bump, enc.cpiBump]), id]),
      keys: [
        { pubkey: pda, isSigner: false, isWritable: true },
        { pubkey: wallet.publicKey, isSigner: true, isWritable: false },
        { pubkey: valueKeypair.publicKey, isSigner: true, isWritable: true },
        ...encryptCpiAccounts(wallet.publicKey, enc),
      ],
    })],
    [valueKeypair]
  );

  setCounterPda(pda);
  setValueCt(valueKeypair.publicKey);
}, [/* deps */]);
```

The `valueKeypair` is a fresh keypair whose public key becomes the ciphertext account address. The Encrypt program creates this account during the CPI. The keypair must sign the transaction.

7. Increment / Decrement Flow

```
const handleOp = useCallback(async (opcode: 1 | 2, label: string) => {
  const enc = getEnc();
  await sendTx([new TransactionInstruction({
    programId: COUNTER_PROGRAM,
    data: Buffer.from([opcode, enc.cpiBump]),
    keys: [
      { pubkey: counterPda, isSigner: false, isWritable: true },
      { pubkey: valueCt, isSigner: false, isWritable: true },
      ...encryptCpiAccounts(wallet.publicKey, enc),
    ],
  })]);

  // Wait for executor to process the FHE computation
  await pollUntil(connection, valueCt, isVerified, 60_000);
}, [/* deps */]);
```

After sending the transaction, poll the ciphertext account until `isVerified` returns true. The executor typically processes within a few seconds on devnet.

8. Decrypt + Reveal Flow

Decryption is a two-step process:

```
const handleDecrypt = useCallback(async () => {
  const enc = getEnc();
  const reqKeypair = Keypair.generate();

  // Step 1: Request decryption
  await sendTx(
    [new TransactionInstruction({
      programId: COUNTER_PROGRAM,
      data: Buffer.from([3, enc.cpiBump]),
      keys: [
        { pubkey: counterPda, isSigner: false, isWritable: true },
        { pubkey: reqKeypair.publicKey, isSigner: true, isWritable: true },
        { pubkey: valueCt, isSigner: false, isWritable: false },
        ...encryptCpiAccounts(wallet.publicKey, enc),
      ],
    })],
    [reqKeypair]
  );

  // Step 2: Wait for decryptor
  await pollUntil(connection, reqKeypair.publicKey, isDecrypted);

  // Step 3: Reveal (copy plaintext into counter state)
  await sendTx([new TransactionInstruction({
    programId: COUNTER_PROGRAM,
    data: Buffer.from([4]),
    keys: [
      { pubkey: counterPda, isSigner: false, isWritable: true },
      { pubkey: reqKeypair.publicKey, isSigner: false, isWritable: false },
      { pubkey: wallet.publicKey, isSigner: true, isWritable: false },
    ],
  })]);

  // Read the revealed value from counter PDA
  const data = (await connection.getAccountInfo(counterPda))!.data as Buffer;
  const revealed = data.readBigUInt64LE(129);
  setDisplayValue(revealed.toString());
}, [/* deps */]);
```

The `reqKeypair` is a fresh keypair for the decryption request account. After the decryptor writes the result, `reveal_value` (opcode 4) copies the verified plaintext into `counter.revealed_value`.

9. Running on Devnet

The app connects to Solana devnet and the pre-alpha executor automatically. No local validator or executor setup is needed.

```
cd chains/solana/examples/counter/react  
bun install  
bun dev
```

Open <http://localhost:5173> , connect a wallet (e.g. Phantom set to devnet), airdrop SOL, and create a counter.

Encrypted Coin Flip

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Provably fair coin flip with on-chain escrow, built on Encrypt + Solana.

What you'll learn

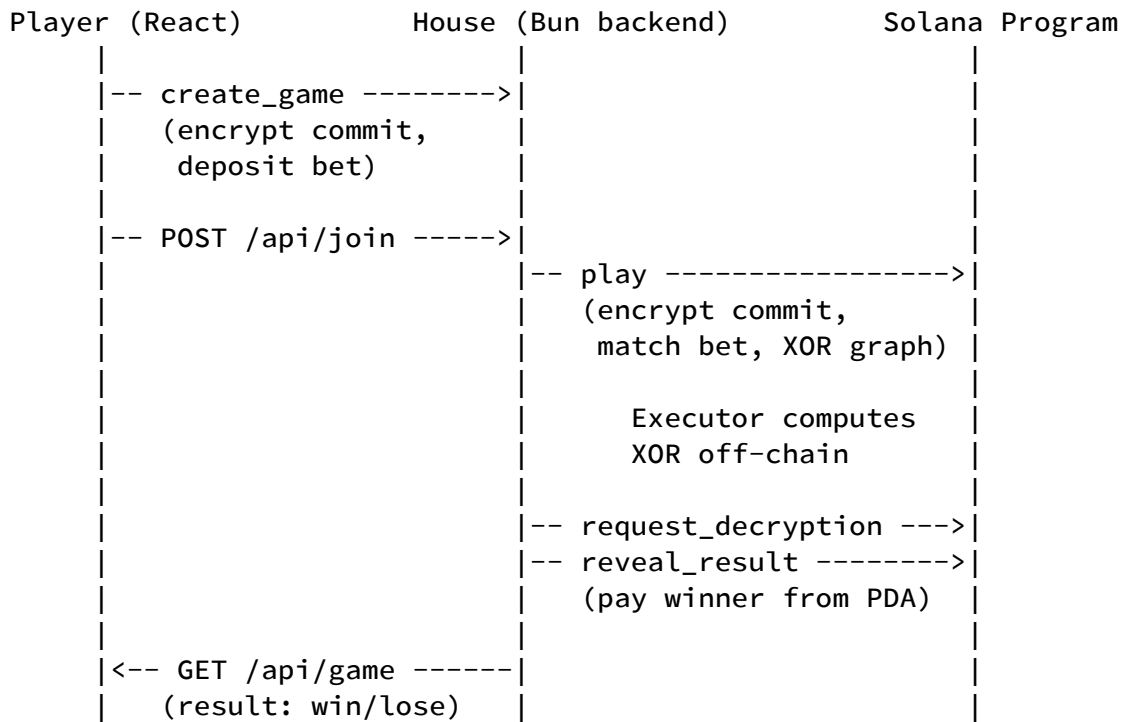
- How XOR on encrypted values produces a provably fair coin flip
- On-chain escrow pattern for trustless betting
- The player-vs-house architecture with an automated backend
- End-to-end flow from encrypted commit to payout

How it works

Two sides each commit an encrypted value (0 or 1). The Encrypt executor computes `result = commit_a XOR commit_b` using FHE – neither side can see the other's value before committing. XOR = 1 means side A wins; XOR = 0 means side B wins.

Both sides deposit equal bets into a game PDA. The winner receives 2x from escrow.

Architecture



Why this is provably fair

1. Both sides commit encrypted values before seeing the other's choice
2. The FHE XOR computation is deterministic – the executor cannot alter it
3. The on-chain program enforces payout rules – neither side can withhold funds
4. The ciphertext digest is verified at reveal time – stale or tampered results are rejected

Components

Component	Location	Role
Solana program (Anchor)	<code>anchor/src/lib.rs</code>	Game state, escrow, CPI to Encrypt
Solana program (Pinocchio)	<code>pinocchio/src/lib.rs</code>	Same logic, low-level
House backend	<code>react/server/house.ts</code>	Auto-joins games, handles decrypt + reveal
React frontend	<code>react/src/App.tsx</code>	Player UI: bet, flip, see result

Building the Coin Flip Program

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Step-by-step guide to the Anchor on-chain program.

What you'll learn

- How to define an FHE graph with `#[encrypt_fn]`
- Game state design with escrow
- CPI to Encrypt for graph execution and decryption
- The full instruction set: create, play, decrypt, reveal, cancel

1. The XOR graph

The entire fairness mechanism is a single line:

```
use encrypt_dsl::prelude::encrypt_fn;
use encrypt_types::encrypted::EUint64;

#[encrypt_fn]
fn coin_flip_graph(commit_a: EUint64, commit_b: EUint64) -> EUint64 {
    commit_a ^ commit_b
}
```

`#[encrypt_fn]` compiles this into a binary graph that the Encrypt executor evaluates using FHE. The function itself never runs on-chain – it generates a static graph at compile time. The macro also generates an extension trait (`CoinFlipGraphCpi`) with a method `coin_flip_graph()` on `EncryptContext` that handles the CPI.

Why XOR is fair: If both sides pick the same value (0^0 or 1^1), result = 0 (side B wins). If they differ (0^1 or 1^0), result = 1 (side A wins). Neither side can predict the other's encrypted value, so both have a 50/50 chance.

2. Game state

```
#[account]
#[derive(InitSpace)]
pub struct Game {
    pub side_a: Pubkey,           // game creator
    pub game_id: [u8; 32],       // unique identifier
    pub commit_a: [u8; 32],      // side A's ciphertext account pubkey
    pub result_ct: [u8; 32],     // result ciphertext account pubkey
    pub side_b: Pubkey,         // joiner (zeroed until play)
    pub is_active: bool,
    pub played: bool,           // false=waiting, true=both committed
    pub pending_digest: [u8; 32], // decryption digest for verification
    pub revealed_result: u8,     // 0=unknown, 1=side_a wins, 2=side_b wins
    pub bet_lamports: u64,
    pub bump: u8,
}
```

Key design choices:

- `commit_a` and `result_ct` store ciphertext account pubkeys (32 bytes each). These are keypair accounts in Encrypt, so pubkey = identifier.
- `pending_digest` is set when decryption is requested. At reveal time, we verify the decrypted value matches this digest – preventing stale or tampered results.
- `bet_lamports` is the per-side bet. The PDA holds both deposits.

3. create_game – side A deposits and commits

```
pub fn create_game(
    ctx: Context<CreateGame>,
    game_id: [u8; 32],
    commit_a_id: [u8; 32],
    result_ct_id: [u8; 32],
    bet_lamports: u64,
) -> Result<()> {
    // Side A deposits bet
    if bet_lamports > 0 {
        system_program::transfer(
            CpiContext::new(
                ctx.accounts.system_program.to_account_info(),
                system_program::Transfer {
                    from: ctx.accounts.payer.to_account_info(),
                    to: ctx.accounts.game.to_account_info(),
                },
            ),
            bet_lamports,
        )?;
    }

    let game = &mut ctx.accounts.game;
    game.side_a = ctx.accounts.side_a.key();
    game.game_id = game_id;
    game.commit_a = commit_a_id;
    game.result_ct = result_ct_id;
    game.side_b = Pubkey::default();
    game.is_active = true;
    game.played = false;
    game.pending_digest = [0u8; 32];
    game.revealed_result = 0;
    game.bet_lamports = bet_lamports;
    game.bump = ctx.bumps.game;
    Ok(())
}
```

The game PDA is derived from `["game", game_id]`. Side A's encrypted commit (`commit_a_id`) is created before this instruction via `gRPC createInput`. The `result_ct_id` is a pre-created plaintext ciphertext (initialized to 0) that will hold the XOR output.

Why pre-create result_ct: Encrypt's `execute_graph` writes results into existing ciphertext accounts. The output account must exist before the graph runs. Side A creates it during `create_game` so it's ready when side B triggers the XOR.

Account validation:

```
#[derive(Accounts)]
#[instruction(game_id: [u8; 32])]
pub struct CreateGame<'info> {
    #[account(
        init,
        payer = payer,
        space = 8 + Game::INIT_SPACE,
        seeds = [b"game", game_id.as_ref()],
        bump,
    )]
    pub game: Account<'info, Game>,
    pub side_a: Signer<'info>,
    #[account(mut)]
    pub payer: Signer<'info>,
    pub system_program: Program<'info, System>,
}
```

4. play – side B matches bet and triggers XOR

```
pub fn play(ctx: Context<Play>, cpi_authority_bump: u8) -> Result<()> {
    let game = &ctx.accounts.game;
    require!(game.is_active, CoinFlipError::GameClosed);
    require!(!game.played, CoinFlipError::AlreadyPlayed);

    // Verify ciphertext accounts match game state
    require!(
        ctx.accounts.commit_a_ct.key().to_bytes() == game.commit_a,
        CoinFlipError::InvalidAccount
    );
    require!(
        ctx.accounts.result_ct.key().to_bytes() == game.result_ct,
        CoinFlipError::InvalidAccount
    );

    // Side B matches bet
    let bet = game.bet_lamports;
    if bet > 0 {
        system_program::transfer(
            CpiContext::new(
                ctx.accounts.system_program.to_account_info(),
                system_program::Transfer {
                    from: ctx.accounts.side_b.to_account_info(),
                    to: ctx.accounts.game.to_account_info(),
                },
            ),
            bet,
        )?;
    }

    let encrypt_ctx = EncryptContext {
        encrypt_program: ctx.accounts.encrypt_program.to_account_info(),
        config: ctx.accounts.config.to_account_info(),
        deposit: ctx.accounts.deposit.to_account_info(),
        cpi_authority: ctx.accounts.cpi_authority.to_account_info(),
        caller_program: ctx.accounts.caller_program.to_account_info(),
        network_encryption_key:
            ctx.accounts.network_encryption_key.to_account_info(),
        payer: ctx.accounts.payer.to_account_info(),
        event_authority: ctx.accounts.event_authority.to_account_info(),
        system_program: ctx.accounts.system_program.to_account_info(),
        cpi_authority_bump,
    };

    let commit_a = ctx.accounts.commit_a_ct.to_account_info();
    let commit_b = ctx.accounts.commit_b_ct.to_account_info();
    let result = ctx.accounts.result_ct.to_account_info();
    encrypt_ctx.coin_flip_graph(commit_a, commit_b, result)?;

    let game = &mut ctx.accounts.game;
    game.side_b = ctx.accounts.side_b.key();
    game.played = true;
    Ok(())
}
```

The `coin_flip_graph()` method is auto-generated by `#[encrypt_fn]`. It CPIs into the Encrypt program with the graph bytecode, input ciphertext accounts (`commit_a`, `commit_b`), and output account (`result`). The executor picks this up off-chain, computes the encrypted XOR, and writes the result back to `result_ct`.

The `EncryptContext` bundles all the Encrypt program accounts needed for CPI. The `cpi_authority` is a PDA derived from your program's ID – it authorizes your program to call Encrypt.

5. request_result_decryption

```
pub fn request_result_decryption(
    ctx: Context<RequestResultDecryption>,
    cpi_authority_bump: u8,
) -> Result<()> {
    let game = &ctx.accounts.game;
    require!(game.played, CoinFlipError::NotPlayed);

    let encrypt_ctx = EncryptContext { /* ... same fields ... */ };

    let digest = encrypt_ctx.request_decryption(
        &ctx.accounts.request_acct.to_account_info(),
        &ctx.accounts.result_ciphertext.to_account_info(),
    );

    let game = &mut ctx.accounts.game;
    game.pending_digest = digest;
    Ok(())
}
```

`request_decryption` creates a decryption request account (keypair, not PDA) and returns a 32-byte digest. This digest is a snapshot of the ciphertext's current state. Storing it in the game ensures that `reveal_result` verifies against the exact value that was requested for decryption.

Anyone can call this after both sides have played.

6. reveal_result – verify and pay winner

```
pub fn reveal_result(ctx: Context<RevealResult>) -> Result<()> {
    let game = &ctx.accounts.game;
    require!(game.played, CoinFlipError::NotPlayed);
    require!(game.revealed_result == 0, CoinFlipError::AlreadyRevealed);

    let expected_digest = &game.pending_digest;

    let req_data = ctx.accounts.request_acct.try_borrow_data()?;
    use encrypt_types::encrypted::Uint64;
    let value = encrypt_anchor::accounts::read_decrypted_verified::<Uint64>(
        &req_data,
        expected_digest,
    )
    .map_err(|_| CoinFlipError::DecryptionNotComplete)?;

    let side_a_wins = *value == 1;
    let expected_winner = if side_a_wins { game.side_a } else { game.side_b };
    require!(
        ctx.accounts.winner.key() == expected_winner,
        CoinFlipError::WrongWinner
    );

    // Pay winner
    let payout = game.bet_lamports * 2;
    if payout > 0 {
        let game_info = ctx.accounts.game.to_account_info();
        let winner_info = ctx.accounts.winner.to_account_info();
        **game_info.lamports.borrow_mut() -= payout;
        **winner_info.lamports.borrow_mut() += payout;
    }

    let game = &mut ctx.accounts.game;
    game.revealed_result = if side_a_wins { 1 } else { 2 };
    game.is_active = false;
    Ok(())
}
```

`read_decrypted_verified::<Uint64>` reads the decrypted value from the request account and verifies it against the stored digest. If the ciphertext was modified after the decryption request, the digest won't match and this fails.

The payout uses direct lamport manipulation – the game PDA is program-owned, so we can debit it directly.

7. cancel_game – refund before play

```
pub fn cancel_game(ctx: Context<CancelGame>) -> Result<()> {  
    let game = &ctx.accounts.game;  
    require!(game.is_active, CoinFlipError::GameClosed);  
    require!(!game.played, CoinFlipError::AlreadyPlayed);  
    require!(  
        ctx.accounts.side_a.key() == game.side_a,  
        CoinFlipError::Unauthorized  
    );  
  
    let bet = game.bet_lamports;  
    if bet > 0 {  
        let game_info = ctx.accounts.game.to_account_info();  
        let side_a_info = ctx.accounts.side_a.to_account_info();  
        **game_info.lamports.borrow_mut() -= bet;  
        **side_a_info.lamports.borrow_mut() += bet;  
    }  
  
    let game = &mut ctx.accounts.game;  
    game.is_active = false;  
    Ok(())  
}
```

Only side A can cancel, and only before side B joins. This prevents griefing – side A can always recover their funds if no opponent shows up.

Instruction summary

Disc	Instruction	Who	When
0	create_game	Side A	Start – deposit bet, commit encrypted value
1	play	Side B	After create – match bet, commit, XOR executes
2	request_result_decryption	Anyone	After play – triggers MPC decryption
3	reveal_result	Anyone	After decryption – pays winner 2x from escrow
4	cancel_game	Side A	Before play – refund bet

On-Chain Escrow Deep Dive

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

How SOL flows through the coin flip game.

What you'll learn

- How the game PDA acts as a trustless escrow
- System transfer CPI for deposits vs direct lamport manipulation for payouts
- Cancel refund logic
- Why this design is secure

SOL flow

Side A wallet —(system transfer)—> Game PDA —(lamport manipulation)—> Winner wallet
Side B wallet —(system transfer)—> Game PDA

1. **Side A deposits** during `create_game` via system program transfer CPI:

```

if bet_lamports > 0 {
  system_program::transfer(
    CpiContext::new(
      ctx.accounts.system_program.to_account_info(),
      system_program::Transfer {
        from: ctx.accounts.payer.to_account_info(),
        to: ctx.accounts.game.to_account_info(),
      },
    ),
    bet_lamports,
  )?;
}

```

2. **Side B matches** during `play` with the same pattern:

```

let bet = game.bet_lamports;
if bet > 0 {
  system_program::transfer(
    CpiContext::new(
      ctx.accounts.system_program.to_account_info(),
      system_program::Transfer {
        from: ctx.accounts.side_b.to_account_info(),
        to: ctx.accounts.game.to_account_info(),
      },
    ),
    bet,
  )?;
}

```

3. **Winner withdraws** during `reveal_result` via direct lamport manipulation:

```

let payout = game.bet_lamports * 2;
if payout > 0 {
  let game_info = ctx.accounts.game.to_account_info();
  let winner_info = ctx.accounts.winner.to_account_info();
  **game_info.lamports.borrow_mut() -= payout;
  **winner_info.lamports.borrow_mut() += payout;
}

```

Why two different transfer methods

Deposits use system program CPI because the source is a user wallet (system-owned account). Only the system program can debit a system-owned account.

Payouts use direct lamport manipulation because the game PDA is owned by our program. The Solana runtime allows a program to freely debit accounts it owns. This is cheaper (no CPI overhead) and simpler.

Cancel refund

Side A can cancel before side B joins:

```
pub fn cancel_game(ctx: Context<CancelGame>) -> Result<()> {
    let game = &ctx.accounts.game;
    require!(game.is_active, CoinFlipError::GameClosed);
    require!(!game.played, CoinFlipError::AlreadyPlayed);
    require!(ctx.accounts.side_a.key() == game.side_a,
CoinFlipError::Unauthorized);

    let bet = game.bet_lamports;
    if bet > 0 {
        let game_info = ctx.accounts.game.to_account_info();
        let side_a_info = ctx.accounts.side_a.to_account_info();
        **game_info.lamports.borrow_mut() -= bet;
        **side_a_info.lamports.borrow_mut() += bet;
    }

    let game = &mut ctx.accounts.game;
    game.is_active = false;
    Ok(())
}
```

Guards:

- `is_active` – can't cancel an already-finished game
- `!played` – can't cancel after side B committed (funds are locked for the outcome)
- `side_a == signer` – only the creator can cancel

Security properties

Neither side can cheat. Both values are encrypted before the other side commits. The XOR graph is deterministic and computed by the executor under FHE – there's no way to influence the result after committing.

Funds cannot be stolen. The game PDA is program-owned. Only the program's instructions can debit it. `reveal_result` requires a valid decrypted value matching the stored digest. The `winner` account is validated against the game state.

No griefing. Side A can cancel and recover funds if no opponent joins. Once both sides play, the game must resolve – anyone can call `request_result_decryption` and `reveal_result`.

No double-payout. `revealed_result` is checked to be 0 (unknown) before reveal. After payout, it's set to 1 or 2, preventing replay.

Testing the Coin Flip

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

What you'll learn

- Unit testing the FHE graph with mock compute
- How the mock evaluator works
- What each test case validates

Graph unit tests

The `#[encrypt_fn]` macro generates a function that returns the graph bytecode. You can test the graph logic without deploying to Solana by running it through a mock evaluator:

```
#[test]
fn xor_same_side_b_wins() {
    let r = run_mock(
        coin_flip_graph,
        &[0, 0],
        &[FheType::EUint64, FheType::EUint64],
    );
    assert_eq!(r[0], 0, "0^0=0 -> side_b wins");
}

#[test]
fn xor_diff_side_a_wins() {
    let r = run_mock(
        coin_flip_graph,
        &[0, 1],
        &[FheType::EUint64, FheType::EUint64],
    );
    assert_eq!(r[0], 1, "0^1=1 -> side_a wins");
}
```

The `run_mock` helper parses the graph bytecode and evaluates each node using mock digest encoding/decoding. This simulates exactly what the executor does, but with plaintext values encoded as mock identifiers.

Test matrix

Inputs	XOR	Winner	Test
0, 0	0	Side B	<code>xor_same_side_b_wins</code>
0, 1	1	Side A	<code>xor_diff_side_a_wins</code>
1, 1	0	Side B	<code>xor_both_one_side_b_wins</code>
1, 0	1	Side A	<code>xor_one_zero_side_a_wins</code>

Graph shape test

```
#[test]
fn graph_shape() {
    let d = coin_flip_graph();
    let pg = parse_graph(&d).unwrap();
    assert_eq!(pg.header().num_inputs(), 2, "commit_a + commit_b");
    assert_eq!(pg.header().num_outputs(), 1, "single flip result");
}
```

Validates that the compiled graph has exactly 2 inputs and 1 output. This catches accidental changes to the graph signature.

Running tests

```
# Unit tests only (no SBF build needed)
cargo test -p encrypt-coin-flip-anchor
```

```
# Or run all example tests
just test-examples
```

E2E tests

The `e2e/` directory contains integration tests that deploy the program to a local validator (LiteSVM or solana-program-test), run the full flow (create game, play, decrypt, reveal), and verify the winner gets paid. These require the SBF binary:

```
just build-sbf-examples
just test-examples-litesvm
```

Building the Full-Stack Coin Flip App

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

React frontend + Bun house backend.

What you'll learn

- The player-vs-house architecture
- How the browser encrypts locally and sends ciphertext via gRPC-Web
- How the house backend auto-resolves games
- Frontend flow: bet, flip, poll, result

Architecture

React App (:5173)	House Backend (:3001)	Executor (:50051)
-- encryptValue() ----->		
-- gRPC-Web createInput =====>		
<- ciphertextId =====		
-- create_game tx ----->	(on-chain)	
-- POST /api/join ----->		
	-- gRPC createInput ----->	
	-- play tx ----->	
	-- poll result_ct ----->	
	-- request_decryption ---->	
	-- poll decryption ----->	
	-- reveal_result ----->	
-- GET /api/game ----->		
<- { status, result } --		

The player encrypts locally in the browser and sends ciphertext directly to the executor via gRPC-Web (`fetch()` -based, no special proxy). The house backend runs as an automated counterparty – it loads a persistent keypair from `HOUSE_SECRET_KEY` in the `.env` file and handles everything after the player creates a game.

House backend

The backend (`react/server/house.ts`) has two responsibilities:

1. Join games as side B.

When the frontend calls `POST /api/join`, the backend:

- Reads the game PDA to get `commit_a`, `result_ct`, and `bet_lamports`
- Creates its own encrypted commit via gRPC
- Sends the `play` instruction (matches bet + triggers XOR graph)

```
// House creates encrypted commit
const houseVal = Math.random() < 0.5 ? 0 : 1;
const { ciphertextIdentifiers } = await encryptClient.createInput({
  chain: Chain.Solana,
  inputs: [{ ciphertextBytes: mockCiphertext(BigInt(houseVal)), fheType:
FHE_UINT64 }],
  authorized: COINFLIP_PROGRAM.toBytes(),
  networkEncryptionPublicKey: networkKey,
});
const commitB = new PublicKey(ciphertextIdentifiers[0]);

// Send play instruction
await sendTx([new TransactionInstruction({
  programId: COINFLIP_PROGRAM,
  data: Buffer.from([1, cpiBump]),
  keys: [
    { pubkey: gamePda, isSigner: false, isWritable: true },
    { pubkey: house.publicKey, isSigner: true, isWritable: true },
    { pubkey: commitA, isSigner: false, isWritable: true },
    { pubkey: commitB, isSigner: false, isWritable: true },
    { pubkey: resultCt, isSigner: false, isWritable: true },
    ...encCpi(),
  ],
})]);
```

2. Resolve the game.

After play, the backend polls `result_ct` until the executor commits the XOR result (status = VERIFIED). Then it requests decryption, polls until complete, reads the result, and sends `reveal_result` to pay the winner:

```

// Poll for XOR computation
await pollUntil(resultCt, isVerified, 60_000);

// Request decryption
const decReq = Keypair.generate();
await sendTx([new TransactionInstruction({
  programId: COINFLIP_PROGRAM,
  data: Buffer.from([2, cpiBump]),
  keys: [
    { pubkey: gamePda, isSigner: false, isWritable: true },
    { pubkey: decReq.publicKey, isSigner: true, isWritable: true },
    { pubkey: resultCt, isSigner: false, isWritable: false },
    ...encCpi(),
  ],
})], [decReq]);

// Poll for decryption
await pollUntil(decReq.publicKey, isDecrypted);

// Read result and reveal
const reqData = (await connection.getAccountInfo(decReq.publicKey))!.data as Buffer;
const xor = reqData.readBigUInt64LE(107);
const sideAWins = xor === 1n;
const winner = sideAWins ? sideA : house.publicKey;

await sendTx([new TransactionInstruction({
  programId: COINFLIP_PROGRAM,
  data: Buffer.from([3]),
  keys: [
    { pubkey: gamePda, isSigner: false, isWritable: true },
    { pubkey: decReq.publicKey, isSigner: false, isWritable: false },
    { pubkey: house.publicKey, isSigner: true, isWritable: false },
    { pubkey: winner, isSigner: false, isWritable: true },
  ],
})]);

```

React frontend

The frontend (`react/src/App.tsx`) handles wallet connection, bet input, and game lifecycle.

Player flow:

1. Connect wallet (Solana wallet adapter)
2. Enter bet amount in SOL
3. Click “Flip”
4. Frontend encrypts commit locally and sends ciphertext to executor via gRPC-Web
5. Frontend sends `create_game` transaction (deposits bet, stores commit)
6. Frontend calls `POST /api/join` to tell house to play
7. Frontend polls `GET /api/game/:pda` for status updates
8. Display result: win (+2x bet) or lose

Creating the game on-chain:

The player's commit is encrypted in the browser – the plaintext never leaves the client.

`encryptValue()` is a client-side mock encryption function (production: WASM FHE encryptor). gRPC-Web works via `fetch()` – no special proxy needed; the executor's tonic-web layer handles it.

```
import { createEncryptWebClient, encryptValue, Chain } from "@encrypt.xyz/pre-alpha-solana-client/grpc-web";

const grpcClient = createEncryptWebClient("https://pre-alpha-dev-1.encrypt.ika-network.net:443");

const playerVal = Math.random() < 0.5 ? 0 : 1;
const ids = await grpcClient.createInput({
  chain: Chain.SOLANA,
  inputs: [{ ciphertextBytes: encryptValue(BigInt(playerVal)), fheType: FHE_UINT64 }],
  authorized: COINFLIP_PROGRAM.toBytes(),
  networkEncryptionPublicKey: networkKey,
});
const commitACT = new PublicKey(ids[0]);

const gameId = Buffer.from(Keypair.generate().publicKey.toBytes());
const [gamePda, gameBump] = findPda([Buffer.from("game"), gameId], COINFLIP_PROGRAM);
const resultCt = Keypair.generate();

const createData = Buffer.alloc(43);
createData[0] = 0; // discriminator
createData[1] = gameBump;
createData[2] = enc.cpiBump;
gameId.copy(createData, 3);
createData.writeBigUInt64LE(BigInt(betLamports), 35);

const tx = new Transaction().add(new TransactionInstruction({
  programId: COINFLIP_PROGRAM,
  data: createData,
  keys: [
    { pubkey: gamePda, isSigner: false, isWritable: true },
    { pubkey: wallet.publicKey, isSigner: true, isWritable: false },
    { pubkey: commitACT, isSigner: false, isWritable: false },
    { pubkey: resultCt.publicKey, isSigner: true, isWritable: true },
    { pubkey: ENCRYPT_PROGRAM, isSigner: false, isWritable: false },
    { pubkey: enc.configPda, isSigner: false, isWritable: false },
    { pubkey: enc.depositPda, isSigner: false, isWritable: true },
    { pubkey: enc.cpiAuthority, isSigner: false, isWritable: false },
    { pubkey: COINFLIP_PROGRAM, isSigner: false, isWritable: false },
    { pubkey: enc.networkKeyPda, isSigner: false, isWritable: false },
    { pubkey: wallet.publicKey, isSigner: true, isWritable: true },
    { pubkey: enc.eventAuthority, isSigner: false, isWritable: false },
    { pubkey: SystemProgram.programId, isSigner: false, isWritable: false },
  ],
}));
await wallet.sendTransaction(tx, connection, { signers: [resultCt] });
```

Polling for result:

```

const start = Date.now();
while (Date.now() - start < 120_000) {
  const r = await fetch(`${HOUSE_API}/api/game/${gamePda.toBase58()}`);
  const state = await r.json();
  if (state.status === "resolved") {
    const won = state.result === 1;
    setResult(won ? "win" : "lose");
    return;
  }
  await new Promise((r) => setTimeout(r, 800));
}

```

Encrypt deposit

Both the frontend and house backend need an Encrypt deposit account before they can use Encrypt CPIs. The frontend creates one on first use:

```

const ensureDeposit = async () => {
  if (await connection.getAccountInfo(enc.depositPda)) return; // already
exists
  const data = Buffer.alloc(18);
  data[0] = 14; // create_deposit discriminator
  data[1] = enc.depositBump;
  const tx = new Transaction().add(new TransactionInstruction({
    programId: ENCRYPT_PROGRAM, data,
    keys: [/* deposit PDA, config, payer, vault, system_program */],
  }));
  await wallet.sendTransaction(tx, connection);
};

```

Running on Devnet

The app connects to Solana devnet and the pre-alpha executor automatically. No local validator or executor setup is needed.

```
# Set the house secret key in the .env (Bun loads from the react/ directory)
# Supports base58 or JSON array format
echo 'HOUSE_SECRET_KEY=[1,2,3,...,64 bytes]' >> chains/solana/examples/coin-
flip/react/.env

# Fund the house wallet on devnet
solana airdrop 2 <HOUSE_PUBLIC_KEY> --url devnet

# Terminal 1: Start the house backend
cd chains/solana/examples/coin-flip/react
bun server/house.ts

# Terminal 2: Start the React dev server
cd chains/solana/examples/coin-flip/react
bun run dev
```

Open <http://localhost:5173> , connect a wallet (e.g. Phantom set to devnet), airdrop SOL, and flip.

Confidential Voting

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Encrypted voting where individual votes are hidden but the tally is computed via FHE.

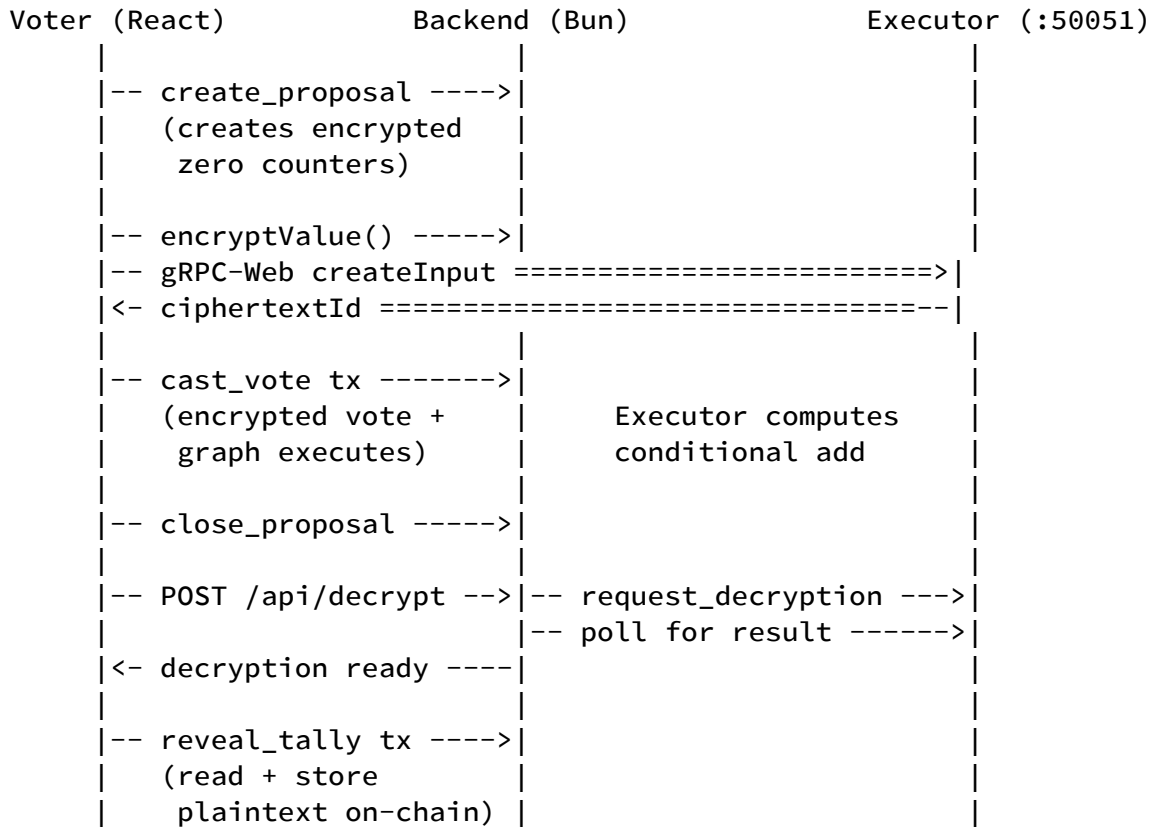
What you'll learn

- How FHE enables private voting with public tallies
- The architecture: React frontend (gRPC-Web) + Bun backend + Solana program + executor
- End-to-end flow from encrypted vote to revealed results

How it works

Voters cast encrypted yes/no votes (EBool). The on-chain program CPIs into Encrypt to run an FHE graph that conditionally increments encrypted yes or no counters. Nobody – not the program, not the executor, not other voters – can see individual votes. Only when the proposal authority closes voting and requests decryption are the final tallies revealed.

Architecture



The browser encrypts votes locally and sends ciphertext directly to the executor via gRPC-Web – the plaintext never leaves the client. The backend only handles decryption requests and polling.

Privacy guarantees

- **Individual votes are hidden.** Each vote is an encrypted boolean. The graph operates on ciphertexts – the executor never sees plaintext votes.
- **Tallies are computed homomorphically.** The yes/no counters are encrypted integers. Each vote conditionally adds 1 to one counter without decrypting either.
- **Only the authority can reveal.** Decryption requires the proposal authority to request it and sign the reveal transaction.
- **Double-voting is prevented.** A VoteRecord PDA per voter per proposal enforces one vote each.

Components

Component	Location	Role
Solana program (Anchor)	<code>anchor/src/lib.rs</code>	Proposal state, vote graph CPI, tally reveal
Solana program (Pinocchio)	<code>pinocchio/src/lib.rs</code>	Same logic, low-level
Backend	<code>react/server/backend.ts</code>	Decryption request + polling
React frontend	<code>react/src/App.tsx</code>	Create proposals, vote, close, reveal

Building the Voting Program

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Step-by-step guide to the Anchor on-chain program.

What you'll learn

- How to define an FHE graph with conditional logic (if/else compiles to Select)
- Proposal state with encrypted counters
- Update-mode ciphertexts (same account as input and output)
- VoteRecord PDA for double-vote prevention
- The decrypt-then-reveal pattern for tallies

1. The cast_vote graph

```
use encrypt_dsl::prelude::encrypt_fn;
use encrypt_types::encrypted::{EBool, EUint64};

#[encrypt_fn]
fn cast_vote_graph(
    yes_count: EUint64,
    no_count: EUint64,
    vote: EBool,
) -> (EUint64, EUint64) {
    let new_yes = if vote { yes_count + 1 } else { yes_count };
    let new_no = if vote { no_count } else { no_count + 1 };
    (new_yes, new_no)
}
```

This graph takes three encrypted inputs and produces two encrypted outputs:

- `yes_count` / `no_count` – current encrypted tallies (EUint64)

- `vote` – the voter's encrypted choice (EBool: `true` = yes, `false` = no)

The `if vote { ... } else { ... }` syntax compiles to a `Select` operation in the FHE graph. `Select` is a ternary: `Select(condition, if_true, if_false)`. The executor evaluates this homomorphically – it never learns whether the voter chose yes or no.

The graph returns a tuple `(new_yes, new_no)`. If `vote = true`, `new_yes = yes_count + 1` and `new_no = no_count` (unchanged). If `vote = false`, the reverse.

`#[encrypt_fn]` generates a `CastVoteGraphCpi` trait with a `cast_vote_graph()` method on `EncryptContext`. The method takes 3 input accounts and 2 output accounts.

2. Proposal state

```
#[account]
#[derive(InitSpace)]
pub struct Proposal {
    pub authority: Pubkey,           // who can close + reveal
    pub proposal_id: [u8; 32],
    pub yes_count: [u8; 32],         // ciphertext account pubkey
    pub no_count: [u8; 32],          // ciphertext account pubkey
    pub is_open: bool,
    pub total_votes: u64,             // plaintext counter (for UI)
    pub revealed_yes: u64,            // written at reveal time
    pub revealed_no: u64,            // written at reveal time
    pub pending_yes_digest: [u8; 32],
    pub pending_no_digest: [u8; 32],
    pub bump: u8,
}
```

`yes_count` and `no_count` store ciphertext account pubkeys. These are the encrypted counters that get updated with every vote. `pending_yes_digest` and `pending_no_digest` are set when decryption is requested, used to verify the reveal.

```
#[account]
#[derive(InitSpace)]
pub struct VoteRecord {
    pub voter: Pubkey,
    pub bump: u8,
}
```

`VoteRecord` is a PDA derived from `["vote", proposal_id, voter_pubkey]`. If it already exists, Anchor's `init` constraint fails, preventing double votes.

3. create_proposal – initialize encrypted zero counters

```
pub fn create_proposal(
  ctx: Context<CreateProposal>,
  proposal_id: [u8; 32],
  initial_yes_id: [u8; 32],
  initial_no_id: [u8; 32],
) -> Result<()> {
  let prop = &mut ctx.accounts.proposal;
  prop.authority = ctx.accounts.authority.key();
  prop.proposal_id = proposal_id;
  prop.yes_count = initial_yes_id;
  prop.no_count = initial_no_id;
  prop.is_open = true;
  prop.total_votes = 0;
  prop.bump = ctx.bumps.proposal;
  Ok(())
}
```

The `initial_yes_id` and `initial_no_id` are ciphertext accounts pre-created with `create_plaintext_typed::<Uint64>(0)`. They start as encrypted zeros. The frontend creates these keypair accounts and passes their pubkeys.

Account validation:

```
#[derive(Accounts)]
#[instruction(proposal_id: [u8; 32])]
pub struct CreateProposal<'info> {
  #[account(
    init,
    payer = payer,
    space = 8 + Proposal::INIT_SPACE,
    seeds = [b"proposal", proposal_id.as_ref()],
    bump,
  )]
  pub proposal: Account<'info, Proposal>,
  pub authority: Signer<'info>,
  #[account(mut)]
  pub payer: Signer<'info>,
  pub system_program: Program<'info, System>,
}
```

4. cast_vote – encrypted vote with update-mode ciphertexts

```
pub fn cast_vote(
    ctx: Context<CastVote>,
    cpi_authority_bump: u8,
) -> Result<()> {
    let prop = &ctx.accounts.proposal;
    require!(prop.is_open, VotingError::ProposalClosed);

    let encrypt_ctx = EncryptContext {
        encrypt_program: ctx.accounts.encrypt_program.to_account_info(),
        config: ctx.accounts.config.to_account_info(),
        deposit: ctx.accounts.deposit.to_account_info(),
        cpi_authority: ctx.accounts.cpi_authority.to_account_info(),
        caller_program: ctx.accounts.caller_program.to_account_info(),
        network_encryption_key:
            ctx.accounts.network_encryption_key.to_account_info(),
        payer: ctx.accounts.payer.to_account_info(),
        event_authority: ctx.accounts.event_authority.to_account_info(),
        system_program: ctx.accounts.system_program.to_account_info(),
        cpi_authority_bump,
    };

    let yes_ct = ctx.accounts.yes_ct.to_account_info();
    let no_ct = ctx.accounts.no_ct.to_account_info();
    let vote_ct = ctx.accounts.vote_ct.to_account_info();
    encrypt_ctx.cast_vote_graph(
        yes_ct.clone(), no_ct.clone(), vote_ct,
        yes_ct, no_ct,
    )?;

    let prop = &mut ctx.accounts.proposal;
    prop.total_votes += 1;

    let vr = &mut ctx.accounts.vote_record;
    vr.voter = ctx.accounts.voter.key();
    vr.bump = ctx.bumps.vote_record;

    Ok(())
}
```

Update mode: Notice that `yes_ct` and `no_ct` appear as both inputs and outputs:

```
encrypt_ctx.cast_vote_graph(
    yes_ct.clone(), no_ct.clone(), vote_ct, // inputs: yes, no, vote
    yes_ct, no_ct,                        // outputs: yes, no
)?;
```

The same ciphertext accounts are read (current tally) and written (new tally). The executor reads the current encrypted value, computes the graph, and writes the result back to the same account. This avoids creating new ciphertext accounts for every vote.

The **vote ciphertext** (`vote_ct`) is created before this instruction. The browser encrypts the vote locally via `encryptValue()` and sends the ciphertext directly to the executor via gRPC-Web `createInput` . It's an encrypted boolean authorized to the voting program.

Double-vote prevention: The `vote_record` account uses Anchor's `init` constraint:

```
#[account(
  init,
  payer = payer,
  space = 8 + VoteRecord::INIT_SPACE,
  seeds = [b"vote", proposal.proposal_id.as_ref(), voter.key().as_ref()],
  bump,
)]
pub vote_record: Account<'info, VoteRecord>,
```

If the voter has already voted on this proposal, the PDA already exists and `init` fails. Simple and gas-efficient.

5. close_proposal – lock voting

```
pub fn close_proposal(ctx: Context<CloseProposal>) -> Result<()> {
  let prop = &mut ctx.accounts.proposal;
  require!(
    prop.authority == ctx.accounts.authority.key(),
    VotingError::Unauthorized
  );
  require!(prop.is_open, VotingError::ProposalClosed);
  prop.is_open = false;
  Ok(())
}
```

Only the authority can close. After closing, no more votes can be cast (the `cast_vote` guard checks `is_open`). Decryption can only be requested after closing.

6. request_tally_decryption – two separate requests

```
pub fn request_tally_decryption(
    ctx: Context<RequestTallyDecryption>,
    is_yes: bool,
    cpi_authority_bump: u8,
) -> Result<()> {
    let prop = &ctx.accounts.proposal;
    require!(!prop.is_open, VotingError::ProposalStillOpen);

    let encrypt_ctx = EncryptContext { /* ... */ };

    let digest = encrypt_ctx.request_decryption(
        &ctx.accounts.request_acct.to_account_info(),
        &ctx.accounts.ciphertext.to_account_info(),
    )?;

    let prop = &mut ctx.accounts.proposal;
    if is_yes {
        prop.pending_yes_digest = digest;
    } else {
        prop.pending_no_digest = digest;
    }
    Ok(())
}
```

Each ciphertext (yes_count, no_count) needs its own decryption request. The `is_yes` flag determines which digest to store. You call this instruction twice – once for yes, once for no.

The `request_acct` is a fresh keypair account that the decryptor network will write the plaintext into.

7. reveal_tally – read decrypted values

```
pub fn reveal_tally(ctx: Context<RevealTally>, is_yes: bool) -> Result<()> {
    let prop = &mut ctx.accounts.proposal;
    require!(
        prop.authority == ctx.accounts.authority.key(),
        VotingError::Unauthorized
    );
    require(!prop.is_open, VotingError::ProposalStillOpen);

    let expected_digest = if is_yes {
        &prop.pending_yes_digest
    } else {
        &prop.pending_no_digest
    };

    let req_data = ctx.accounts.request_acct.try_borrow_data()?;
    use encrypt_types::encrypted::Uint64;
    let value = encrypt_anchor::accounts::read_decrypted_verified::<Uint64>(
        &req_data, expected_digest,
    ).map_err(|_| VotingError::DecryptionNotComplete)?;

    if is_yes {
        prop.revealed_yes = *value;
    } else {
        prop.revealed_no = *value;
    }
    Ok(())
}
```

`read_decrypted_verified` checks that the decrypted value's digest matches what was stored at request time. This prevents reading stale or tampered values. Called twice – once for yes, once for no. Only the authority can reveal.

Instruction summary

Disc	Instruction	Who	When
0	<code>create_proposal</code>	Authority	Start – creates encrypted zero counters
1	<code>cast_vote</code>	Any voter	While open – encrypted vote, graph updates counters
2	<code>close_proposal</code>	Authority	After voting ends – locks further votes
3	<code>request_tally_decryption</code>	Anyone	After close – one call per counter (yes/no)

Disc	Instruction	Who	When
4	reveal_tally	Authority	After decryption - writes plaintext to proposal

Testing Confidential Voting

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

What you'll learn

- Unit testing the cast_vote FHE graph
- How conditional logic (Select) is tested
- What each test case validates

Graph unit tests

The `#[encrypt_fn]` macro generates a function returning the graph bytecode. Test it with a mock evaluator:

```

#[test]
fn vote_yes_increments_yes_count() {
    let r = run_mock(
        cast_vote_graph,
        &[10, 5, 1], // yes_count=10, no_count=5, vote=true
        &[FheType::EUint64, FheType::EUint64, FheType::EBool],
    );
    assert_eq!(r[0], 11); // yes_count incremented
    assert_eq!(r[1], 5);  // no_count unchanged
}

#[test]
fn vote_no_increments_no_count() {
    let r = run_mock(
        cast_vote_graph,
        &[10, 5, 0], // yes_count=10, no_count=5, vote=false
        &[FheType::EUint64, FheType::EUint64, FheType::EBool],
    );
    assert_eq!(r[0], 10); // yes_count unchanged
    assert_eq!(r[1], 6);  // no_count incremented
}

#[test]
fn vote_from_zero() {
    let r = run_mock(
        cast_vote_graph,
        &[0, 0, 1], // both counters at zero, vote yes
        &[FheType::EUint64, FheType::EUint64, FheType::EBool],
    );
    assert_eq!(r[0], 1);
    assert_eq!(r[1], 0);
}

```

The `run_mock` helper parses the graph bytecode and evaluates nodes using mock digest encoding. It handles the `select` operation (op_type 60) which is what `if vote { ... } else { ... }` compiles to.

Test matrix

yes_count	no_count	vote	new_yes	new_no	Test
10	5	true	11	5	vote_yes_increment
10	5	false	10	6	vote_no_increments
0	0	true	1	0	vote_from_zero

Graph shape test

```
#[test]
fn graph_shape() {
    let d = cast_vote_graph();
    let pg = parse_graph(&d).unwrap();
    assert_eq!(pg.header().num_inputs(), 3); // yes_count, no_count, vote
    assert_eq!(pg.header().num_outputs(), 2); // new_yes, new_no
}
```

The graph has 3 inputs (two counters + one boolean vote) and 2 outputs (updated counters). This catches signature changes.

Running tests

```
# Unit tests only (no SBF build needed)
cargo test -p encrypt-voting-anchor

# All example tests
just test-examples

# E2E with LiteSVM
just build-sbf-examples
just test-examples-litesvm
```

E2E tests

The `e2e/` directory contains integration tests that deploy the program, create a proposal, cast multiple votes, close, decrypt, and verify the tallies match. These require the SBF binary and exercise the full Encrypt CPI flow.

Building the Voting App

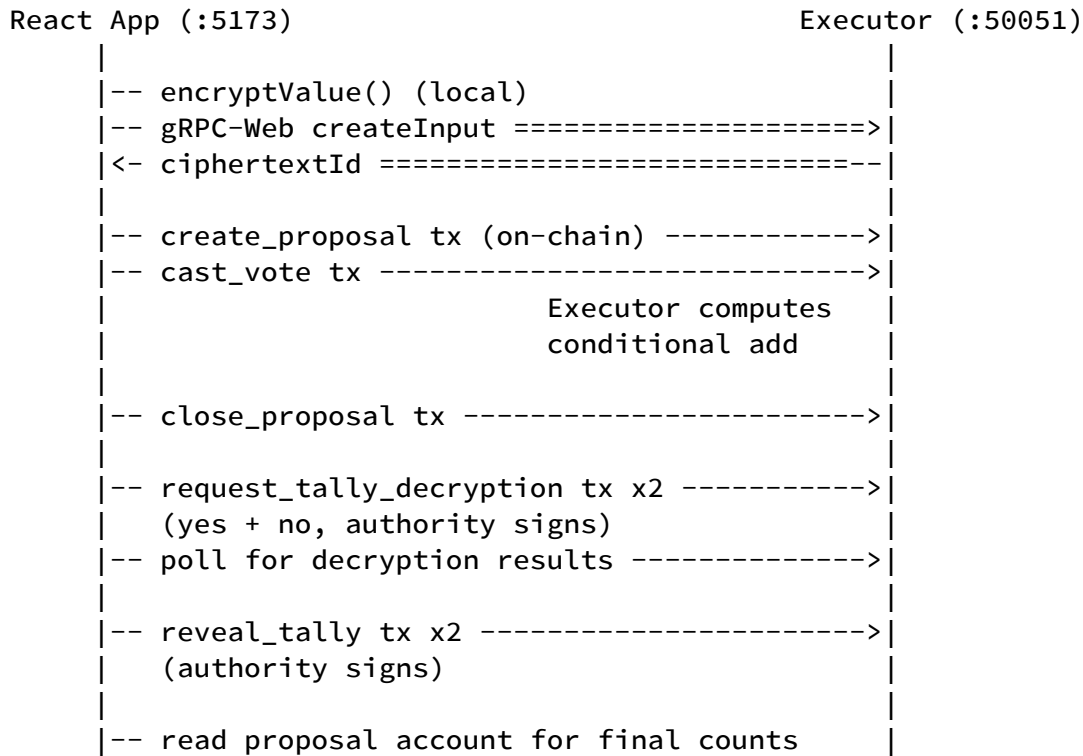
Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

React frontend — fully client-side, no backend needed.

What you'll learn

- How encrypted votes are created locally and cast
- How the authority requests decryption and reveals results directly from the browser
- Multi-wallet support via URL sharing

Architecture



Everything happens in the browser. The voter encrypts locally and sends ciphertext to the executor via gRPC-Web. The authority requests decryption and reveals results by signing transactions with their wallet — no backend keypair needed.

React frontend

The frontend (`react/src/App.tsx`) handles the full proposal lifecycle.

Creating a proposal:

The frontend creates the proposal PDA and two ciphertext keypair accounts (yes_count, no_count) initialized to encrypted zero:

```

const proposalId = Buffer.from(Keypair.generate().publicKey.toBytes());
const [pda, bump] = findPda([Buffer.from("proposal"), proposalId],
VOTING_PROGRAM);
const yesCt = Keypair.generate();
const noCt = Keypair.generate();

const tx = new Transaction().add(new TransactionInstruction({
  programId: VOTING_PROGRAM,
  data: createData,
  keys: [
    { pubkey: pda, isSigner: false, isWritable: true },
    { pubkey: wallet.publicKey, isSigner: true, isWritable: false },
    { pubkey: yesCt.publicKey, isSigner: true, isWritable: true },
    { pubkey: noCt.publicKey, isSigner: true, isWritable: true },
    // ... encrypt program accounts ...
  ],
}));
await wallet.sendTransaction(tx, connection, { signers: [yesCt, noCt] });

```

Casting a vote:

1. Encrypt the vote locally via `encryptValue()` and send ciphertext to executor via gRPC-Web
2. If previous votes exist, wait for ciphertext accounts to reach VERIFIED status (the executor must finish the previous graph before a new one can use the same accounts)
3. Send `cast_vote` transaction with the encrypted vote + proposal's yes/no ciphertext accounts

The plaintext never leaves the browser. `encryptValue()` is client-side mock encryption (production: WASM FHE encryptor). gRPC-Web works via `fetch()` – no special proxy needed; the executor uses `tonic-web`.

```

import { createEncryptWebClient, encryptValue, Chain } from "@encrypt.xyz/pre-alpha-solana-client/grpc-web";

const grpcClient = createEncryptWebClient("https://pre-alpha-dev-1.encrypt.ika-network.net:443");
const FHE_BOOL = 0 // Check 'crates/encrypt-types/src/types.rs' for other ciphertext variable values.

const voteVal = voteYes ? 1 : 0;
const ids = await grpcClient.createInput({
  chain: Chain.SOLANA,
  inputs: [{ ciphertextBytes: encryptValue(voteVal, FHE_BOOL), fheType: FHE_BOOL }],
  authorized: VOTING_PROGRAM.toBytes(),
  networkEncryptionPublicKey: networkKey,
});
const voteCt = new PublicKey(ids[0]);

// Wait for previous vote's computation to finish
if (proposal.totalVotes > 0) {
  await pollUntil(connection, proposal.yesCt, isVerified, 60_000);
}

const ix = new TransactionInstruction({
  programId: VOTING_PROGRAM,
  data: Buffer.from([1, vrBump, cpiBump]),
  keys: [
    { pubkey: proposal.pda, isSigner: false, isWritable: true },
    { pubkey: voteRecord, isSigner: false, isWritable: true },
    { pubkey: wallet.publicKey, isSigner: true, isWritable: false },
    { pubkey: voteCt, isSigner: false, isWritable: true },
    { pubkey: proposal.yesCt, isSigner: false, isWritable: true },
    { pubkey: proposal.noCt, isSigner: false, isWritable: true },
    // ... encrypt program accounts ...
  ],
});
await wallet.sendTransaction(new Transaction().add(ix), connection);

```

Decrypting and revealing:

The authority handles decryption entirely from the browser — no backend needed. The wallet signs the decryption request and reveal transactions directly:

```

// 1. Request decryption for yes tally
const yesReq = Keypair.generate();
await sendTx([new TransactionInstruction({
  programId: VOTING_PROGRAM,
  data: Buffer.from([3, cpiBump, 1]), // disc=3, is_yes=1
  keys: [
    { pubkey: proposal.pda, isSigner: false, isWritable: true },
    { pubkey: yesReq.publicKey, isSigner: true, isWritable: true },
    { pubkey: proposal.yesCt, isSigner: false, isWritable: false },
    ...encCpi(),
  ],
})), [yesReq]);

// 2. Request decryption for no tally
const noReq = Keypair.generate();
await sendTx([new TransactionInstruction({
  programId: VOTING_PROGRAM,
  data: Buffer.from([3, cpiBump, 0]), // disc=3, is_yes=0
  keys: [
    { pubkey: proposal.pda, isSigner: false, isWritable: true },
    { pubkey: noReq.publicKey, isSigner: true, isWritable: true },
    { pubkey: proposal.noCt, isSigner: false, isWritable: false },
    ...encCpi(),
  ],
})), [noReq]);

// 3. Poll until both are decrypted
await pollUntil(connection, yesReq.publicKey, isDecrypted);
await pollUntil(connection, noReq.publicKey, isDecrypted);

// 4. Reveal yes (authority signature required)
await sendTx([new TransactionInstruction({
  programId: VOTING_PROGRAM,
  data: Buffer.from([4, 1]), // disc=4, is_yes=1
  keys: [
    { pubkey: proposal.pda, isSigner: false, isWritable: true },
    { pubkey: yesReq.publicKey, isSigner: false, isWritable: false },
    { pubkey: wallet.publicKey, isSigner: true, isWritable: false },
  ],
})));

// 5. Reveal no
await sendTx([new TransactionInstruction({
  programId: VOTING_PROGRAM,
  data: Buffer.from([4, 0]), // disc=4, is_yes=0
  keys: [
    { pubkey: proposal.pda, isSigner: false, isWritable: true },
    { pubkey: noReq.publicKey, isSigner: false, isWritable: false },
    { pubkey: wallet.publicKey, isSigner: true, isWritable: false },
  ],
})));

// 6. Read final results from on-chain proposal account
const propData = (await connection.getAccountInfo(proposal.pda))!.data as Buffer;
const yesCount = Number(propData.readBigUInt64LE(138));
const noCount = Number(propData.readBigUInt64LE(146));

```

Multi-wallet support via URL sharing

When a proposal is created, the URL is updated with query params:

```
const params = new URLSearchParams({
  proposal: pda.toBase58(),
  yesCt: yesCt.toBase58(),
  noCt: noCt.toBase58(),
});
window.history.replaceState({}, "", `?${params}`);
```

Other voters can open this URL in their browser. On mount, the app reads the URL params and loads the proposal from on-chain state:

```
useEffect(() => {
  const params = new URLSearchParams(window.location.search);
  const pdaStr = params.get("proposal");
  const yesStr = params.get("yesCt");
  const noStr = params.get("noCt");
  if (pdaStr && yesStr && noStr) {
    const pda = new PublicKey(pdaStr);
    connection.getAccountInfo(pda).then((info) => {
      if (!info) return;
      const d = info.data as Buffer;
      const isOpen = d[129] === 1;
      const totalVotes = Number(d.readBigUInt64LE(130));
      setProposal({ pda, yesCt: new PublicKey(yesStr), noCt: new
        PublicKey(noStr), isOpen, totalVotes, /* ... */ });
    });
  }
}, [connection]);
```

A “Copy Voting Link” button makes sharing easy.

Running on Devnet

The app connects to Solana devnet and the pre-alpha executor automatically. No local validator or executor setup is needed.

```
cd chains/solana/examples/voting/react
bun run dev
```

Open `http://localhost:5173`, connect a wallet (e.g. Phantom set to devnet), airdrop SOL, create a proposal, share the link with other wallets, vote, close, and decrypt.

E2E Confidential Voting Demo

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Run a full confidential voting scenario against the Encrypt pre-alpha on Solana devnet.

Prerequisites

- Rust toolchain (edition 2024)
- [just](#) command runner
- [bun](#) (for TypeScript demos)
- Solana CLI (for airdrop)

Install JS dependencies:

```
bun install
```

Quick Start

The demos connect to the pre-alpha executor on devnet automatically:

```
just demo-web3 <ENCRYPT_PROGRAM_ID> <VOTING_PROGRAM_ID>
```

Available Demos

Command	SDK	File
<code>just demo-web3 <ENC> <VOTE></code>	@solana/web3.js (v1)	e2e-voting-web3.ts

Command	SDK	File
<code>just demo-kit <ENC> <VOTE></code>	@solana/kit (v2)	<code>e2e-voting-kit.ts</code>
<code>just demo-gill <ENC> <VOTE></code>	<code>gill</code>	<code>e2e-voting-gill.ts</code>
<code>just demo-rust <ENC> <VOTE></code>	<code>solana-sdk (Rust)</code>	<code>e2e-voting-rust/</code>
<code>just demo <ENC> <VOTE></code>	All four sequentially	

What the Demo Does

1. **Create proposal** — initializes encrypted yes/no tally counters (both start at 0)
2. **Cast 5 votes** — 3 YES + 2 NO, each as an encrypted boolean via `execute_graph`
3. **Close proposal** — authority locks voting
4. **Request decryption** — asks executor to decrypt the tally ciphertexts
5. **Reveal results** — reads decrypted values on-chain

Expected output:

== Results ==

- Total votes: 5
- Yes votes: 3
- No votes: 2

Proposal PASSED (3 yes / 2 no)

How It Works

The demo script acts as a **user client only** — it encrypts values client-side, submits them via gRPC, and sends on-chain transactions. The pre-alpha executor running on devnet handles everything else:

1. Script encrypts vote value and submits via **gRPC** `CreateInput` → executor verifies proof + creates ciphertext on-chain → returns account address
2. Script sends `cast_vote` on voting program (CPI to `execute_graph`)
3. Executor detects `GraphExecuted` event, evaluates the graph, and calls `commit_ciphertext`
4. Script sends `request_decryption` on voting program (CPI to `request_decryption`)
5. Executor detects `DecryptionRequested` event, decrypts, and calls `respond_decryption`

No authority keypair needed — the client never touches the executor's keys.

Client SDK Usage

Rust:

```
let mut encrypt = EncryptClient::connect().await?;
let vote_ct = encrypt.create_input::<Bool>(true, &program_id,
&network_key).await?;
```

TypeScript:

```
const encrypt = createEncryptClient(); // connects to pre-alpha endpoint
const { ciphertextIdentifiers } = await encrypt.createInput({
  chain: Chain.Solana,
  inputs: [{ ciphertextBytes: mockCiphertext(1n), fheType: 0 }],
  authorized: programId.toBytes(),
  networkEncryptionPublicKey: networkKey,
});
```

Reading Ciphertexts Off-Chain

Rust:

```
let result = client.read_ciphertext(&ct_pubkey, &reencryption_key, epoch,
&keypair).await?;
// result.value = plaintext bytes (mock) or re-encrypted ct (production)
```

TypeScript:

```
import { encodeReadCiphertextMessage } from "@encrypt.xyz/pre-alpha-solana-
client/grpc";

const msg = encodeReadCiphertextMessage(Chain.Solana, ctId, new Uint8Array(),
1n);
const result = await encrypt.readCiphertext({
  message: msg,
  signature: Buffer.alloc(64), // not needed for public ciphertexts
  signer: Buffer.alloc(32),
});
```

For private ciphertexts, sign the BCS message with your ed25519 keypair.

Troubleshooting

Airdrop failed — Solana devnet faucet may be rate-limited. Wait a few seconds and retry, or fund the keypair manually.

Transaction simulation failed — the program may have been wiped (pre-alpha data is reset periodically). Check the program ID is still deployed.

Timeout waiting for executor — the pre-alpha executor may be restarting. Wait a minute and retry.

Encrypted ACL: Overview

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

What We’re Building

An on-chain access control list where permissions are stored as encrypted bitmasks. Nobody – not validators, not explorers, not other users – can see what permissions are set. Grant, revoke, and check operations happen via FHE bitwise operations on encrypted `u64` values.

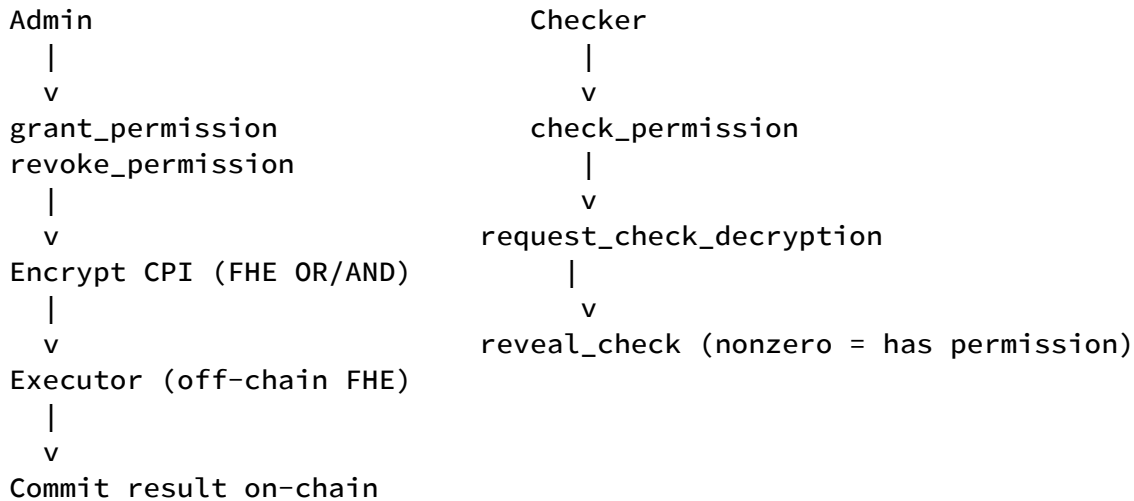
Permission Model

Permissions are a 64-bit bitmask. Each bit represents a capability:

Bit	Value	Permission
0	1	READ
1	2	WRITE
2	4	EXECUTE
3	8	ADMIN
...	...	Custom

All operations work on `EUint64` (encrypted unsigned 64-bit integer).

Architecture



Three FHE operations:

- **Grant:** `permissions | permission_bit` (bitwise OR)
- **Revoke:** `permissions & revoke_mask` (bitwise AND with inverse mask)
- **Check:** `permissions & permission_bit` (bitwise AND; nonzero = permitted)

What You'll Learn

- Multiple FHE graphs in one program (grant, revoke, check)
- The inverse mask pattern for revocation
- Two state accounts (`Resource` + `AccessCheck`) with separate decryption flows
- Admin-gated operations vs. public permission checks

Prerequisites

- Rust (edition 2024)
- Solana CLI + Platform Tools v1.54
- Anchor framework

Encrypted ACL: Building the Program

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

1. Cargo.toml

```
[package]
name = "encrypted-acl-anchor"
edition.workspace = true

[dependencies]
encrypt-types = { workspace = true }
encrypt-dsl = { package = "encrypt-solana-dsl", path = "../../program-sdk/dsl" }
encrypt-anchor = { workspace = true }
anchor-lang = { workspace = true }

[lib]
crate-type = ["cdylib", "lib"]
```

Same three Encrypt crates as the counter example.

2. FHE Graphs

Three graphs, all operating on `EUint64` bitmasks:

```

use encrypt_dsl::prelude::encrypt_fn;
use encrypt_types::encrypted::EUint64;

/// Grant: permissions = permissions | permission_bit
#[encrypt_fn]
fn grant_permission_graph(permissions: EUint64, permission_bit: EUint64) ->
EUint64 {
    permissions | permission_bit
}

/// Revoke: permissions = permissions & revoke_mask
#[encrypt_fn]
fn revoke_permission_graph(permissions: EUint64, revoke_mask: EUint64) ->
EUint64 {
    permissions & revoke_mask
}

/// Check: result = permissions & permission_bit
#[encrypt_fn]
fn check_permission_graph(permissions: EUint64, permission_bit: EUint64) ->
EUint64 {
    permissions & permission_bit
}

```

Each `#[encrypt_fn]` generates:

- A function returning the serialized graph bytes (e.g. `grant_permission_graph() -> Vec<u8>`)
- A CPI trait method on `EncryptContext` (e.g. `encrypt_ctx.grant_permission_graph(in1, in2, out)`)

All three graphs have 2 inputs and 1 output. The graph nodes are: `Input(0)` , `Input(1)` , `Op(BitOr or BitAnd)` , `Output` .

3. State Accounts

Resource

```

#[account]
#[derive(InitSpace)]
pub struct Resource {
    pub admin: Pubkey,           // who can grant/revoke
    pub resource_id: [u8; 32],   // unique ID, PDA seed
    pub permissions: [u8; 32],   // ciphertext account pubkey
    pub pending_digest: [u8; 32], // for permissions decryption
    pub revealed_permissions: u64, // plaintext after admin decrypts
    pub bump: u8,
}

```

`permissions` stores the pubkey of a ciphertext account holding the encrypted bitmask. Only the `admin` can grant, revoke, or decrypt.

AccessCheck

```
#[account]
#[derive(InitSpace)]
pub struct AccessCheck {
    pub checker: Pubkey,           // who requested the check
    pub result_ct: [u8; 32],       // ciphertext account pubkey (AND result)
    pub pending_digest: [u8; 32],  // for check decryption
    pub revealed_result: u64,      // nonzero = has permission
    pub bump: u8,
}
```

Created per-check. The PDA is seeded by `["check", resource_id, checker_pubkey]`. The `result_ct` holds the encrypted AND result. After decryption, `revealed_result > 0` means the permission is granted.

4. Instructions Walkthrough

create_resource

```
pub fn create_resource(
    ctx: Context<CreateResource>,
    resource_id: [u8; 32],
    permissions_ct_id: [u8; 32],
) -> Result<()> {
    let res = &mut ctx.accounts.resource;
    res.admin = ctx.accounts.admin.key();
    res.resource_id = resource_id;
    res.permissions = permissions_ct_id;
    res.pending_digest = [0u8; 32];
    res.revealed_permissions = 0;
    res.bump = ctx.bumps.resource;
    Ok(())
}
```

The caller creates an encrypted zero off-chain and passes its pubkey as `permissions_ct_id`. The PDA seeds are `["resource", resource_id]`.

grant_permission

```
pub fn grant_permission(
    ctx: Context<GrantPermission>,
    cpi_authority_bump: u8,
) -> Result<()> {
    let res = &ctx.accounts.resource;
    require!(
        res.admin == ctx.accounts.admin.key(),
        AclError::Unauthorized
    );

    let encrypt_ctx = EncryptContext { /* ... */ };

    let permissions_ct = ctx.accounts.permissions_ct.to_account_info();
    let permission_bit_ct = ctx.accounts.permission_bit_ct.to_account_info();
    encrypt_ctx.grant_permission_graph(
        permissions_ct.clone(), // input: current permissions
        permission_bit_ct,      // input: bit to grant
        permissions_ct,         // output: updated permissions (in-place)
    )?;

    Ok(())
}
```

Admin-only. The `permission_bit_ct` is an encrypted ciphertext containing the bit value to grant (e.g., encrypted `1` for READ, encrypted `2` for WRITE). The output overwrites the input – in-place update via `permissions | bit`.

revoke_permission

```
pub fn revoke_permission(
    ctx: Context<RevokePermission>,
    cpi_authority_bump: u8,
) -> Result<()> {
    let res = &ctx.accounts.resource;
    require!(
        res.admin == ctx.accounts.admin.key(),
        AclError::Unauthorized
    );

    let encrypt_ctx = EncryptContext { /* ... */ };

    let permissions_ct = ctx.accounts.permissions_ct.to_account_info();
    let revoke_mask_ct = ctx.accounts.revoke_mask_ct.to_account_info();
    encrypt_ctx.revoke_permission_graph(
        permissions_ct.clone(), // input: current permissions
        revoke_mask_ct,         // input: inverse mask
        permissions_ct,         // output: updated permissions (in-place)
    );

    Ok(())
}
```

The Revoke Mask Pattern

To revoke a permission, the caller passes an **inverse mask** – all bits set except the one to revoke. For example:

- Revoke READ (bit 0): mask = 0xFFFFFFFFFFFFFFFE
- Revoke WRITE (bit 1): mask = 0xFFFFFFFFFFFFFFFD
- Revoke EXECUTE (bit 2): mask = 0xFFFFFFFFFFFFFFFB

The FHE operation is `permissions & mask`, which clears exactly the target bit while preserving all others.

Why not use NOT + AND? Because FHE NOT on the permission bit would require an extra graph node and the caller already knows which bit to revoke. Passing the inverse mask is simpler and more gas-efficient.

check_permission

```
pub fn check_permission(
    ctx: Context<CheckPermission>,
    cpi_authority_bump: u8,
) -> Result<()> {
    let encrypt_ctx = EncryptContext { /* ... */ };

    let permissions_ct = ctx.accounts.permissions_ct.to_account_info();
    let permission_bit_ct = ctx.accounts.permission_bit_ct.to_account_info();
    let result_ct = ctx.accounts.result_ct.to_account_info();
    encrypt_ctx.check_permission_graph(
        permissions_ct,      // input: current permissions (read-only)
        permission_bit_ct,   // input: bit to check
        result_ct,           // output: AND result (separate account)
    )?;

    let chk = &mut ctx.accounts.access_check;
    chk.checker = ctx.accounts.checker.key();
    chk.result_ct = ctx.accounts.result_ct.key().to_bytes();
    chk.pending_digest = [0u8; 32];
    chk.revealed_result = 0;
    chk.bump = ctx.bumps.access_check;

    Ok(())
}
```

Unlike grant/revoke, check uses a **separate output account** (`result_ct`) so the permissions bitmask is not modified. Anyone can check – no admin requirement.

The `AccessCheck` PDA is created in the same instruction:

```
#[derive(Accounts)]
pub struct CheckPermission<'info> {
    pub resource: Account<'info, Resource>,
    #[account(
        init,
        payer = payer,
        space = 8 + AccessCheck::INIT_SPACE,
        seeds = [b"check", resource.resource_id.as_ref(),
        checker.key().as_ref()],
        bump,
    )]
    pub access_check: Account<'info, AccessCheck>,
    pub checker: Signer<'info>,
    // ... encrypt CPI accounts ...
}
```

request_check_decryption

```
pub fn request_check_decryption(
    ctx: Context<RequestCheckDecryption>,
    cpi_authority_bump: u8,
) -> Result<()> {
    let encrypt_ctx = EncryptContext { /* ... */ };

    let digest = encrypt_ctx.request_decryption(
        &ctx.accounts.request_acct.to_account_info(),
        &ctx.accounts.result_ciphertext.to_account_info(),
    );

    let chk = &mut ctx.accounts.access_check;
    chk.pending_digest = digest;

    Ok(())
}
```

Same digest pattern as the counter. The checker requests decryption of the AND result, stores the digest, then waits for the decryptor.

reveal_check

```
pub fn reveal_check(ctx: Context<RevealCheck>) -> Result<()> {
    let chk = &ctx.accounts.access_check;
    require!(
        chk.checker == ctx.accounts.checker.key(),
        AclError::Unauthorized
    );

    let expected_digest = &chk.pending_digest;
    let req_data = ctx.accounts.request_acct.try_borrow_data()?;
    use encrypt_types::encrypted::Uint64;
    let value = encrypt_anchor::accounts::read_decrypted_verified::<Uint64>(
        &req_data,
        expected_digest,
    )
    .map_err(|_| AclError::DecryptionNotComplete)?;

    let chk = &mut ctx.accounts.access_check;
    chk.revealed_result = *value;

    Ok(())
}
```

After reveal, `revealed_result > 0` means the user has the checked permission.
`revealed_result == 0` means they don't.

request_permissions_decryption / reveal_permissions

Admin-only decryption of the full permissions bitmask. Same pattern as `request_check_decryption / reveal_check`, but writes to `resource.revealed_permissions`.

```
pub fn request_permissions_decryption(
    ctx: Context<RequestPermissionsDecryption>,
    cpi_authority_bump: u8,
) -> Result<()> {
    let encrypt_ctx = EncryptContext { /* ... */ };

    let digest = encrypt_ctx.request_decryption(
        &ctx.accounts.request_acct.to_account_info(),
        &ctx.accounts.permissions_ciphertext.to_account_info(),
    );

    let res = &mut ctx.accounts.resource;
    res.pending_digest = digest;
    Ok(())
}

pub fn reveal_permissions(ctx: Context<RevealPermissions>) -> Result<()> {
    let res = &ctx.accounts.resource;
    require!(
        res.admin == ctx.accounts.admin.key(),
        AclError::Unauthorized
    );

    let expected_digest = &res.pending_digest;
    let req_data = ctx.accounts.request_acct.try_borrow_data()?;
    use encrypt_types::encrypted::Uint64;
    let value = encrypt_anchor::accounts::read_decrypted_verified::<Uint64>(
        &req_data,
        expected_digest,
    )
    .map_err(|_| AclError::DecryptionNotComplete)?;

    let res = &mut ctx.accounts.resource;
    res.revealed_permissions = *value;
    Ok(())
}
```

5. Instruction Summary

#	Instruction	Who	FHE Op	Modifies permissions?
1	create_resource	admin	none	initializes
2	grant_permission	admin	OR	yes (in-place)

#	Instruction	Who	FHE Op	Modifies permissions?
3	revoke_permission	admin	AND	yes (in-place)
4	check_permission	anyone	AND	no (separate output)
5	request_check_decryption	checker	none	no
6	reveal_check	checker	none	no
7	request_permissions_decryption	admin	none	no
8	reveal_permissions	admin	none	no

6. Error Codes

```
#[error_code]
pub enum AclError {
    #[msg("Unauthorized")]
    Unauthorized,
    #[msg("Decryption not complete")]
    DecryptionNotComplete,
}
```

Encrypted ACL: Testing

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

1. Unit Tests (Graph Logic)

Verify the FHE graphs produce correct bitwise results using a mock evaluator. No SBF build needed.

```
cargo test -p encrypted-acl-anchor --lib
```

```
#[test]
fn grant_single_permission() {
    let r = run_mock(
        grant_permission_graph,
        &[0, 1],
        &[FheType::EUint64, FheType::EUint64],
    );
    assert_eq!(r[0], 1, "granting READ (bit 0) to 0 should yield 1");
}

#[test]
fn grant_multiple_permissions() {
    let r = run_mock(
        grant_permission_graph,
        &[1, 2],
        &[FheType::EUint64, FheType::EUint64],
    );
    assert_eq!(r[0], 3, "granting WRITE (bit 1) to READ (1) should yield 3");
}

#[test]
fn revoke_permission() {
    let r = run_mock(
        revoke_permission_graph,
        &[3, 0xFFFFFFFFFFFFFFFF],
        &[FheType::EUint64, FheType::EUint64],
    );
    assert_eq!(r[0], 2, "revoking READ (bit 0) from 3 should yield 2");
}

#[test]
fn check_has_permission() {
    let r = run_mock(
        check_permission_graph,
        &[5, 1],
        &[FheType::EUint64, FheType::EUint64],
    );
    assert_eq!(r[0], 1, "checking READ on 5 (READ|EXECUTE) should yield 1");
}

#[test]
fn check_missing_permission() {
    let r = run_mock(
        check_permission_graph,
        &[4, 1],
        &[FheType::EUint64, FheType::EUint64],
    );
    assert_eq!(r[0], 0, "checking READ on 4 (EXECUTE only) should yield 0");
}

#[test]
fn graph_shapes() {
    let d = grant_permission_graph();
    let pg = parse_graph(&d).unwrap();
    assert_eq!(pg.header().num_inputs(), 2);
    assert_eq!(pg.header().num_outputs(), 1);
}
```

```
    // Same shape for revoke and check  
}
```

2. LiteSVM Integration Tests (E2E)

Full lifecycle tests using LiteSVM with `EncryptTestContext`. Tests the complete flow: create resource, grant, revoke, check, and decrypt.

```
just build-sbf-examples  
cargo test -p encrypted-acl-anchor --test litesvm
```

The test helpers abstract common patterns:

```
// Create a resource with encrypted-zero permissions  
fn create_resource(ctx, program_id, admin) -> (resource_pda, permissions_ct,  
resource_id)  
  
// Grant a permission: create encrypted bit, CPI, evaluate graph  
fn do_grant(ctx, program_id, ..., permission_value: u128)  
  
// Revoke a permission: create encrypted mask, CPI, evaluate graph  
fn do_revoke(ctx, program_id, ..., revoke_mask: u128)  
  
// Check a permission: create encrypted bit + result, CPI, evaluate, decrypt  
fn do_check(ctx, program_id, ..., permission_value: u128) -> u128
```

Full lifecycle test:

```
#[test]
fn test_full_acl_lifecycle() {
    let mut ctx = EncryptTestContext::new_default();
    let (program_id, cpi_authority, cpi_bump) = setup_anchor_program(&mut ctx);
    let admin = ctx.new_funded_keypair();

    // 1. Create resource
    let (resource_pda, perm_ct, resource_id) =
        create_resource(&mut ctx, &program_id, &admin);

    // 2. Grant READ (bit 0 = 1)
    do_grant(&mut ctx, &program_id, &cpi_authority, cpi_bump, &admin,
        &resource_pda, &perm_ct, 1);

    // 3. Grant WRITE (bit 1 = 2)
    do_grant(&mut ctx, &program_id, &cpi_authority, cpi_bump, &admin,
        &resource_pda, &perm_ct, 2);

    // 4. Check READ -- should pass
    let checker1 = ctx.new_funded_keypair();
    let result = do_check(&mut ctx, &program_id, &cpi_authority, cpi_bump,
        &checker1, &resource_pda, &perm_ct, &resource_id, 1);
    assert_eq!(result, 1, "should have READ after granting");

    // 5. Revoke READ (mask = 0xFFFFFFFFFFFFFFFE)
    do_revoke(&mut ctx, &program_id, &cpi_authority, cpi_bump, &admin,
        &resource_pda, &perm_ct, 0xFFFFFFFFFFFFFFFE);

    // 6. Check READ -- should fail
    let checker2 = ctx.new_funded_keypair();
    let result = do_check(&mut ctx, &program_id, &cpi_authority, cpi_bump,
        &checker2, &resource_pda, &perm_ct, &resource_id, 1);
    assert_eq!(result, 0, "should NOT have READ after revoking");

    // 7. Decrypt permissions to verify = 2 (WRITE only)
    let perm_value = ctx.decrypt_from_store(&perm_ct);
    assert_eq!(perm_value, 2, "permissions should be 2 (WRITE only)");
}
```

3. Mollusk Instruction-Level Tests

Mollusk tests `reveal_check` and `reveal_permissions` in isolation. No CPI or Encrypt program needed – just raw account data and instruction processing.

```
just build-sbf-examples
cargo test -p encrypted-acl-anchor --test mollusk
```

Tests cover:

- `reveal_check` succeeds with matching digest
- `reveal_check` rejects wrong checker

- `reveal_check` rejects digest mismatch
- `reveal_permissions` succeeds with matching digest
- `reveal_permissions` rejects wrong admin

```
#[test]
fn test_reveal_check_success() {
    let (mollusk, pid) = setup();
    let checker = Pubkey::new_unique();
    let digest = [0xABu8; 32];

    let check_data = build_anchor_access_check(&checker, &Pubkey::new_unique(),
&digest, 0);
    let request_data = build_decryption_request_data(&digest, 1);

    let result = mollusk.process_instruction(/* ... */);
    assert!(result.program_result.is_ok());
    let revealed = u64::from_le_bytes(
        result.resulting_accounts[0].1.data[104..112].try_into().unwrap(),
    );
    assert_eq!(revealed, 1);
}

#[test]
fn test_reveal_permissions_success() {
    // Same pattern, checks resource.revealed_permissions at offset 136..144
}
```

4. Running All Tests

```
# Everything (build + all test types)
just test-examples

# Just LiteSVM e2e
just test-examples-litesvm

# Just Mollusk
just test-examples-mollusk

# Just program-test
just test-examples-program-test

# Single crate, all tests
cargo test -p encrypted-acl-anchor
```

PC-Token: Overview

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

What It Is

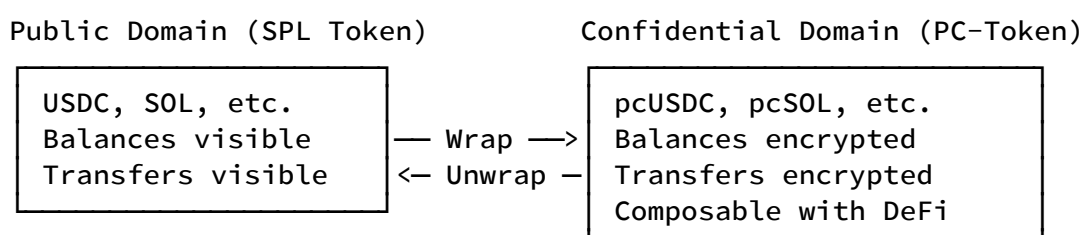
PC-Token (Confidential Performant Token) is a confidential token standard built on Encrypt FHE, modeled after Anza’s [P-Token](#). It replaces all plaintext balances and amounts with FHE-encrypted ciphertexts — same API surface as P-Token, full confidentiality.

What’s Confidential

- **Balances** — always encrypted. Nobody can see how much anyone holds.
- **Transfer amounts** — client-encrypted via gRPC. Never plaintext on-chain.
- **Allowances** — encrypted delegation for composability.
- **LP positions** — when used with PC-Swap, LP ownership is encrypted.

The only plaintext that ever appears is the withdrawal amount during unwrap, on a temporary receipt account that gets closed immediately.

Architecture



Instructions

Disc	Instruction	Description
0	InitializeMint	Create a new token mint
1	InitializeAccount	Create token account with encrypted zero balance
3	Transfer	Encrypted owner transfer
4	Approve	Approve delegate with encrypted allowance
5	Revoke	Revoke delegation
10	FreezeAccount	Freeze authority freezes account
11	ThawAccount	Freeze authority thaws account
20	TransferFrom	Delegated transfer (allowance-based composability)
22	TransferWithReceipt	Owner-signed transfer that emits a binary receipt ciphertext
23	InitializeVault	Create vault linking PC-Token mint to SPL mint
30	Wrap	Deposit SPL → mint pcTokens (vault-backed)
31	UnwrapBurn	Burn pcTokens, create withdrawal receipt
32	UnwrapDecrypt	Decrypt burned amount
33	UnwrapComplete	Verify + release SPL, close receipt

Vault-Backed Only

There is no standalone `MintTo` or `Burn`. Tokens can only enter through `Wrap` (backed 1:1 by SPL tokens in the vault) and exit through the 3-step unwrap. Every pcToken is backed.

Composability

PC-Token offers two composition patterns. Pick the one that matches the trust the calling program needs.

Allowance-based (Approve + TransferFrom)

The user `Approve`s a delegate program with an encrypted allowance, then the delegate calls `TransferFrom` to move tokens. The delegate never sees plaintext — the allowance and amount are checked in FHE atomically. Suitable for flows where the calling program's only

role is *authorized delivery* and it never needs proof that the transfer actually happened (e.g. a streaming-payments program that never reads downstream state).

Receipt-based (TransferWithReceipt)

The owner CPIs (or directly calls) `TransferWithReceipt`, supplying a fresh receipt-ciphertext keypair and a `target_program` (typically the calling program's ID). The same FHE graph that updates `from.balance` and `to.balance` also outputs a *binary* receipt ciphertext:

```
receipt = amount    if from_balance >= amount
receipt = 0         otherwise
```

`TransferWithReceipt` then transfers the receipt's ACL to `target_program`, so the caller can read it in its own FHE graphs. This gives a downstream program a faithful, encrypted, *gated* signal of the deposit — never a partial value, never the from-balance — without any plaintext crossing the boundary. It is the pattern PC-Swap uses to keep its reserves consistent with what users actually deposited (see [PC-Swap: Overview](#)).

```
// In the calling program (e.g. PC-Swap):
// 1. Make sure amount_ct is authorized to pc-token
ctx.transfer_ciphertext(amount_ct, pc_token_program)?;
// 2. CPI TransferWithReceipt – emits receipt_ct, ACL goes to caller
cpi_pc_token_transfer_with_receipt(receipt_ct, target = caller_program_id)?;
// 3. Use receipt_ct in the calling program's own graphs (gates state
updates)
ctx.my_graph(reserve_in, reserve_out, receipt_ct, ...)?;
// 4. Reclaim the receipt's rent
ctx.close_ciphertext(receipt_ct, payer)?;
```

The receipt is **owner-signed only** — there is no delegated variant. The receipt's `authorized` ACL is the calling program, so the receipt can also be decrypted by that program if it explicitly requests decryption; in practice composable programs never call `request_decryption` on intermediate ciphertexts and the audit trail of the source code is the assurance that they don't.

PC-Token: Building the Program

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Account Layouts

Mint

Follows P-Token’s COption pattern for optional authorities:

```
pub struct Mint {  
    pub mint_authority_flag: [u8; 4],    // COption  
    pub mint_authority: [u8; 32],  
    pub decimals: u8,  
    pub is_initialized: u8,  
    pub freeze_authority_flag: [u8; 4], // COption  
    pub freeze_authority: [u8; 32],  
    pub bump: u8,  
}
```

TokenAccount

No plaintext fields. Balance is always encrypted:

```
pub struct TokenAccount {
    pub mint: [u8; 32],
    pub owner: [u8; 32],
    pub balance: EUint64,           // encrypted balance
    pub delegate_flag: [u8; 4],    // COption
    pub delegate: [u8; 32],
    pub state: u8,                 // Uninitialized/Initialized/Frozen
    pub allowance: EUint64,        // encrypted delegate allowance
    pub close_authority_flag: [u8; 4], // COption
    pub close_authority: [u8; 32],
    pub bump: u8,
}
```

FHE Graphs

Transfer (conditional)

```
#[encrypt_fn]
fn transfer_graph(
    from_balance: EUint64, to_balance: EUint64, amount: EUint64,
) -> (EUint64, EUint64) {
    let sufficient = from_balance >= amount;
    let new_from = if sufficient { from_balance - amount } else { from_balance };
};
let new_to = if sufficient { to_balance + amount } else { to_balance };
(new_from, new_to)
}
```

If the sender has insufficient funds, both balances remain unchanged — a privacy-preserving silent no-op. The chain cannot distinguish success from failure.

Delegated Transfer (allowance composability)

```
#[encrypt_fn]
fn transfer_from_graph(
    from_balance: EUint64, to_balance: EUint64,
    allowance: EUint64, amount: EUint64,
) -> (EUint64, EUint64, EUint64) {
    let sufficient_balance = from_balance >= amount;
    let sufficient_allowance = allowance >= amount;
    let can_transfer = sufficient_balance & sufficient_allowance;
    // if either check fails → no-op
    let new_from = if can_transfer { from_balance - amount } else {
from_balance };
    let new_to = if can_transfer { to_balance + amount } else { to_balance };
    let new_allowance = if can_transfer { allowance - amount } else { allowance
};
    (new_from, new_to, new_allowance)
}
```

Both balance AND allowance are checked atomically in the encrypted domain.

Transfer with Receipt (receipt-gated composability)

Same balance arithmetic as `transfer_graph`, plus a third output that is a *binary* receipt — exactly `amount` on a successful transfer, exactly `0` on insufficient balance, never a partial value. Downstream programs multiply their state updates by the receipt to gate them in lockstep with the actual transfer:

```
#[encrypt_fn]
fn transfer_receipt_graph(
    from_balance: EUint64, to_balance: EUint64, amount: EUint64,
) -> (EUint64, EUint64, EUint64) {
    let sufficient = from_balance >= amount;
    let zero = amount - amount;
    let actual = if sufficient { amount } else { zero };
    let new_from = from_balance - actual;
    let new_to = to_balance + actual;
    (new_from, new_to, actual)
}
```

The `TransferWithReceipt` handler:

1. Authorizes the transfer (owner-signer only — there is no delegated variant).
2. Calls `create_plaintext_typed::<Uint64>(0, receipt_ct)` to allocate the receipt account at a caller-supplied keypair, initially authorized to `pc-token`.
3. Runs `transfer_receipt_graph` with the three balance ciphertexts plus the receipt as outputs.
4. Calls `transfer_ciphertext(receipt_ct, target_program)` to move the receipt's ACL to whatever program the caller asked for.

After the instruction returns, the receipt sits Pending on-chain with its digest still being committed by the executor's normal event processing. By the time a downstream graph in the same transaction reads it, the digest has been written.

The **receipt invariant** (`actual ∈ {amount, 0}` , never partial) is what makes downstream gating work cleanly: a program that multiplies its state updates by the receipt either advances by the full intended amount or doesn't advance at all — there's no partial-credit edge case to write FHE branches for.

Wrap / Unwrap

Wrap (SPL → pcToken)

1. SPL transfer from user to vault (plaintext — the deposit is visible)
2. `mint_to_graph(balance, amount)` adds to encrypted balance
3. Amount ciphertext pre-created via gRPC (not `create_plaintext_typed`)

Unwrap (pcToken → SPL)

Three-step flow that only reveals the withdrawal amount:

1. **UnwrapBurn** — `unwrap_burn_graph(balance, amount) → (new_balance, burned)` .
 `burned` = amount if sufficient, 0 if not. Creates a temporary `WithdrawalReceipt` .
2. **UnwrapDecrypt** — requests decryption of `burned` ciphertext.
3. **UnwrapComplete** — verifies `burned == requested_amount` . If yes → SPL transfer from vault. If no → no-op. Closes receipt.

The balance is never decrypted. Only the withdrawal amount appears on the temporary receipt.

PC-Token: Testing

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Unit Tests

FHE graph logic tested via mock compute engine:

- `mint_to` — balance + amount
- `transfer_ok` / `transfer_insufficient` — conditional transfer
- `burn_ok` / `burn_insufficient` — conditional burn
- `transfer_from_ok` / `transfer_from_insufficient` — delegated with allowance check
- `transfer_receipt_ok` / `transfer_receipt_insufficient` — receipt is amount on success, exactly 0 on insufficient balance
- `unwrap_burn_sufficient` / `unwrap_burn_insufficient` — burn with receipt output
- `graph_shapes` — verify input/output counts

LiteSVM Integration Tests

Full on-chain lifecycle with Encrypt CPI:

- `test_initialize_mint` / `test_initialize_mint_with_freeze_authority`
- `test_initialize_account` — create token account, verify encrypted zero balance
- `test_mint_to` — set balance via harness, verify encrypted value
- `test_transfer` / `test_transfer_insufficient_funds` — encrypted transfer
- `test_approve_and_transfer_from` — delegation + delegated transfer
- `test_transfer_with_receipt_sufficient` / `test_transfer_with_receipt_insufficient_emits_zero` — `TransferWithReceipt` end-to-end, asserts the receipt’s plaintext value matches the binary invariant
- `test_freeze_blocks_transfer` — freeze/thaw cycle

- `test_full_lifecycle` — mint → transfer → freeze → thaw → approve → transfer_from

E2E Devnet Test

Full USDC → pcUSDC → USDC flow on Solana devnet:

Alice wraps 10 USDC → sends 5 pcUSDC to Bob → Bob unwraps 5 →
Alice sends 3 to Mark → Mark unwraps 2 → Alice unwraps 1 →
Final: Alice=1 USDC+1cp, Bob=5 USDC, Mark=2 USDC+1cp, Vault=2 USDC

Run:

```
cargo build-sbf --manifest-path chains/solana/examples/pc-  
token/pinocchio/Cargo.toml  
solana program deploy target/deploy/pc_token.so  
bun chains/solana/examples/pc-token/e2e/main.ts <ENCRYPT_ID> <PC_TOKEN_ID>
```

PC-Swap: Overview

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

What It Is

PC-Swap is a confidential UniV2-style AMM built on Encrypt FHE. Reserves, swap amounts, and LP positions are all encrypted, and the pool’s vaults are real PC-Token TokenAccounts owned by the pool PDA — so deposits and payouts compose with the rest of the confidential token domain through Encrypt CPI rather than through any plaintext bridge.

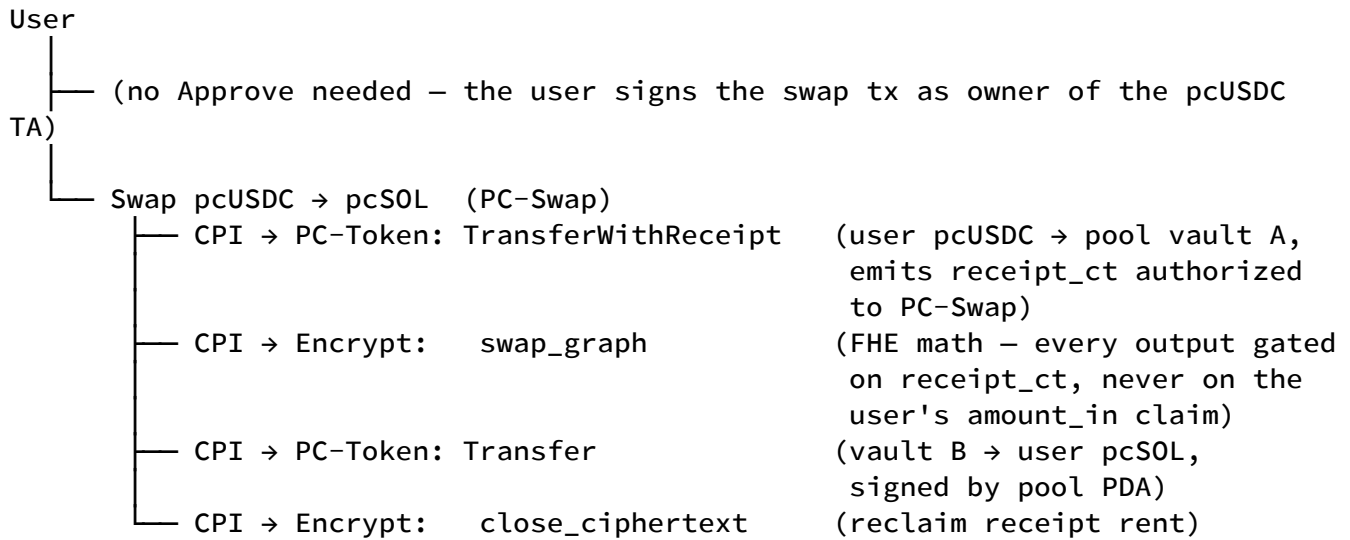
What’s Confidential

Data	Visibility
Pool reserves (TVL)	Encrypted
Swap amounts (trade sizes)	Encrypted
LP positions	Encrypted
Withdrawn amounts	Encrypted
Transaction activity	Visible — that swaps happened, not what

Composability with PC-Token

The pool keeps its own encrypted reserve mirrors (`reserve_a` , `reserve_b` , `total_supply`) inside the `Pool` account, and a pair of vault PC-Token accounts (`vault_a` , `vault_b`) owned by the pool PDA holds the actual encrypted balances of the two underlying tokens.

Every flow goes through PC-Token's `TransferWithReceipt` (disc 22, see [PC-Token: Composability](#)). The receipt that comes out is binary — `amount` on a successful deposit, `0` on insufficient balance — and PC-Swap multiplies its mirror updates and payouts by it. That single design choice keeps the pool sound under any user-supplied `amount_in`:



`add_liquidity` is the same idea with two `TransferWithReceipt` calls (one per side) feeding `add_liquidity_graph`. `remove_liquidity` doesn't need a receipt — it gates on the user's encrypted `LpPosition.balance`, which is PC-Swap-internal and already trusted.

Constant-Product Math, Receipt-Gated

The swap graph computes UniV2's $x \cdot y = k$ entirely in the encrypted domain, treating the **receipt** as the authoritative input — not the user's claim:

```

amount_in_with_fee = receipt * 997
amount_out          = (amount_in_with_fee * reserve_out)
                    / (reserve_in * 1000 + amount_in_with_fee)
  
```

- 0.3% fee baked in via the 997/1000 constants.
- Slippage protection: `amount_out >= min_amount_out`.
- If slippage fails → `final_in` and `final_out` collapse to 0 (no-op).
- If the user lied about `amount_in` (claimed more than they have) → `receipt = 0` from PC-Token → `final_in`, `final_out`, and both reserve deltas are all 0. The pool stays consistent, the user gets nothing.

LP Token Enforcement

LP shares are per-user encrypted balances in `LpPosition` accounts (PC-Swap-internal — not real PC-Token mints). The graphs atomically update reserves, total supply, and the user's LP balance:

- **AddLiquidity** — first deposit: `lp = receipt_a`. Subsequent: proportional to $\min(\text{receipt_a} \cdot \text{supply} / \text{reserve_a}, \text{receipt_b} \cdot \text{supply} / \text{reserve_b})$. Both arms gated on the receipts.
- **RemoveLiquidity** — checks `user_lp >= burn_amount` in FHE. If insufficient, the entire operation is a no-op — reserves, supply, and LP balance unchanged.

Nobody can drain more LP than they own, and the ownership check happens inside the encrypted computation.

Soundness Note

The receipt's ACL after `TransferWithReceipt` is PC-Swap. That means an audited PC-Swap binary that never calls `request_decryption` cannot leak the receipt's value — and the receipt's only value is one bit ("user was solvent for `amount_in`"). The user's signature on the swap transaction is consent for this. The principled long-term fix — splitting Encrypt's `authorized` into separate `compute_authorized` / `decrypt_authorized` ACLs so a program can read a ciphertext without being able to decrypt it — lives in the Encrypt program itself; until then, audited program source is the trust anchor.

PC-Swap: Building the Program

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Account Layouts

Pool

The pool stores its own encrypted reserve mirrors plus the two vault accounts (real PC-Token TokenAccounts owned by the pool PDA — that’s where the actual confidential balances live). PDA: `["pc_pool", mint_a, mint_b]`.

```
pub struct Pool {
  pub mint_a: [u8; 32],
  pub mint_b: [u8; 32],
  pub vault_a: [u8; 32],      // pc-token TokenAccount, owner = pool PDA
  pub vault_b: [u8; 32],      // pc-token TokenAccount, owner = pool PDA
  pub reserve_a: [u8; 32],    // encrypted reserve mirror (EUint64)
  pub reserve_b: [u8; 32],    // encrypted reserve mirror (EUint64)
  pub total_supply: [u8; 32], // encrypted LP total supply (EUint64)
  pub is_initialized: u8,
  pub bump: u8,
}
```

The reserve mirrors are kept encrypted alongside the vault state because the FHE swap math needs ciphertexts as inputs — pulling them out of the vault accounts on every swap would mean re-reading and re-validating those accounts in the graph, which would couple the pool layout to PC-Token’s. Keeping the mirrors as the source of truth for the math while the vaults hold the *actual* tokens is the same separation P-Token uses for `Mint.supply` versus per-account `balance`.

LpPosition

Per-user encrypted LP balance. PC-Swap-internal accounting — not a real PC-Token mint.

PDA: ["pc_lp", pool, owner].

```
pub struct LpPosition {
  pub pool: [u8; 32],
  pub owner: [u8; 32],
  pub balance: [u8; 32], // encrypted LP balance (EUint64)
  pub bump: u8,
}
```

FHE Graphs

Every reserve update, payout amount, and LP balance change is a function of an encrypted *receipt* — never of a user-supplied amount directly. That's the soundness invariant the graphs are organized around.

Swap

```
#[encrypt_fn]
fn swap_graph(
  reserve_in: EUint64,
  reserve_out: EUint64,
  receipt: EUint64,           // from pc-token::TransferWithReceipt
  min_amount_out: EUint64,
) -> (EUint64, EUint64, EUint64, EUint64) {
  let amount_in_with_fee = receipt * 997;
  let numer = amount_in_with_fee * reserve_out;
  let denom = (reserve_in * 1000) + amount_in_with_fee;
  let amount_out = numer / denom;

  let slip_ok = amount_out >= min_amount_out;
  let zero     = min_amount_out - min_amount_out; // typed-zero source
  let final_in = if slip_ok { receipt } else { zero };
  let final_out = if slip_ok { amount_out } else { zero };

  let new_reserve_in = reserve_in + final_in;
  let new_reserve_out = reserve_out - final_out;

  (final_in, final_out, new_reserve_in, new_reserve_out)
}
```

Note that the user's `amount_in_ct` is not an input to `swap_graph` — it has already been handed over to PC-Token for `TransferWithReceipt`. The only gate that matters for soundness is `receipt`, which is `amount_in` on a real deposit and `0` on a no-op.

`min_amount_out` doubles as the typed-zero source (`min_amount_out - min_amount_out`) for the slippage no-op branch.

Add Liquidity

Two receipts (one per token) gate everything: reserve mirrors, total supply, and the user's LP balance.

```
#[encrypt_fn]
fn add_liquidity_graph(
    reserve_a: EUint64, reserve_b: EUint64, total_supply: EUint64,
    receipt_a: EUint64, receipt_b: EUint64,
    user_lp: EUint64,
) -> (EUint64, EUint64, EUint64, EUint64, EUint64, EUint64) {
    let new_reserve_a = reserve_a + receipt_a;
    let new_reserve_b = reserve_b + receipt_b;

    let initial_lp      = receipt_a;
    let lp_from_a       = (receipt_a * total_supply) / (reserve_a + 1);
    let lp_from_b       = (receipt_b * total_supply) / (reserve_b + 1);
    let subsequent_lp   = if lp_from_a >= lp_from_b { lp_from_b } else {
lp_from_a };
    let is_subsequent    = total_supply >= 1;
    let lp_to_mint       = if is_subsequent { subsequent_lp } else { initial_lp
};

    let new_total_supply = total_supply + lp_to_mint;
    let new_user_lp      = user_lp + lp_to_mint;

    (receipt_a, receipt_b, new_reserve_a, new_reserve_b,
     new_total_supply, new_user_lp)
}
```

If either `TransferWithReceipt` no-ops (insufficient balance on that side), the corresponding receipt is `0` and that arm of the LP math collapses, so reserves and supply stay consistent.

Remove Liquidity

The withdrawal direction doesn't need a receipt — it's gated on the user's `LpPosition.balance`, which is PC-Swap-internal and already trusted.

```
#[encrypt_fn]
fn remove_liquidity_graph(
    reserve_a: EUint64, reserve_b: EUint64, total_supply: EUint64,
    burn: EUint64, user_lp: EUint64,
) -> (EUint64, EUint64, EUint64, EUint64, EUint64, EUint64) {
    let sufficient_lp = user_lp >= burn;
    // proportional withdrawal: amount = reserve * burn / supply
    // if !sufficient_lp → all outputs collapse to no-op
}
```

Instructions

Disc	Instruction	Description
0	CreatePool	Create pool, vaults, encrypted reserves + LP supply (zero plaintexts)
1	Swap	TransferWithReceipt → swap_graph → vault→user transfer → close receipt
2	AddLiquidity	TransferWithReceipt × 2 → add_liquidity_graph → close both receipts
4	RemoveLiquidity	remove_liquidity_graph → vault→user transfer × 2
5	CreateLpPosition	Create user's LP position account

Dispatch — swap step by step

- transfer_ciphertext(amount_in_ct, target = pc-token-program)
↳ hand the user's swap-amount ciphertext over to pc-token so it can read it
- CPI pc-token::TransferWithReceipt
↳ user → vault_in deposit, emits receipt_ct, ACL goes back to pc-swap
- swap_graph(reserve_in, reserve_out, receipt_ct, min_out_ct)
↳ receipt-gated math; outputs final_in, final_out, new reserves
- transfer_ciphertext(amount_out_ct, target = pc-token-program)
↳ hand the payout ciphertext to pc-token for the second leg
- CPI pc-token::Transfer (signed by pool PDA)
↳ vault_out → user payout
- close_ciphertext(receipt_ct, payer)
↳ reclaim the receipt's rent

add_liquidity is two of step (2) before its graph; remove_liquidity is just steps (3)-(5) in the reverse direction with no receipt because the gate is user_lp instead.

Security

- **Receipt-gated soundness** — every reserve / payout / LP-supply update is a function of a `TransferWithReceipt` receipt. A user lying about `amount_in` produces `receipt = 0` and every output collapses uniformly.
- **LP ownership** — `lp_pos.owner == payer` checked on every add/remove.
- **LP balance** — `user_lp >= burn_amount` checked in FHE on remove.
- **Slippage** — `amount_out >= min_amount_out` checked in FHE on swap.
- **No drain** — can't use another user's `LpPosition` (owner check).
- **No decrypt** — reserves and LP positions are permanently encrypted; the binary receipt is the only ciphertext PC-Swap holds an ACL for, and PC-Swap never calls `request_decryption`.

PC-Swap: Testing

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Unit Tests

FHE graph soundness against the mock compute engine:

- `swap_basic` — receipt = amount_in case: output and reserves move, k-invariant holds.
- `swap_slippage` — `min_out` higher than achievable: `final_in` and `final_out` collapse to 0, reserves unchanged.
- `swap_lying_user_receipt_zero` — receipt = 0 simulates a no-op deposit: every output is 0, reserves stay put.
- `add_liq_first` — first deposit, `lp = receipt_a`.
- `add_liq_second` — proportional LP minting against existing reserves.
- `add_liq_lying_a_receipt_zero` — one side's receipt is 0: only the honest side's reserve advances, `lp_to_mint = 0`, supply unchanged, user LP unchanged.
- `remove_liq_sufficient` — proportional withdrawal, LP decremented.
- `remove_liq_insufficient` — burn more than `user_lp`: full no-op.
- `graph_shapes` — verify input/output counts (`swap_graph` 4/4, `add_liquidity_graph` 6/6, `remove_liquidity_graph` 5/6).

E2E Test

The end-to-end runner exercises the program against a live executor (devnet or localnet — `RPC_URL` and `GRPC_URL` env vars override the defaults). Stages:

1. Create mock SPL mints (USD, DOGE)
2. Create pcUSD + pcDOGE mints, vaults, user accounts
3. Wrap 1000 USD + 1000 DOGE into pcTokens
4. Create the pool (encrypted zero reserves + LP supply) + LP position
5. AddLiquidity (500 + 500): two TransferWithReceipt CPIs feed
add_liquidity_graph
6. Swap 50 pcUSD → pcDOGE: TransferWithReceipt feeds swap_graph
7. Swap with unattainable min_out: in-FHE no-op, no plaintext difference
8. Lying user – claim amount_in = 99,999 pcUSD with ~450 left:
receipt = 0 → reserves unchanged, user pcDOGE balance unchanged
9. RemoveLiquidity: vault → user transfers signed by the pool PDA

Stage 8 is the soundness assertion: the only way to pass it is for receipt-gating to flow all the way through `swap_graph` and the cluster commit, leaving the pool exactly as it was before the lying tx.

Run:

```
cargo build-sbf --manifest-path chains/solana/examples/pc-  
token/pinocchio/Cargo.toml  
cargo build-sbf --manifest-path chains/solana/examples/pc-  
swap/pinocchio/Cargo.toml  
solana program deploy target/deploy/pc_token.so  
solana program deploy target/deploy/pc_swap.so  
bun chains/solana/examples/pc-swap/e2e/main.ts <ENCRYPT_ID> <PC_TOKEN_ID>  
<PC_SWAP_ID>
```

Instruction Reference

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

All 22 instructions in the Encrypt Solana program. The first byte of instruction data is the discriminator.

Instruction Groups

Group	Disc Range	Instructions
Setup	0	initialize
Executor	1–6	create_input_ciphertext, create_plaintext_ciphertext, commit_ciphertext, execute_graph, register_graph, execute_registered_graph
Ownership	7–9	transfer_ciphertext, copy_ciphertext, make_public
Gateway	10–12	request_decryption, respond_decryption, close_decryption_request
Fees	13–18	create_deposit, top_up, withdraw, update_config_fees, reimburse, request_withdraw
Authority	19–21	add_authority, remove_authority, register_network_encryption_key
Event	228	emit_event

Setup

initialize (disc 0)

One-time program initialization. Creates the EncryptConfig and initial Authority PDAs.

Accounts:

#	Account	W	S	Description
0	config	yes	no	EncryptConfig PDA (must be empty)
1	authority_pda	yes	no	Authority PDA (must be empty)
2	initializer	no	yes	Initial authority signer
3	payer	yes	yes	Rent payer
4	system_program	no	no	System program

Data (2 bytes): config_bump(1) | authority_bump(1)

Executor

create_input_ciphertext (disc 1)

Authority-driven: creates a verified ciphertext from off-chain encrypted data + ZK proof.
Status = VERIFIED immediately.

Accounts:

#	Account	W	S	Description
0	authority_pda	no	no	Authority PDA
1	signer	no	yes	Authority signer
2	config	no	no	EncryptConfig
3	deposit	yes	no	EncryptDeposit (fee source)
4	ciphertext	yes	no	New Ciphertext account (must be empty)
5	creator	no	no	Who gets authorized
6	network_encryption_key	no	no	NetworkEncryptionKey PDA
7	payer	yes	yes	Rent payer
8	system_program	no	no	System program

#	Account	W	S	Description
9	event_authority	no	no	Event authority PDA
10	program	no	no	Encrypt program

Data (33 bytes): `fhe_type(1)` | `ciphertext_digest(32)`

create_plaintext_ciphertext (disc 2)

User-signed: creates a ciphertext from a plaintext value. The executor encrypts off-chain and commits later. Status = PENDING.

Supports both signer and CPI (program) callers. CPI path inserts `cpi_authority` at position 4.

Accounts (signer path):

#	Account	W	S	Description
0	config	no	no	EncryptConfig
1	deposit	yes	no	EncryptDeposit
2	ciphertext	yes	no	New Ciphertext account (must be empty)
3	creator	no	yes	Signer (gets authorized)
4	network_encryption_key	no	no	NetworkEncryptionKey PDA
5	payer	yes	yes	Rent payer
6	system_program	no	no	System program
7	event_authority	no	no	Event authority PDA
8	program	no	no	Encrypt program

Accounts (CPI path): Same as above but `cpi_authority` is inserted at position 4, shifting positions 4–8 to 5–9.

Data (1+ bytes): `fhe_type(1)` | `[plaintext_bytes(N)]`

commit_ciphertext (disc 3)

Authority writes the ciphertext digest after off-chain FHE evaluation. Sets status from PENDING to VERIFIED.

Accounts:

#	Account	W	S	Description
0	authority_pda	no	no	Authority PDA
1	signer	no	yes	Authority signer
2	ciphertext	yes	no	Ciphertext account
3	event_authority	no	no	Event authority PDA
4	program	no	no	Encrypt program

Data (32 bytes): `ciphertext_digest(32)`

execute_graph (disc 4)

Execute a computation graph. Creates/updates output ciphertext accounts. Emits `GraphExecuted` event.

Supports both signer and CPI callers. CPI path inserts `cpi_authority` at position 3.

Accounts (signer path):

#	Account	W	S	Description
0	config	no	no	EncryptConfig
1	deposit	yes	no	EncryptDeposit
2	caller	no	yes	Signer
3	network_encryption_key	no	no	NetworkEncryptionKey PDA
4	payer	yes	yes	Rent payer
5	event_authority	no	no	Event authority PDA
6	program	no	no	Encrypt program
7..7+N	input ciphertexts	no	no	Input ciphertext accounts
7+N..7+N+M	output ciphertexts	yes	no	Output ciphertext accounts

Accounts (CPI path): `cpi_authority` at position 3, remaining shifted by 1. Fixed accounts = 8 instead of 7.

Data: `graph_data_len(2)` | `graph_data(N)` | `num_inputs(2)`

register_graph (disc 5)

Register a reusable computation graph on-chain. Creates a RegisteredGraph PDA.

Accounts:

#	Account	W	S	Description
0	graph_pda	yes	no	RegisteredGraph PDA (must be empty)
1	registrar	no	yes	Signer
2	payer	yes	yes	Rent payer
3	system_program	no	no	System program

Data (35+ bytes): bump(1) | graph_hash(32) | graph_data_len(2) | graph_data(N)

execute_registered_graph (disc 6)

Execute a previously registered graph. Uses the on-chain graph data (no need to re-send).

Supports both signer and CPI callers.

Accounts (signer path):

#	Account	W	S	Description
0	config	no	no	EncryptConfig
1	deposit	yes	no	EncryptDeposit
2	graph_pda	no	no	RegisteredGraph PDA
3	caller	no	yes	Signer
4	network_encryption_key	no	no	NetworkEncryptionKey PDA
5	payer	yes	yes	Rent payer
6	event_authority	no	no	Event authority PDA
7	program	no	no	Encrypt program
8+	remaining	varies	no	Input + output ciphertexts

Accounts (CPI path): cpi_authority at position 4, fixed = 9.

Data (2 bytes): num_inputs(2)

Ownership

transfer_ciphertext (disc 7)

Transfer authorization to a new party by updating the `authorized` field.

Accounts (signer path):

#	Account	W	S	Description
0	ciphertext	yes	no	Ciphertext account
1	current_authorized	no	yes	Current authorized signer
2	new_authorized	no	no	New authorized party

Accounts (CPI path): `cpi_authority` at position 2, `new_authorized` at position 3.

Data: none

copy_ciphertext (disc 8)

Create a copy of a ciphertext with a different authorized party.

Accounts (signer path):

#	Account	W	S	Description
0	source_ciphertext	no	no	Source Ciphertext
1	new_ciphertext	yes	no	New Ciphertext account (must be empty)
2	current_authorized	no	yes	Current authorized signer
3	new_authorized	no	no	New authorized party
4	payer	yes	yes	Rent payer
5	system_program	no	no	System program

Accounts (CPI path): `cpi_authority` at position 3, remaining shifted.

Data (1 byte): `transient(1)` (0 = permanent/rent-exempt, 1 = transient/0 lamports)

make_public (disc 9)

Set `authorized` to zero (public). Irreversible and idempotent.

Accounts (signer path):

#	Account	W	S	Description
0	ciphertext	yes	no	Ciphertext account
1	caller	no	yes	Current authorized signer

Accounts (CPI path): `cpi_authority` at position 2.

Data (32 bytes): `ciphertext_id(32)`

Gateway

request_decryption (disc 10)

Request decryption of a ciphertext. Creates a DecryptionRequest account and stores a digest snapshot.

Supports both signer and CPI callers.

Accounts (signer path):

#	Account	W	S	Description
0	config	no	no	EncryptConfig
1	deposit	yes	no	EncryptDeposit
2	request_acct	yes	no	DecryptionRequest account (must be empty)
3	caller	no	yes	Signer
4	ciphertext	no	no	Ciphertext to decrypt
5	payer	yes	yes	Rent payer
6	system_program	no	no	System program
7	event_authority	no	no	Event authority PDA
8	program	no	no	Encrypt program

Accounts (CPI path): `cpi_authority` at position 4, remaining shifted. Fixed = 10.

Data: none

respond_decryption (disc 11)

Authority writes the decrypted plaintext bytes into the DecryptionRequest account.

Accounts:

#	Account	W	S	Description
0	authority_pda	no	no	Authority PDA
1	request_acct	yes	no	DecryptionRequest account
2	signer	no	yes	Authority signer
3	event_authority	no	no	Event authority PDA
4	program	no	no	Encrypt program

Data (variable): plaintext bytes chunk to write

close_decryption_request (disc 12)

Close a decryption request and reclaim rent. Only the original requester can close.

Accounts (signer path):

#	Account	W	S	Description
0	request	yes	no	DecryptionRequest account
1	caller	no	yes	Requester signer
2	destination	yes	no	Rent destination

Accounts (CPI path): `cpi_authority` at position 2, `destination` at position 3.

Data: none

Fees

create_deposit (disc 13)

Create an EncryptDeposit PDA for a user. Transfers initial ENC tokens and SOL gas.

Accounts:

#	Account	W	S	Description
0	deposit	yes	no	EncryptDeposit PDA (must be empty)
1	config	no	no	EncryptConfig
2	user	no	yes	Deposit owner
3	payer	yes	yes	Rent payer
4	user_ata	yes	no	User's ENC token account
5	vault	yes	no	Program's ENC vault token account
6	token_program	no	no	SPL Token program
7	system_program	no	no	System program

Data (17 bytes): bump(1) | initial_enc_amount(8) | initial_gas_amount(8)

top_up (disc 14)

Add ENC tokens and/or SOL gas to an existing deposit.

Accounts:

#	Account	W	S	Description
0	deposit	yes	no	EncryptDeposit PDA
1	config	no	no	EncryptConfig
2	user	no	yes	Deposit owner
3	user_ata	yes	no	User's ENC token account
4	vault	yes	no	ENC vault
5	token_program	no	no	SPL Token program
6	system_program	no	no	System program

Data (16 bytes): enc_amount(8) | gas_amount(8)

withdraw (disc 15)

Execute a pending withdrawal. Available when `current_epoch >= withdrawal_epoch`.

Accounts:

#	Account	W	S	Description
0	deposit	yes	no	EncryptDeposit PDA
1	config	no	no	EncryptConfig

#	Account	W	S	Description
2	user	no	yes	Deposit owner
3	user_ata	yes	no	User's ENC token account
4	vault	yes	no	ENC vault
5	vault_authority	no	no	Vault authority PDA
6	token_program	no	no	SPL Token program

Data: none

update_config_fees (disc 16)

Authority updates the fee schedule in EncryptConfig.

Accounts:

#	Account	W	S	Description
0	config	yes	no	EncryptConfig PDA
1	authority_pda	no	no	Authority PDA
2	signer	no	yes	Authority signer

Data (58 bytes): enc_per_input(8) | enc_per_output(8) | max_enc_per_op(8) | max_ops_per_graph(2) | gas_base(8) | gas_per_input(8) | gas_per_output(8) | gas_per_byte(8)

reimburse (disc 17)

Authority credits back the per-op max-charge overcharge after computing actual costs.

Accounts:

#	Account	W	S	Description
0	authority_pda	no	no	Authority PDA
1	signer	no	yes	Authority signer
2	deposit	yes	no	EncryptDeposit PDA

Data (16 bytes): enc_amount(8) | gas_amount(8)

request_withdraw (disc 18)

Set pending withdrawal amounts. Actual withdrawal available next epoch.

Accounts:

#	Account	W	S	Description
0	deposit	yes	no	EncryptDeposit PDA
1	config	no	no	EncryptConfig
2	user	no	yes	Deposit owner

Data (16 bytes): `enc_amount(8)` | `gas_amount(8)`

Authority

add_authority (disc 19)

Add a new authority. Must be signed by an existing authority.

Accounts:

#	Account	W	S	Description
0	new_auth	yes	no	New Authority PDA (must be empty)
1	existing_auth	no	no	Existing Authority PDA
2	signer	no	yes	Existing authority signer
3	payer	yes	yes	Rent payer
4	system_program	no	no	System program

Data (33 bytes): `bump(1)` | `new_pubkey(32)`

remove_authority (disc 20)

Deactivate an authority.

Accounts:

#	Account	W	S	Description
0	target_auth	yes	no	Authority PDA to deactivate
1	signer_auth	no	no	Signer's Authority PDA

#	Account	W	S	Description
2	signer	no	yes	Authority signer

Data: none

register_network_encryption_key (disc 21)

Register a new FHE network encryption public key.

Accounts:

#	Account	W	S	Description
0	network_encryption_key_pda	yes	no	NetworkEncryptionKey PDA (must be empty)
1	authority_pda	no	no	Authority PDA
2	signer	no	yes	Authority signer
3	payer	yes	yes	Rent payer
4	system_program	no	no	System program

Data (33 bytes): bump(1) | network_public_key(32)

Event

emit_event (disc 228)

Self-CPI event handler. Called internally by the Encrypt program to emit Anchor-compatible events. Not called by external programs.

Accounts:

#	Account	W	S	Description
0	event_authority	no	no	Event authority PDA (must match)
1	program	no	no	Encrypt program

Data: Event payload (prefixed with `EVENT_IX_TAG_LE`)

Account Reference

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

All 7 account types in the Encrypt Solana program. Each account starts with a 2-byte prefix: `discriminator(1) | version(1)`, followed by the account data.

Account Discriminators

Discriminator	Account Type
1	EncryptConfig
2	Authority
3	DecryptionRequest
4	EncryptDeposit
5	RegisteredGraph
6	Ciphertext
7	NetworkEncryptionKey

EncryptConfig (disc 1)

Program-wide configuration. PDA seeds: `["encrypt_config"]`.

Offset	Field	Size	Description
0	discriminator	1	1
1	version	1	1
2	current_epoch	8	Current epoch (LE u64)

Offset	Field	Size	Description
10	enc_per_input	8	ENC fee per input (LE u64)
18	enc_per_output	8	ENC fee per output (LE u64)
26	max_enc_per_op	8	Max ENC fee per operation (LE u64)
34	max_ops_per_graph	2	Max operations per graph (LE u16)
36	gas_base	8	Base SOL gas fee (LE u64)
44	gas_per_input	8	SOL gas fee per input (LE u64)
52	gas_per_output	8	SOL gas fee per output (LE u64)
60	gas_per_byte	8	SOL gas fee per byte (LE u64)
68	enc_mint	32	ENC SPL token mint address
100	enc_vault	32	ENC vault token account address
132	bump	1	PDA bump

Total: 2 + 131 = 133 bytes

Authority (disc 2)

Authorized operator (executor/decryptor). PDA seeds: ["authority", pubkey] .

Offset	Field	Size	Description
0	discriminator	1	2
1	version	1	1
2	pubkey	32	Authority's public key
34	active	1	Active flag (0 = deactivated)
35	bump	1	PDA bump

Total: 2 + 34 = 36 bytes

DecryptionRequest (disc 3)

Decryption request with result storage. **Keypair account** (not PDA) – no seed conflicts on multiple requests.

Offset	Field	Size	Description
0	discriminator	1	3

Offset	Field	Size	Description
1	version	1	1
2	ciphertext	32	Ciphertext account pubkey
34	ciphertext_digest	32	Digest snapshot at request time
66	requester	32	Who requested decryption
98	fhe_type	1	FHE type (determines result size)
99	total_len	4	Expected result byte count (LE u32)
103	bytes_written	4	Bytes written so far (LE u32)
107	<i>result data</i>	N	Plaintext bytes (N = byte_width of fhe_type)

Total: 2 + 105 + byte_width(fhe_type) bytes

Status is determined by `bytes_written`:

- `0` = pending (decryptor has not responded)
- `== total_len` = complete (result is ready)

EncryptDeposit (disc 4)

Fee deposit for a user. PDA seeds: `["encrypt_deposit", owner]`.

Offset	Field	Size	Description
0	discriminator	1	4
1	version	1	1
2	owner	32	Deposit owner pubkey
34	enc_balance	8	ENC token balance (LE u64)
42	gas_balance	8	SOL gas balance (LE u64)
50	pending_enc_withdrawal	8	Pending ENC withdrawal (LE u64)
58	pending_gas_withdrawal	8	Pending SOL withdrawal (LE u64)
66	withdrawal_epoch	8	Epoch when withdrawal becomes available (LE u64)
74	num_txs	8	Transaction counter (LE u64)
82	bump	1	PDA bump

Total: 2 + 81 = 83 bytes

RegisteredGraph (disc 5)

A reusable computation graph stored on-chain. PDA seeds: ["registered_graph", graph_hash] .

Offset	Field	Size	Description
0	discriminator	1	5
1	version	1	1
2	graph_hash	32	SHA-256 hash of graph data
34	registrar	32	Who registered the graph
66	num_inputs	2	Number of inputs (LE u16)
68	num_outputs	2	Number of outputs (LE u16)
70	num_ops	2	Number of operations (LE u16)
72	finalized	1	Finalized flag
73	bump	1	PDA bump
74	graph_data_len	2	Actual graph data length (LE u16)
76	graph_data	4096	Graph data (padded to max)

Total: 2 + 4170 = 4172 bytes

Maximum graph data: 4096 bytes.

Ciphertext (disc 6)

An encrypted value. **Keypair account** (not PDA) – the account pubkey IS the ciphertext identifier.

Offset	Field	Size	Description
0	discriminator	1	6
1	version	1	1
2	ciphertext_digest	32	Hash of the encrypted blob (zero until committed)
34	authorized	32	Who can use this ([0; 32] = public)
66	network_encryption_public_key	32	FHE key it was encrypted under
98	fhe_type	1	Type discriminant (EBool=0, EUint64=4, etc.)

Offset	Field	Size	Description
99	status	1	Pending(0) or Verified(1)

Total: 2 + 98 = 100 bytes

Status values:

- 0 = PENDING – waiting for executor to commit
- 1 = VERIFIED – digest is valid, ciphertext can be used as input

NetworkEncryptionKey (disc 7)

FHE network public key. PDA seeds: ["network_encryption_key", key_bytes] .

Offset	Field	Size	Description
0	discriminator	1	7
1	version	1	1
2	network_encryption_public_key	32	FHE network public key bytes
34	active	1	Active flag (0 = deactivated)
35	bump	1	PDA bump

Total: 2 + 34 = 36 bytes

Account Type Summary

Account	Disc	Type	Size (bytes)	PDA Seeds
EncryptConfig	1	PDA	133	["encrypt_config"]
Authority	2	PDA	36	["authority", pubkey]
DecryptionRequest	3	Keypair	107 + N	–
EncryptDeposit	4	PDA	83	["encrypt_deposit", owner]
RegisteredGraph	5	PDA	4172	["registered_graph", graph_hash]
Ciphertext	6	Keypair	100	–

Account	Disc	Type	Size (bytes)	PDA Seeds
NetworkEncryptionKey	7	PDA	36	["network_encryption_key_bytes"]

Event Reference

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

The Encrypt program emits 5 event types via Anchor-compatible self-CPI. Each event is prefixed with `EVENT_IX_TAG_LE` followed by a 1-byte event discriminator.

`EVENT_IX_TAG_LE` is `(0x1d9acb512ea545e4u64).to_le_bytes()`, i.e. the wire bytes `e4 45 a5 2e 51 cb 9a 1d`. Anchor's `emit_cpi!` overlaps the first byte of this tag (`0xe4 = 228`) with the program's instruction discriminator for `EmitEvent`, so there is no separate dispatch byte ahead of the tag — `data[0..8]` is the full tag, `data[8]` is the event discriminator.

Event Discriminators

Discriminator	Event
0	CiphertextCreated
1	CiphertextCommitted
2	GraphExecuted
3	DecryptionRequested
4	DecryptionResponded

CiphertextCreated (disc 0)

Emitted when a new ciphertext account is created (`create_input_ciphertext` or `create_plaintext_ciphertext`).

Field	Size	Description
ciphertext	32	Ciphertext account pubkey

Field	Size	Description
ciphertext_digest	32	Initial digest (zero for plaintext, real for input)
fhe_type	1	FHE type discriminant

Data size: 65 bytes

Used by the executor to detect new ciphertexts that need processing (plaintext ciphertexts need encryption and commit).

CiphertextCommitted (disc 1)

Emitted when an authority commits a ciphertext digest (`commit_ciphertext`), transitioning status from PENDING to VERIFIED.

Field	Size	Description
ciphertext	32	Ciphertext account pubkey
ciphertext_digest	32	The committed digest

Data size: 64 bytes

Used by off-chain services to track when ciphertexts become usable as inputs.

GraphExecuted (disc 2)

Emitted when a computation graph is executed (`execute_graph` or `execute_registered_graph`). Output ciphertext accounts are created/updated with status=PENDING.

Field	Size	Description
num_outputs	2	Number of output ciphertexts (LE u16)
num_inputs	2	Number of input ciphertexts (LE u16)
caller_program	32	Program that invoked <code>execute_graph</code> via CPI

Data size: 36 bytes

This is the primary event the executor listens for. Upon detection, the executor:

1. Reads the graph data from the transaction
2. Fetches the input ciphertext blobs

3. Evaluates the computation graph using FHE
 4. Calls `commit_ciphertext` for each output
-

DecryptionRequested (disc 3)

Emitted when a decryption request is created (`request_decryption`).

Field	Size	Description
ciphertext	32	Ciphertext account pubkey
requester	32	Who requested decryption

Data size: 64 bytes

The decryptor listens for this event and:

1. Performs threshold MPC decryption (or mock decryption locally)
 2. Calls `respond_decryption` to write the plaintext result
-

DecryptionResponded (disc 4)

Emitted when the decryptor writes the plaintext result (`respond_decryption`).

Field	Size	Description
ciphertext	32	Ciphertext account pubkey
requester	32	Who requested decryption

Data size: 64 bytes

Off-chain clients listen for this event to know when a decryption result is ready to read.

Event Wire Format

Each event is emitted as a self-CPI instruction with the following data layout:

```
EVENT_IX_TAG_LE(8) | event_discriminator(1) | event_data(N)
```

Total on-wire size per event = 9 + data size.

Event	On-Wire Size
CiphertextCreated	9 + 65 = 74 bytes
CiphertextCommitted	9 + 64 = 73 bytes
GraphExecuted	9 + 36 = 45 bytes
DecryptionRequested	9 + 64 = 73 bytes
DecryptionResponded	9 + 64 = 73 bytes

Parsing Events

Events are emitted as inner instructions in the transaction. To parse them:

1. Find inner instructions targeting the Encrypt program with discriminator `228` (`EmitEvent`)
2. Skip the first 8 bytes (`EVENT_IX_TAG_LE`)
3. Read the 1-byte event discriminator
4. Deserialize the remaining bytes according to the event schema

The `chains/solana/dev` crate provides an event parser for use in tests and off-chain services.

Fee Model

Pre-Alpha Disclaimer: This is an early pre-alpha release for exploring the SDK and starting development only. There is no real encryption — all data is completely public and stored as plaintext on-chain. Do not submit any sensitive or real data. Encryption keys and the trust model are not final; do not rely on any encryption guarantees or key material until mainnet. All interfaces, APIs, and data formats are subject to change without notice. The Solana program and all on-chain data will be wiped periodically and everything will be deleted when we transition to Encrypt Alpha 1. This software is provided “as is” without warranty of any kind; use is entirely at your own risk and dWallet Labs assumes no liability for any damages arising from its use.

Encrypt uses a dual-token fee model: **ENC** (SPL token) for FHE computation costs and **SOL gas** for Solana transaction costs. Fees are charged upfront from the user's `EncryptDeposit` account and partially reimbursed after actual costs are known.

Overview

User creates `EncryptDeposit`

- ENC balance (SPL token transfer to vault)
- Gas balance (SOL transfer to deposit PDA)

`execute_graph` charges:

- ENC: $\text{enc_per_input} \times \text{total_inputs} + \text{enc_per_output} \times \text{outputs} + \text{max_enc_per_op} \times \text{ops}$
- Gas: $\text{gas_base} + \text{gas_per_input} \times \text{inputs} + \text{gas_per_output} \times \text{outputs}$

Authority reimburses $(\text{max_charge} - \text{actual_cost})$ after off-chain evaluation

Fee Parameters

Stored in the `EncryptConfig` account, updatable by authorities via `update_config_fees`:

Parameter	Size	Description
<code>enc_per_input</code>	u64	ENC charged per input (encrypted + plaintext + constant)
<code>enc_per_output</code>	u64	ENC charged per output ciphertext
<code>max_enc_per_op</code>	u64	Maximum ENC charged per FHE operation

Parameter	Size	Description
max_ops_per_graph	u16	Maximum operations allowed per graph
gas_base	u64	Base SOL gas fee per graph execution
gas_per_input	u64	SOL gas fee per input
gas_per_output	u64	SOL gas fee per output
gas_per_byte	u64	SOL gas fee per byte of graph data

ENC Fee Calculation

When `execute_graph` is called, the ENC fee is calculated as:

```
total_inputs = num_inputs + num_plaintext_inputs + num_constants
enc_fee = enc_per_input * total_inputs
         + enc_per_output * num_outputs
         + max_enc_per_op * num_ops
```

The `max_enc_per_op` is a **worst-case** charge. Different FHE operations have vastly different costs (e.g., multiplication is far more expensive than addition). Since the on-chain processor cannot determine actual costs without performing the FHE computation, it charges the maximum. The authority reimburses the difference after off-chain evaluation.

Gas Fee Calculation

SOL gas covers the Solana transaction costs:

```
gas_fee = gas_base
         + gas_per_input * num_inputs
         + gas_per_output * num_outputs
```

Deposit Lifecycle

1. Create Deposit

```
// Instruction: create_deposit (disc 13)
// Data: bump(1) | initial_enc_amount(8) | initial_gas_amount(8)
```

Creates an `EncryptDeposit` PDA for the user. Transfers `initial_enc_amount` ENC tokens from the user's ATA to the program vault, and `initial_gas_amount` lamports as gas.

2. Top Up

```
// Instruction: top_up (disc 14)
// Data: enc_amount(8) | gas_amount(8)
```

Add more ENC and/or SOL to an existing deposit. Either amount can be zero.

3. Use (Automatic)

Every `execute_graph`, `create_input_ciphertext`, `create_plaintext_ciphertext`, and `request_decryption` call deducts fees from the deposit automatically. The deposit account is passed as a writable account in each of these instructions.

4. Reimburse

```
// Instruction: reimburse (disc 17)
// Data: enc_amount(8) | gas_amount(8)
```

After the executor evaluates a computation graph, it knows the actual per-operation costs. The authority calls `reimburse` to credit back the difference between `max_enc_per_op * ops` and the actual cost.

5. Request Withdraw

```
// Instruction: request_withdraw (disc 18)
// Data: enc_amount(8) | gas_amount(8)
```

Requests a withdrawal. Sets `pending_enc_withdrawal`, `pending_gas_withdrawal`, and `withdrawal_epoch = current_epoch + 1`. The withdrawal is delayed by one epoch to prevent front-running.

6. Withdraw

```
// Instruction: withdraw (disc 15)
// No data
```

Executes the pending withdrawal if `current_epoch >= withdrawal_epoch`. Actual amounts are capped at current balances (charges during the delay may have reduced them).

Registered Graph Fee Optimization

When using `execute_registered_graph` instead of `execute_graph`, the authority can compute exact per-operation costs because the graph is known ahead of time. This eliminates the max-charge gap and the need for reimbursement.

```
// Register a graph once
ctx.register_graph(graph_pda, bump, &graph_hash, &graph_data)?;

// Execute with exact fees (no max-charge overcharge)
ctx.execute_registered_graph(graph_pda, ix_data, remaining)?;
```

Fee Example

Given fee parameters:

- `enc_per_input` = 100
- `enc_per_output` = 50
- `max_enc_per_op` = 200
- `gas_base` = 5000
- `gas_per_input` = 1000
- `gas_per_output` = 500

For `cast_vote_graph` (3 inputs, 2 outputs, ~5 ops, 1 constant):

```
ENC upfront = 100 * (3 + 1) + 50 * 2 + 200 * 5 = 400 + 100 + 1000 = 1500
Gas          = 5000 + 1000 * 3 + 500 * 2 = 5000 + 3000 + 1000 = 9000
```

If actual per-op costs total 600 ENC (instead of max 1000), the authority reimburses 400 ENC.

EncryptDeposit Account Fields

Field	Size	Description
owner	32	Deposit owner pubkey
enc_balance	8	Current ENC balance

Field	Size	Description
gas_balance	8	Current SOL gas balance
pending_enc_withdrawal	8	Pending ENC withdrawal amount
pending_gas_withdrawal	8	Pending SOL gas withdrawal amount
withdrawal_epoch	8	Epoch when withdrawal is available
num_txs	8	Total transaction count
bump	1	PDA bump